



МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет экономики, финансов и информационных технологий
Кафедра прикладной математики и экономико-математических методов

ОТЧЁТ

по дисциплине:

«Математическое моделирование»

Направление: 01.03.02 Прикладная математика и информатика

Направленность: Прикладная математика и информатика в экономике
и управлении

Студентка: Палий Анастасия Андреевна

Группа: ПМ-2301

Подпись _____

Проверил Лебедева Людмила Николаевна

Должность к.ф.-м.н., доцент

Оценка _____

Дата _____

Подпись _____

Санкт-Петербург
2025 г.

СОДЕРЖАНИЕ

1. СТАТИЧЕСКИЕ МОДЕЛИ	3
1.1. Производственная функция Кобба-Дугласа	3
1.2. Модель Леонтьева	14
2. АНАЛИЗ ДИНАМИЧЕСКИХ МОДЕЛЕЙ	21
2.1. Модель Солоу	21
2.2. Модели распространения инфекционных заболеваний	29
2.2.1. Модель SIR	29
2.2.2. Модель SEIR	32
2.2.3. Модель SEIRD	34
2.2.4. Модель SEIRD + вакцинация	36
2.3. Модель Лотки-Вольтерра	39
2.4. Модель взаимодействия двух конкурирующих видов	44
2.5. Аттрактор Лоренца	56
3. МОДЕЛИРОВАНИЕ ФИЗИЧЕСКИХ СИСТЕМ	66
3.1. Двойной математический маятник	66
3.2. Связанные пружинные маятники	78
3.3. Маятник Фуко	84
3.4. Симпатические маятники	86
4. ДИНАМИЧЕСКИЕ МОДЕЛИ ЭКОНОМИКИ	89
4.1. Модель Самуэльсона-Хикса	89
4.2. Двухсекторная модель экономики	94
4.2.1. Модель двухсекторной закрытой экономики	94
4.2.2. Взаимодействие двух экономик	97
4.3. Трёхсекторная модель экономики	99
5. МОДЕЛИ С РАСПРЕДЕЛЁННЫМИ ПАРАМЕТРАМИ	104
5.1. Модель распространения тепла в однородном тонком стержне	104
5.1.1. Явная разностная схема. Метод сеток	104
5.1.2. Неявная разностная схема. Метод прогонки	111
5.2. Уравнение колебания струны	119
5.3. Двумерное волновое уравнение. Колебание мембраны	125
6. ОПТИМИЗАЦИОННЫЕ МОДЕЛИ	137
6.1. Оптимальный план производства	137

1. СТАТИЧЕСКИЕ МОДЕЛИ

1.1. Производственная функция Кобба-Дугласа

Производственной функцией называется технологическая связь между выпуском продукции и затратами.

Функция Кобба-Дугласа – это производственная функция вида

$$Y = A \cdot K^{\alpha} \cdot L^{\beta}, \quad A > 0, \quad \alpha, \beta \geq 0, \quad (1)$$

где Y – объём выпущенной продукции;

K – объём основного капитала (основные фонды);

L – затраты труда (численность занятых);

A – технологический коэффициент;

α – эластичность выпуска по капиталу;

β – эластичность выпуска по труду.

Параметрическая идентификация производственной функции Кобба-Дугласа как статической модели макроэкономической системы заключается в определении параметров A, α, β на основе имеющегося набора статистических данных $\{(Y_i, K_i, L_i)\}_{i=1}^N$. Для оценки параметров можно воспользоваться методом наименьших квадратов (МНК).

Для начала приведём ПФ (1) к линейному виду с помощью логарифмирования:

$$\ln Y = \ln A + \alpha \cdot \ln K + \beta \cdot \ln L \quad (2)$$

Для нахождения параметров A, α, β по МНК минимизируется сумма квадратичных отклонений между фактическими и аппроксимированными данными:

$$S(A, \alpha, \beta) = \sum_{i=1}^N (\ln A + \alpha \cdot \ln K_i + \beta \cdot \ln L_i - \ln Y_i)^2 \rightarrow \min \quad (3)$$

Приравнивание частных производных к нулю сводит задачу к решению системы уравнений (4).

$$\begin{cases} \frac{\partial S}{\partial (\ln A)} = 0, \\ \frac{\partial S}{\partial \alpha} = 0, \\ \frac{\partial S}{\partial \beta} = 0. \end{cases} \quad (4)$$

Выполнив частное дифференцирование, преобразуем систему к следующему виду:

$$\begin{cases} N \cdot \ln A + \alpha \cdot \sum_{i=1}^N \ln K_i + \beta \cdot \sum_{i=1}^N \ln L_i = \sum_{i=1}^N \ln Y_i, \\ \ln A \cdot \sum_{i=1}^N \ln K_i + \alpha \cdot \sum_{i=1}^N \ln^2 K_i + \beta \cdot \sum_{i=1}^N \ln K_i \cdot \ln L_i = \sum_{i=1}^N \ln Y_i \cdot \ln K_i, \\ \ln A \cdot \sum_{i=1}^N \ln L_i + \alpha \cdot \sum_{i=1}^N \ln K_i \cdot \ln L_i + \beta \cdot \sum_{i=1}^N \ln^2 L_i = \sum_{i=1}^N \ln Y_i \cdot \ln L_i. \end{cases} \quad (5)$$

Полученную систему линейных алгебраических уравнений можно записать в матричном виде:

$$B \cdot \bar{C} = \bar{Z}, \quad (6)$$

$$\text{где } B = \begin{pmatrix} N & \sum_{i=1}^N \ln K_i & \sum_{i=1}^N \ln L_i \\ \sum_{i=1}^N \ln K_i & \sum_{i=1}^N \ln^2 K_i & \sum_{i=1}^N \ln K_i \cdot \ln L_i \\ \sum_{i=1}^N \ln L_i & \sum_{i=1}^N \ln K_i \cdot \ln L_i & \sum_{i=1}^N \ln^2 L_i \end{pmatrix};$$

$$\bar{C} = \begin{pmatrix} \ln A \\ \alpha \\ \beta \end{pmatrix};$$

$$\bar{Z} = \begin{pmatrix} \sum_{i=1}^N \ln Y_i \\ \sum_{i=1}^N \ln Y_i \cdot \ln K_i \\ \sum_{i=1}^N \ln Y_i \cdot \ln L_i \end{pmatrix}.$$

Из уравнения (6) получаем формулу для вычисления искомого вектора.

$$\bar{C} = B^{-1} \cdot \bar{Z} \quad (7)$$

Частный случай $\alpha + \beta = 1$

Если в уравнении (2) произвести замену $\beta = 1 - \alpha$ и привести подобные слагаемые с учётом свойств логарифма, равенство примет следующий вид:

$$z = a + \alpha \cdot u, \quad (8)$$

где $z = \ln \frac{Y}{L}$;

$a = \ln A$;

$u = \ln \frac{K}{L}$.

Тогда по методу наименьших квадратов получим:

$$S(a, \alpha) = \sum_{i=1}^N (a + \alpha \cdot u_i - z_i)^2 \rightarrow \min \quad (9)$$

Приравнивание частных производных к нулю сводит задачу к решению СЛАУ второго порядка относительно переменных a и α :

$$\begin{cases} N \cdot a + \alpha \cdot \sum_{i=1}^N u_i = \sum_{i=1}^N z_i, \\ a \cdot \sum_{i=1}^N u_i + \alpha \cdot \sum_{i=1}^N u_i^2 = \sum_{i=1}^N z_i \cdot u_i. \end{cases} \quad (10)$$

Полученная система может быть записана в виде матричного уравнения:

$$B \cdot \bar{C} = \bar{Z}, \quad (11)$$

где $B = \begin{pmatrix} N & \sum_{i=1}^N u_i \\ \sum_{i=1}^N u_i & \sum_{i=1}^N u_i^2 \end{pmatrix}$;

$\bar{C} = \begin{pmatrix} a \\ \alpha \end{pmatrix}$;

$\bar{Z} = \begin{pmatrix} \sum_{i=1}^N z_i \\ \sum_{i=1}^N z_i \cdot u_i \end{pmatrix}$.

Тогда искомые параметры могут быть найдены по формуле (7).

Задание

Идентифицировать параметры производственной функции Кобба-Дугласа по статистическим данным о затратах-выпусках. Построить графики ВВП (по историческим данным и по результатам моделирования) и изобразить линию изокванты. Рассмотреть частный случай $\alpha + \beta = 1$.

Решение

Соответствующие расчёты были проведены на основе исторических данных об экономике США в период с 1936 по 1950 год с помощью программных средств Microsoft Excel и Wolfram Mathematica.

1. Решение в Microsoft Excel

1.1. Решение в общем виде

Прежде всего, для упрощения вычисления элементов матрицы коэффициентов B и вектор-столбца правых частей $\bar{Z}$ была создана таблица, приведённая на рисунке ниже.

Экономика США			
Год	ВВП(млн.\$)	K(млн.\$)	L(тыс.чел.)
1936	83278	234236	73426
1937	90884	254890	77568
1938	83743	217606	70460
1939	91530	221746	75131
1940	101313	228757	79694
1941	116415	250238	89276
1942	127434	266469	97056
1943	136274	266154	101633
1944	146470	269520	100124
1945	145052	263098	94920
1946	140288	252357	96671
1947	142022	262536	100072
1948	149895	285700	101304
1949	147122	277522	96784
1950	163620	307946	100352

lnY	lnK	lnL	ln ² K	ln ² L	lnK*lnL	lnY*lnK	lnY*lnL
11,32994	12,36408	11,20403	152,8706	125,5304	138,5276	140,0843	126,941
11,41734	12,44859	11,25891	154,9673	126,7631	140,1575	142,1297	128,5468
11,33551	12,29044	11,1628	151,0549	124,6081	137,1957	139,3184	126,536
11,42442	12,30929	11,22699	151,5186	126,0453	138,1962	140,6265	128,2619
11,52597	12,34042	11,28595	152,2859	127,3727	139,2733	142,2353	130,0815
11,66492	12,43017	11,39949	154,5091	129,9483	141,6975	144,9969	132,9741
11,75535	12,49301	11,48304	156,0754	131,8603	143,4578	146,8598	134,9872
11,82242	12,49183	11,52912	156,0458	132,9207	144,0199	147,6837	136,3022
11,89458	12,5044	11,51416	156,36	132,576	143,9777	148,7345	136,9561
11,88485	12,48028	11,46079	155,7574	131,3497	143,0339	148,3262	136,2097
11,85145	12,4386	11,47907	154,7188	131,769	142,7835	147,4155	136,0436
11,86374	12,47814	11,51365	155,7041	132,564	143,6689	148,0374	136,5949
11,91769	12,5627	11,52588	157,8214	132,8459	144,7962	149,7183	137,3619
11,89902	12,53366	11,48024	157,0925	131,7958	143,8893	149,1382	136,6035
12,0053	12,63768	11,51644	159,7109	132,6284	145,5411	151,7192	138,2583

Рисунок 1 – Предварительные расчёты в Excel

Элементы матрицы B и вектора $\bar{Z}$ были получены путём суммирования элементов соответствующих столбцов согласно полученным ранее формулам. Затем с помощью встроенных функций Excel была найдена обратная матрица B^{-1} и рассчитан вектор $\bar{C}$ как произведение B^{-1} и $\bar{Z}$. Поскольку первый элемент вектора $\bar{C}$ – это $\ln A$, для нахождения искомого параметра A было выполнено потенцирование. На рисунке 2 представлены промежуточные вычисления для решения матричного уравнения (7), а также полученные в результате значения параметров ПФ A, α, β .

B		
15	186,8033	171,0406
186,8033	2326,493	2130,216
171,0406	2130,216	1950,578

B ⁻¹		
1391,734	-158,908	51,50524
-158,908	28,78799	-17,5051
51,50524	-17,5051	14,60135

Z
175,5925
2187,024
2002,659

C
-9,65104
0,358166
1,48182

A	6,44E-05
α	0,358166
β	1,48182

Рисунок 2 – Решение матричного уравнения (6) в Excel

На основе полученных значений параметров A, α, β по формуле (1) были рассчитаны модельные данные, после чего для сравнения реальных данных с аппроксимацией были построены графики, приведённые на рисунке 3.

Год	ВВП (hist)	ВВП (f)
1936	83278	87529,1826
1937	90884	97861,775
1938	83743	80197,7935
1939	91530	88797,7571
1940	101313	97991,4245
1941	116415	119733,648
1942	127434	138600,119
1943	136274	148331,823
1944	146470	145734,535
1945	145052	133494,066
1946	140288	135126,887
1947	142022	144259,46
1948	149895	151415,759
1949	147122	140048,81
1950	163620	153376,289

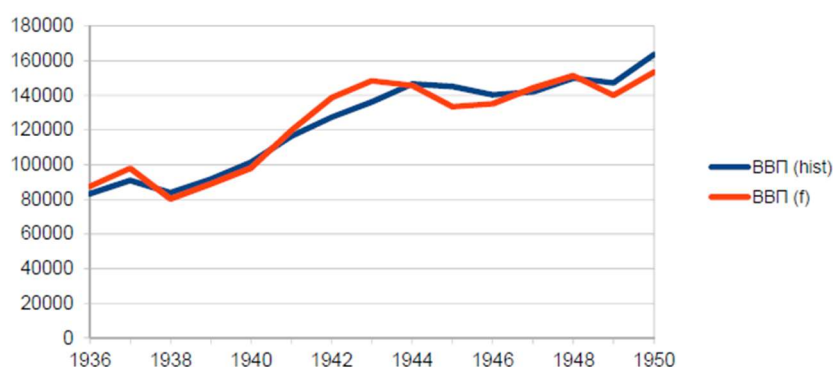


Рисунок 3 – Сравнение исторических данных с аппроксимацией

Для построения *изокванты* – линии равного выпуска ($Y = const$) – формула (1) была приведена к следующему виду:

$$L = \left(\frac{Y}{A \cdot K^\alpha} \right)^{\frac{1}{\beta}} \quad (12)$$

Изокванта $Y = 10^5$, построенная на промежутке $K \in [2 \cdot 10^5; 3,4 \cdot 10^5]$, демонстрируется на рисунке 4.

Изокванта ВВП = 100000	
K	L
200000	83459,2665
210000	82480,8186
220000	81558,5815
230000	80686,9824
240000	79861,2136
250000	79077,101
260000	78330,9985
270000	77619,7045
280000	76940,3931
290000	76290,5586
300000	75667,9697
310000	75070,6313
320000	74496,753
330000	73944,7222
340000	73413,0819

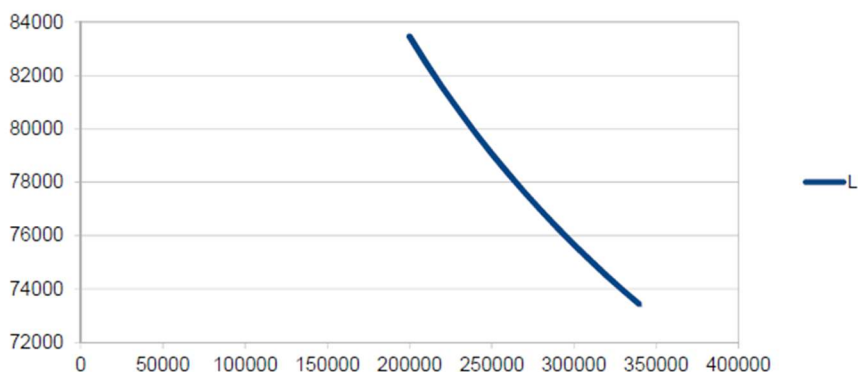


Рисунок 4 – Линия равного выпуска $Y = 10^5$

1.2. Решение для частного случая $\alpha + \beta = 1$

Как и в общем случае, сперва была создана вспомогательная таблица для упрощения расчётов элементов матрицы коэффициентов B и вектор-столбца правых частей $\bar{Z}$ согласно формулам (8) и (11). Полученная таблица приведена на рисунке 5.

z	u	z*u	u ²
0,125906	1,160051	0,146058	1,345718
0,158429	1,189677	0,188479	1,415332
0,172707	1,127641	0,194752	1,271574
0,197434	1,082299	0,213682	1,171372
0,240020	1,054466	0,253093	1,111899
0,265429	1,030680	0,273572	1,062301
0,272310	1,009970	0,275025	1,020039
0,293299	0,962707	0,282361	0,926804
0,380411	0,990233	0,376696	0,980562
0,424058	1,019492	0,432324	1,039364
0,372384	0,959531	0,357314	0,920700
0,350092	0,964498	0,337663	0,930257
0,391809	1,036816	0,406234	1,074988
0,418780	1,053419	0,441151	1,109691
0,488863	1,121240	0,548133	1,257180

Рисунок 5 – Предварительные расчёты в Excel

Затем аналогичным образом были получены искомые параметры A и α . Промежуточные вычисления и значения параметров приведены на рисунке 6.

B	B ⁻¹	Z	C	A
15	15,07950	4,551933	1,115669	0,109454
15,76272	-14,28639	4,726538	-0,772906	α -0,772906
16,63778	13,59511			

Рисунок 6 – Решение матричного уравнения (11) в Excel

Значение параметра α оказалось отрицательным, т. е. вне установленного диапазона (1). Это означает, что построить функцию Кобба-Дугласа невозможно. На рисунке 7 представлен график полученной аппроксимации.

Год	ВВП (hist)	ВВП (f)
1936	83278	91408,331
1937	90884	94378,693
1938	83743	89940,981
1939	91530	99323,933
1940	101313	107647,300
1941	116415	122827,762
1942	127434	135686,269
1943	136274	147371,317
1944	146470	142127,025
1945	145052	131727,023
1946	140288	140520,715
1947	142022	144907,030
1948	149895	138716,630
1949	147122	130837,637
1950	163620	128732,871

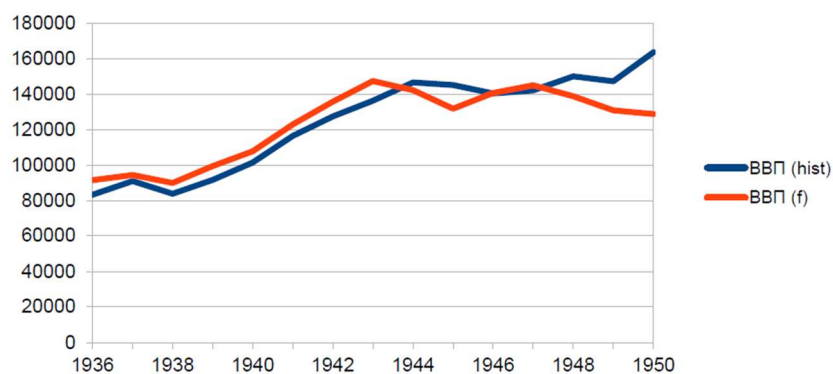


Рисунок 7 – Сравнение исторических данных с аппроксимацией

2. Решение в Wolfram Mathematica

Аналогичные действия были выполнены в среде Wolfram Mathematica.

2.1. Решение в общем виде

Для начала с помощью вспомогательных функций табличные данные были импортированы и приведены в надлежащий вид, как показано на рисунке 8, а также были выполнены промежуточные вычисления.

```
In[*]:= ClearAll[filterTable, path, data, table, table0, table1,
            table2, A,  $\alpha$ ,  $\beta$ , B, iB, Z, V, v, years, hist, theory, isoquant]
```

Вспомогательная функция, чтобы убрать шапку таблицы и пустые строки

```
In[*]:= filterTable[table_List] :=
            Module[{newTable = table, n = Length[table]},
            Select[newTable, NumberQ[#[[1]]] &]];
```

Указываем путь к данным и импортируем их

```
In[*]:= SetDirectory[NotebookDirectory[]];
            path = "data1.xls";
```

```
In[*]:= data = Import[path][[1]];
```

Проводим первичную обработку данных. Таблицу транспонируем для удобства вычислений

```
In[*]:= table = Transpose@filterTable[data];
```

Убираем строку/столбец с годом

```
In[*]:= table0 = table[[{2, 3, 4}]];
```

Вспомогательная таблица со строками/столбцами $\ln Y$, $\ln K$, $\ln L$

```
In[*]:= table1 = Log@table0;
```

Вспомогательная таблица со строками/столбцами $\ln^2 K$, $\ln^2 L$, $\ln K * \ln L$, $\ln Y * \ln K$, $\ln Y * \ln L$

```
In[*]:= table2 = {(table1[[2]])^2, (table1[[3]])^2,
            table1[[2]] * table1[[3]], table1[[1]] * table1[[2]], table1[[1]] * table1[[3]]};
```

```
In[*]:= table2 // Transpose // TableForm
```

Out[*]//TableForm=

152.871	125.53	138.528	140.084	126.941
154.967	126.763	140.158	142.13	128.547
151.055	124.608	137.196	139.318	126.536
151.519	126.045	138.196	140.626	128.262
152.286	127.373	139.273	142.235	130.082
154.509	129.948	141.698	144.997	132.974
156.075	131.86	143.458	146.86	134.987
156.046	132.921	144.02	147.684	136.302
156.36	132.576	143.978	148.735	136.956
155.757	131.35	143.034	148.326	136.21
154.719	131.769	142.784	147.415	136.044
155.704	132.564	143.669	148.037	136.595
157.821	132.846	144.796	149.718	137.362
157.093	131.796	143.889	149.138	136.604
159.711	132.628	145.541	151.719	138.258

Рисунок 8 – Обработка исходных данных в Wolfram Mathematica

Затем были проведены соответствующие расчёты в матричном виде, отражённые на рисунке 9, в результате которых были получены искомые параметры ПФ. Следует отметить, что, как и в случае решения средствами Excel, при вычислении элементов матрицы B была учтена её симметричность относительно главной диагонали.

Составляем матрицу коэффициентов B

$$\text{In[*]:= } B = \begin{pmatrix} \text{Length@First@table2} & \text{Total@table1}[[2]] & \text{Total@table1}[[3]] \\ \text{Total@table1}[[2]] & \text{Total@table2}[[1]] & \text{Total@table2}[[3]] \\ \text{Total@table1}[[3]] & \text{Total@table2}[[3]] & \text{Total@table2}[[2]] \end{pmatrix};$$

Находим обратную матрицу B

$$\text{In[*]:= } \mathbf{iB} = \text{Inverse@B};$$

Составляем вектор-столбец правых частей

$$\text{In[*]:= } Z = \begin{pmatrix} \text{Total@table1}[[1]] \\ \text{Total@table2}[[4]] \\ \text{Total@table2}[[5]] \end{pmatrix};$$

Получаем искомый вектор, умножая обратную матрицу на вектор правых частей

$$\text{In[*]:= } V = \mathbf{iB}.Z;$$

Первый элемент столбца -- $\ln A$. Заменяем его на значение коэффициента A

$$\text{In[*]:= } v = \text{ReplacePart}[V, 1 \rightarrow \text{Exp}[V[[1]]]];$$

$$\text{In[*]:= } v // \text{MatrixForm}$$

$$\text{Out[*]//MatrixForm} = \begin{pmatrix} 0.0000643589 \\ 0.358166 \\ 1.48182 \end{pmatrix}$$

Получаем искомые параметры

$$\text{In[*]:= } A = v[[1, 1]];$$

$$\alpha = v[[2, 1]];$$

$$\beta = v[[3, 1]];$$

Рисунок 9 – Матричные вычисления в Wolfram Mathematica

Для сравнения полученной аппроксимации с исходными статистическими данными были построены графики, которые демонстрируются на рисунке 10.

Строим графики

```

In[*]:= years = IntegerPart@table[[1]];

In[*]:= hist = table[[2]]

In[*]:= theory = A * (table[[3]])α * (table[[4]])β;

In[*]:= ListLinePlot[{Transpose[{years, hist}], Transpose[{years, theory}]],
  PlotLabels → {"hist", "theory"}, AxesLabel → {"year", "GDP"}]

```

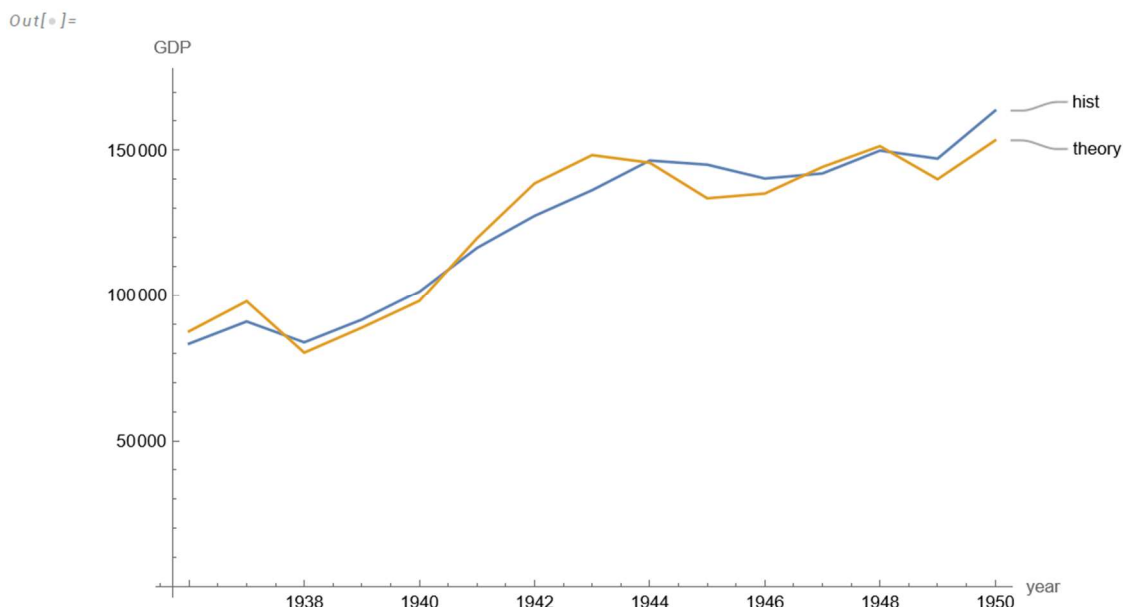


Рисунок 10 – Построение графиков на исторических и модельных данных

Затем была построена изокванта $Y = 10^5$, представленная на рисунке 11.

```

In[*]:= isoquant = Table[{k, (100000 / (A * kα))1/β}, {k, 100000, 400000, 10000}];

In[*]:= ListLinePlot[isoquant, AxesLabel → {"K", "L"}]

```

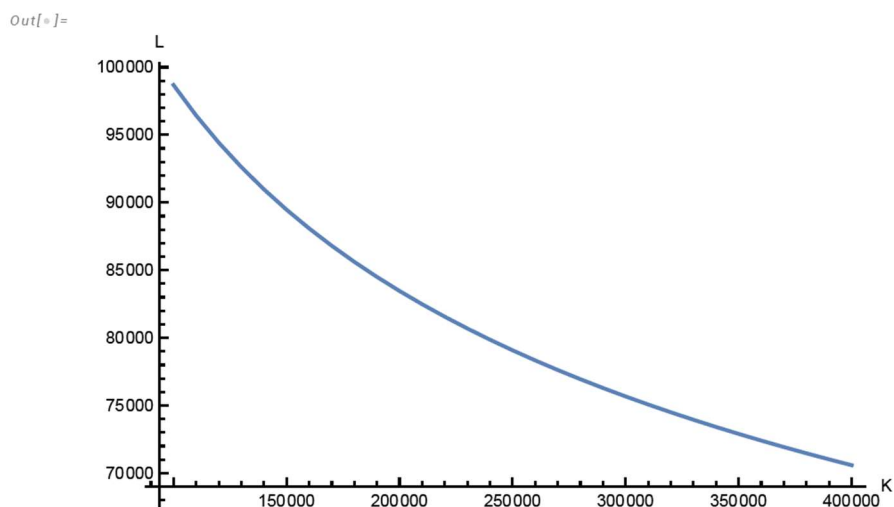


Рисунок 11 – Построение линии равного выпуска $Y = 10^5$

Можно заметить, что K - L -изокванта имеет вид ветви гиперболы.

1.2. Решение для частного случая $\alpha + \beta = 1$

Решение для частного случая $\alpha + \beta = 1$ приведено на рисунке 12.

Вспомогательная таблица со строками/столбцами $z = \ln \frac{Y}{L}$, $u = \ln \frac{K}{L}$.

```
In[*]:= table3 = Log@{table0[[1]], table0[[2]]  
table0[[3]]}, {table0[[3]]},
```

Вспомогательная таблица со строками/столбцами $u^2, u * z$.

```
In[*]:= table4 = {(table3[[2]])^2, table3[[1]] * table3[[2]]};
```

Составляем матрицу коэффициентов B_1

```
In[*]:= B1 = {Length@First@table4 Total@table3[[2]]  
Total@table3[[2]] Total@table4[[1]]};
```

```
In[*]:= B1 // MatrixForm
```

```
Out[*]//MatrixForm=  
( 15 15.7627  
15.7627 16.6378 )
```

Находим обратную матрицу B_1

```
In[*]:= iB1 = Inverse@B1;
```

```
In[*]:= iB1 // MatrixForm
```

```
Out[*]//MatrixForm=  
( 15.0795 -14.2864  
-14.2864 13.5951 )
```

Составляем вектор-столбец правых частей

```
In[*]:= Z1 = {Total@table3[[1]]  
Total@table4[[2]]};
```

```
In[*]:= Z1 // MatrixForm
```

```
Out[*]//MatrixForm=  
( 4.55193  
4.72654 )
```

Получаем искомый вектор, умножая обратную матрицу на вектор правых частей

```
In[*]:= V1 = iB1.Z1;
```

```
In[*]:= V1 // MatrixForm
```

```
Out[*]//MatrixForm=  
( 1.11567  
-0.772906 )
```

Первый элемент столбца -- $\ln A$. Заменяем его на значение коэффициента A

```
In[*]:= v1 = ReplacePart[V1, 1 -> Exp[V1[[1]]];
```

```
In[*]:= v1 // MatrixForm
```

```
Out[*]//MatrixForm=  
( 3.05161  
-0.772906 )
```


На основании полученных результатов можно сделать несколько выводов:

1. Построенная в общем случае функция Кобба-Дугласа достаточно хорошо аппроксимирует данный временной ряд, что видно из приведённых графиков.

2. Поскольку $\alpha + \beta > 1$, производственная функция характеризуется возрастающей отдачей при изменении масштабов производства.

3. Вид изокванты свидетельствует о том, что для поддержания постоянного объёма выпуска в условиях сокращения одного фактора производства потребуется увеличение другого. Аналогично увеличение одного из факторов позволит уменьшить другой с сохранением выпуска на прежнем уровне.

4. Построить функцию Кобба-Дугласа (1), которая аппроксимирует данный временной ряд по методу наименьших квадратов и при этом удовлетворяет условию $\alpha + \beta = 1$, не представляется возможным.

1.2. Модель Леонтьева

Модель Леонтьева (модель межотраслевого баланса) – это статическая линейная модель, характеризующая межотраслевые производственные взаимосвязи в экономике.

Предположения модели:

- в экономике n отраслей;
- каждая отрасль производит единственный вид продукции и потребляет продукцию других отраслей;
- разные отрасли производят разные виды продукции.

Обозначения:

- x_{ij} – объём продукции, произведённой в отрасли i и потребляемой отраслью j ;
- X_i – валовый продукт отрасли i ;
- Y_i – конечный продукт отрасли i .

Стоит отметить, что объёмы можно считать как в натуральном, так и в стоимостном выражении, однако второй вариант предпочтительнее.

Валовый продукт каждой отрасли складывается из объёма продукции, произведённой для межотраслевого потребления, и конечного продукта, что с учётом введённых обозначений выражается формулой:

$$X_i = \sum_{j=1}^n x_{ij} + Y_i, \quad i = 1, \dots, n \quad (13)$$

Поскольку технологии меняются медленно, на большом временном интервале *удельное внутриотраслевое потребление* (a_{ij}) – объём продукции, потребляемый отраслью i для производства единицы продукции другой отрасли j , – остаётся неизменным.

$$a_{ij} = \frac{x_{ij}}{X_j}, \quad i = 1, \dots, n, \quad (14)$$

где a_{ij} – коэффициенты прямых производственных затрат.

Выразив x_{ij} из формулы (14) и подставив в уравнение (13), получим:

$$X_i = \sum_{j=1}^n a_{ij} \cdot X_j + Y_i, \quad i = 1, \dots, n \quad (15)$$

Уравнение (15) можно записать в матричном виде:

$$\bar{X} = A \cdot \bar{X} + \bar{Y}, \quad (16)$$

где $A = \{a_{ij}\}$ – матрица прямых производственных затрат.

Выразим вектор валового выпуска $\bar{X}$:

$$\bar{X} = B \cdot \bar{Y}, \quad (17)$$

где $B = (E - A)^{-1}$ – матрица полных затрат.

Элементы b_{ij} матрицы B отражают количество продукции отрасли i , используемой для производства единицы конечной продукции отрасли j .

В рамках применения теоретических сведений о модели Леонтьева на практике были выполнены две задачи средствами Microsoft Excel и Python.

Задача 1

В таблице приведены данные МОБ в трёхотраслевой экономике. Найти коэффициенты прямых материальных затрат. Вычислить объемы конечного продукта при увеличении валового выпуска каждой отрасли соответственно на 10%, 50% и 20%.

Решение

В таблице ниже приведена исходная таблица варианта 15.

Таблица 1 – Исходные данные задачи 1

Производство	Потребление			Конечный продукт
	А	В	С	
А	50	120	80	60
В	50	180	80	50
С	25	120	40	35

1. Решение в Microsoft Excel

Для начала таблица с исходными данными была дополнена столбцом валового выпуска $\{X_i\}$, рассчитанного по формуле (13). Объёмы валового выпуска по отраслям приведены в соответствующем столбце таблицы 2.

Таблица 2 – Исходная таблица, дополненная столбцом валового выпуска

Производство	Потребление			Конечный продукт	Валовый выпуск
	A	B	C		
A	50	120	80	60	310
B	50	180	80	50	360
C	25	120	40	35	220

Затем по формуле (14) были найдены коэффициенты прямых производственных затрат. Результат представлен в таблице 3.

Таблица 3 – Матрица прямых производственных затрат

Коэффициенты прямых материальных затрат			
	A	B	C
A	0,1612903	0,3870968	0,2580645
B	0,1388889	0,5	0,2222222
C	0,1136364	0,5454545	0,1818182

Поскольку матрица полных затрат остаётся неизменной, из формулы (17) следует, что изменение валового выпуска влечёт пропорциональное изменение конечного продукта. Поэтому для расчёта нового вектора $\bar{Y}$ его компоненты Y_i достаточно умножить на соответствующие коэффициенты согласно условию задачи. При этом объёмы x_{ij} изменятся в аналогичных пропорциях согласно формуле (14). Установившийся в таком случае МОБ отражён в таблице 4.

Таблица 4 – Таблица нового межотраслевого баланса

МОБ после изменения валового выпуска					
Производство	Потребление			Конечный продукт	Валовый выпуск
	A	B	C		
A	55	132	88	66	341
B	75	270	120	75	540
C	30	144	48	42	264

2. Программная реализация на Python

Для решения данной задачи на языке программирования Python были реализованы две функции:

5. Функция `get_A` принимает на вход матрицу $\{x_{ij}\}$ и вектор $\bar{Y}$ и возвращает матрицу прямых производственных затрат A .

6. Функция `get_new_Y` принимает на вход исходный вектор конечного продукта $\bar{Y}$ и относительные величины изменения валового выпуска и возвращает новый вектор $\bar{Y}$.

Исходный код вышеуказанных функций представлен на рисунке 13.

```
import numpy as np

def get_A(x, Y):
    """
    Returns input-output matrix (A)
    :param x: inter-industry matrix (np.array matrix)
    :param Y: external demand vector (np.array column)
    :return: input-output matrix (np.array matrix)
    """
    # X - production level vector (column)
    X = np.array(list(map(lambda Xi, Yi: sum(Xi) + Yi, x, Y)))
    # print('X =\n', X)
    A = x / X
    return A

def get_new_Y(prev_Y, k):
    """
    Returns new external demand vector (Y) after production level vector (X) changed
    :param prev_Y: external demand vector before X changed (np.array column)
    :param k: coefficients of relative change in X[i] (list)
    :return: new external demand vector Y (np.array column)
    """
    new_Y = prev_Y * np.array([k]).transpose()
    return new_Y

x = np.array([[50, 120, 80],
              [50, 180, 80],
              [25, 120, 40]])
Y = np.array([[60, 50, 35]]).transpose()

A = get_A(x, Y)
new_Y = get_new_Y(Y, [1.1, 1.5, 1.2])

print('A =\n', A)
print('new Y =\n', new_Y)
```

Рисунок 13 – Решение задачи 1 в Python

Вывод программы приведён на рисунке 14.

```
A =  
[[0.16129032 0.38709677 0.25806452]  
 [0.13888889 0.5          0.22222222]  
 [0.11363636 0.54545455 0.18181818]]  
new Y =  
[[66.]  
 [75.]  
 [42.]]
```

Рисунок 14 – Ответ задачи 1 в Python

Как видно из Рисунок 14 – выше, вывод совпал с результатами, полученными в Excel (см. таблицы 3 и 4).

Задача 2

По заданной матрице прямых затрат и вектору конечной продукции вычислить валовые выпуски отраслей.

Решение

Ниже приведена таблица 5, содержащая исходные данные варианта 15.

Таблица 5 – Исходные данные задачи 2

Отрасль	Коэффициенты затрат			Конечная продукция Y_i
	1	2	3	
1	0,2	0,3	0,1	240
2	0,3	0,1	0,2	20
3	0,1	0,2	0,3	60

1. Решение в Microsoft Excel

Для нахождения вектора валового выпуска $\{X_i\}$ воспользуемся формулой (17). Из единичной матрицы E соответствующей размерности было произведено вычитание заданной матрицы прямых производственных затрат A . Промежуточные вычисления приведены в таблице 6.

Таблица 6 – Разность единичной матрицы и матрицы прямых затрат

E-A		
0,8	-0,3	-0,1
-0,3	0,9	-0,2
-0,1	-0,2	0,7

Затем с помощью встроенных средств Excel к полученной матрице была найдена обратная, представляющая собой матрицу полных затрат B . Обратная матрица Леонтьева отображена в виде таблицы 7.

Таблица 7 – Матрица полных затрат

$(E-A)^{-1}$		
1,5206186	0,5927835	0,3865979
0,5927835	1,4175258	0,4896907
0,3865979	0,4896907	1,6237113

После чего осталось лишь выполнить матричное умножение полученной матрицы B на заданный вектор конечного продукта $\bar{Y}$. Результат отражён в последнем столбце таблицы 8.

Таблица 8 – Исходная таблица, дополненная столбцом валового выпуска

Отрасль	Коэффициенты затрат			Конечная продукция Y_i	Валовый выпуск
	1	2	3		
1	0,2	0,3	0,1	240	400
2	0,3	0,1	0,2	20	200
3	0,1	0,2	0,3	60	200

2. Программная реализация на Python

Для решения данной задачи на языке программирования Python была реализована функция `get_X`, которая принимает на вход матрицу прямых производственных затрат A и вектор конечного продукта $\bar{Y}$ и возвращает вектор валового выпуска $\bar{X}$.

Исходный код описанной функции представлен на рисунке 15.

```

import numpy as np

def get_X(A, Y):
    """
    Returns production level vector X
    :param A: input-output matrix (np.array matrix)
    :param Y: external demand vector (np.array column)
    :return: production level vector (np.array column)
    """
    # I - Identity matrix
    I = np.eye(max(A.shape))
    # B - Leontief inverse matrix
    B = np.linalg.inv(I-A)
    X = np.dot(B, Y)
    return X

A = np.array([[0.2, 0.3, 0.1],
              [0.3, 0.1, 0.2],
              [0.1, 0.2, 0.3]])
Y = np.array([[240, 20, 60]]).transpose()

X = get_X(A, Y)

print('X =', X.transpose())

```

Рисунок 15 – Решение задачи 2 в Python

Вывод программы приведён на рисунке 16.

```
X = [[400. 200. 200.]
```

Рисунок 16 – Ответ задачи 2 в Python

Как видно из таблицы 8 и рисунка 16, результаты, полученные с помощью Excel и Python, совпали.

Стоит отметить, что несмотря на то, что решения в Excel отличаются большей наглядностью, реализации на языке Python могут быть многократно использованы для решения вышеуказанных задач на входных данных любой размерности, что является важным преимуществом.

2. АНАЛИЗ ДИНАМИЧЕСКИХ МОДЕЛЕЙ

2.1. Модель Солоу

Модель Солоу – первая неоклассическая модель экзогенного экономического роста.

Обозначения:

- Y – валовый внутренний продукт (ВВП);
- K – капитал;
- L – трудовые ресурсы;
- I – инвестиции;
- C – конечное потребление.

Предположения модели:

- зависимость ВВП от ресурсов выражается производственной функцией, удовлетворяющей неоклассическим предпосылкам;
- экономика замкнута, вследствие чего имеет место баланс:

$$Y = C + I \quad (18)$$

- прирост трудовых ресурсов происходит с постоянным темпом:

$$\frac{dL}{dt} = \lambda \cdot L, \quad L_0 = L(0), \quad (19)$$

где λ – темп прироста трудовых ресурсов, $\lambda = const$.

- динамика фондов имеет следующий вид:

$$\frac{dK}{dt} = -\mu \cdot K + I, \quad K_0 = K(0), \quad (20)$$

где μ – норма амортизации – доля фондов, которые выбывают за год, $\mu = const, 0 < \mu < 1$.

- инвестируется фиксированная доля ВВП:

$$I = s \cdot Y, \quad (21)$$

где s – склонность к накоплению, $s = const, 0 < s < 1$.

В качестве примера неоклассической ПФ, удовлетворяющей требованиям модели, будет рассмотрена функция Кобба-Дугласа (1) с показателем однородности 1 ($\alpha + \beta = 1$):

$$Y(K, L) = A \cdot K^\alpha \cdot L^{1-\alpha}, \quad A > 0, \quad 0 < \alpha < 1, \quad (22)$$

Формулы (18) - (22) составляют математическую модель Солоу. Заметим, что $A, \alpha, \lambda, \mu, s$ – постоянные параметры, задаваемые экзогенно.

Для поиска решения модели и повышения информативности удобно перейти к удельным показателям (в расчёте на одного занятого):

$$\begin{cases} y = \frac{Y}{L}, \\ k = \frac{K}{L}, \\ i = \frac{I}{L}. \end{cases} \quad (23)$$

Тогда можно построить модель Солоу в относительных показателях. Динамика фондов на единицу труда k в таком случае будет выражаться следующим образом:

$$\frac{dk}{dt} = -(\lambda + \mu) \cdot k + s \cdot A \cdot k^\alpha \quad (24)$$

Приравнивание $\frac{dk}{dt}$ к нулю позволит найти равновесный режим. Основное уравнение динамики (24) имеет одно глобально устойчивое для $k > 0$ стационарное состояние, которое выражается формулой:

$$\hat{k} = \left(\frac{s \cdot A}{\lambda + \mu} \right)^{\frac{1}{1-\alpha}} \quad (25)$$

В стационарном состоянии капитал и доход на душу населения остаются неизменными. Долгосрочный экономический рост возможен лишь при изменении A . Точка равновесия демонстрируется на рисунке 17.

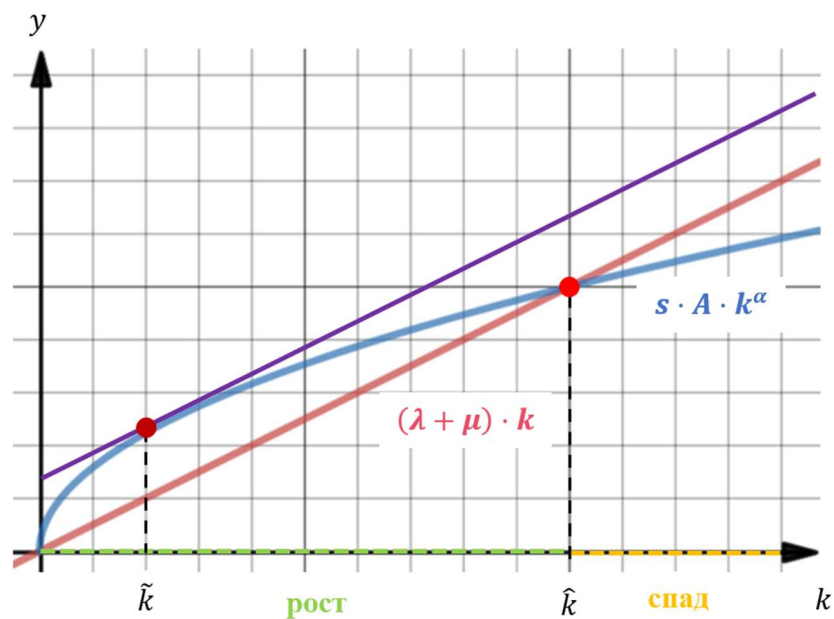


Рисунок 17 – Качественный анализ динамики капиталовооружённости

Основное уравнение динамики (24) представляет собой обыкновенное дифференциальное уравнение первого порядка. Его задача Коши имеет вид:

$$\begin{cases} \frac{dx}{dt} = g(t, x), \\ x(t_0) = x_0. \end{cases} \quad (26)$$

Наиболее распространёнными методами численного решения ОДУ являются метод Эйлера и метод Рунге-Кутты.

Метод Эйлера:

$$x(t + \Delta t) = x(t) + \Delta t \cdot g(t, x) \quad (27)$$

Метод Рунге-Кутты (IV порядка):

$$x(t_i + \Delta t) = x(t_i) + \Delta x(t_i), \quad (28)$$

где $\Delta x(t_i) = \frac{1}{6} \cdot (K_1^i + 2 \cdot K_2^i + 2 \cdot K_3^i + K_4^i)$;

$$K_1^i = \Delta t \cdot g(t_i, x_i);$$

$$K_2^i = \Delta t \cdot g(t_i + \frac{\Delta t}{2}, x_i + \frac{K_1^i}{2});$$

$$K_3^i = \Delta t \cdot g(t_i + \frac{\Delta t}{2}, x_i + \frac{K_2^i}{2});$$

$$K_4^i = \Delta t \cdot g(t_i + \Delta t, x_i + K_3^i).$$

Задание 1

Выполнить численный анализ модели Солоу. Построить интегральные кривые, соответствующие равновесию, росту и спаду. Провести сравнительный анализ результатов численного интегрирования ОДУ методом Эйлера и методом Рунге-Кутты.

Решение

Первым делом на языке программирования Python согласно формулам (27) и (28) были реализованы методы Эйлера и Рунге-Кутты в виде универсальных вспомогательных функций для численного решения ОДУ. Исходный код функций представлен на рисунке 18.

```
import numpy as np

def Euler(g, x0, t0, T, dt):
    time = np.arange(t0, T, dt)
    x = np.empty(time.shape[0])
    x[0] = x0
    for t in range(0, time.shape[0]-1):
        x[t+1] = x[t] + dt * g(t, x[t])

    return time, x

def RK4(g, x0, t0, T, dt):
    time = np.arange(t0, T, dt)
    x = np.empty(time.shape[0])
    x[0] = x0

    for t in range(0, time.shape[0] - 1):
        k1 = dt * g(t, x[t])
        k2 = dt * g(t + dt / 2, x[t] + k1 / 2)
        k3 = dt * g(t + dt / 2, x[t] + k2 / 2)
        k4 = dt * g(t + dt, x[t] + k3)
        x[t + 1] = x[t] + (k1 + 2 * k2 + 2 * k3 + k4) / 6

    return time, x
```

Рисунок 18 – Реализация метода Эйлера и метода Рунге-Кутты

Затем в рамках численного анализа модели Солоу были написаны две вспомогательные функции, одна из которых задаёт функциональную зависимость $\frac{dk}{dt} = g(t, k)$ из уравнения (24), а другая позволяет вычислить точку рав-

новесия по формуле (25). Значения экзогенных параметров были выбраны с учётом их экономического смысла. С помощью реализованных ранее методов численного решения ОДУ были построены интегральные кривые, а также было проведено сравнение результатов численного интегрирования ОДУ методом Эйлера и методом Рунге-Кутты на примере интегральной кривой, соответствующей росту. Исходный код фрагмента программы приведён на рисунке 19.

```
import matplotlib.pyplot as plt
from NSolveDE import RK4, Euler

def get_k0(l, m, A, a, s):
    """
    Calculates the equilibrium point  $k^*$  of the Solow-Swan model
    :param l:  $\lambda$  - labor growth rate
    :param m:  $\mu$  - depreciation rate
    :param A:  $A$  - technological coefficient
    :param a:  $a$  - output elasticity of capital
    :param s:  $s$  - propensity to save
    :return:  $k^*$  - steady-state value of  $k(t)$ 
    """
    return pow(s * A / (l + m), 1 / (1 - a))

def dk(t, k):
    """  $dk/dt = f(t, k)$  """
    return -(l + m) * k + s * A * pow(k, a)

l = 0.001
m = 0.05
A = 1
a = 0.4
s = 0.2

k0 = get_k0(l, m, A, a, s)
print(f' $k^* = {k0}$ ')
t0 = 0
T = 250
step = 0.1

k1 = 0.01
k2 = 20

# Comparison of Euler and RK4 methods
# on the example of an integral curve passing through point (0, k1).
timeline1, euler = Euler(dk, k1, t0, T, step)
timeline2, rk4 = RK4(dk, k1, t0, T, step)
print(max(abs(euler - rk4)))
```

Рисунок 19 – Численный анализ модели Солоу

Значение точки равновесия при заданных параметрах и результат сравнительного анализа решений приведены на рисунке 20.

$$k^* = 9.75213487948481$$

$$0.013415284193425858$$

Рисунок 20 – Вывод программы

Стоит отметить, что такое незначительное отличие кусочно-линейных функций, полученных различными методами численного интегрирования ОДУ, во многом обусловлено малым шагом (рангом разбиения) и большим числом выбранных узлов (точек разбиения), которые влияют на степень приближённости аппроксимации к решению задачи Коши.

Интегральные кривые, построенные по методу Эйлера и методу Рунге-Кутты, демонстрируются на рисунке 21.

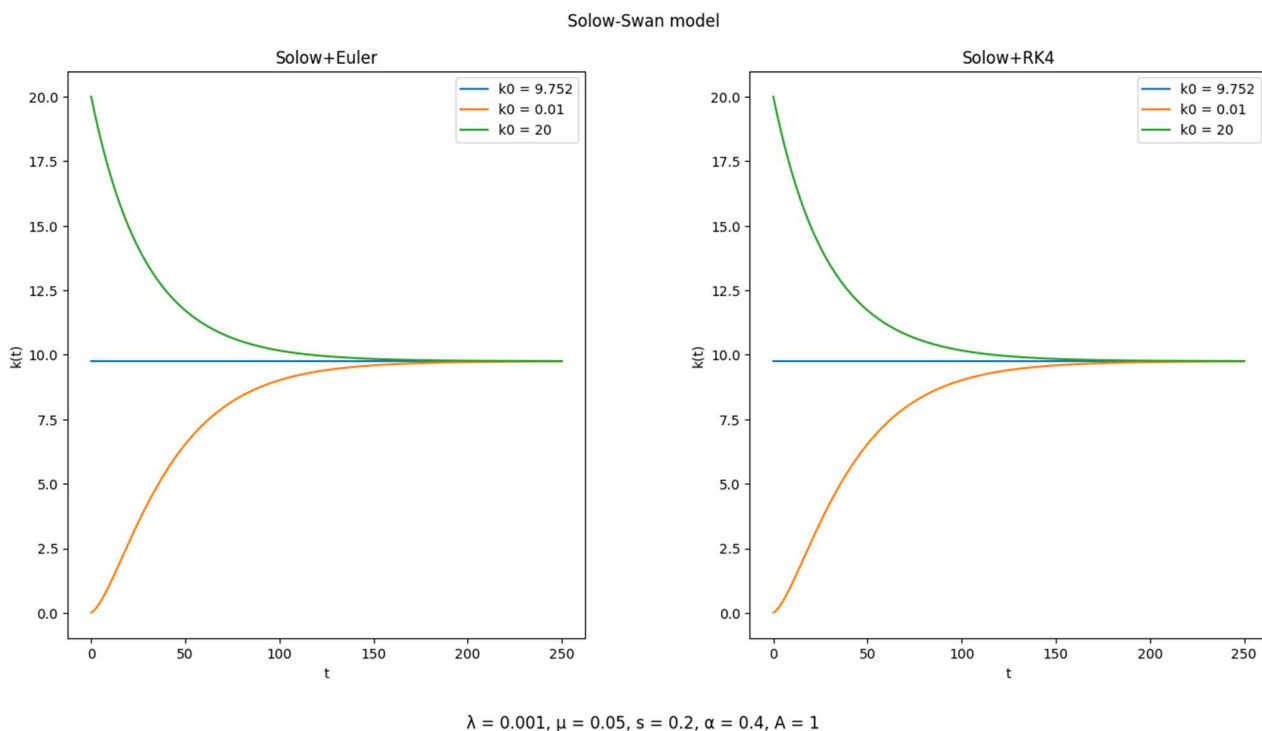


Рисунок 21 – Интегральные кривые, соответствующие росту, спаду и стационарному состоянию

Как видно из графиков, приведённых на рисунке выше, вне зависимости от начального значения капиталовооружённости, с течением времени количество фондов, приходящихся на одного занятого, приходит к стационарному уровню.

Задание 2

Создать имитационную модель Солоу в абсолютных показателях средствами системной динамики AnyLogic.

Реализация

Имитационная модель была построена согласно математической модели Солоу, описываемой формулами (18) - (22). Значения экзогенных параметров были выбраны те же, что и в задании 1. При этом управляющие параметры s и α были сделаны доступными для изменения в режиме реального времени в целях оценки их влияния на динамику. Развитие экономики, описываемой созданной симуляцией, за 100 лет отражено на рисунке 22.

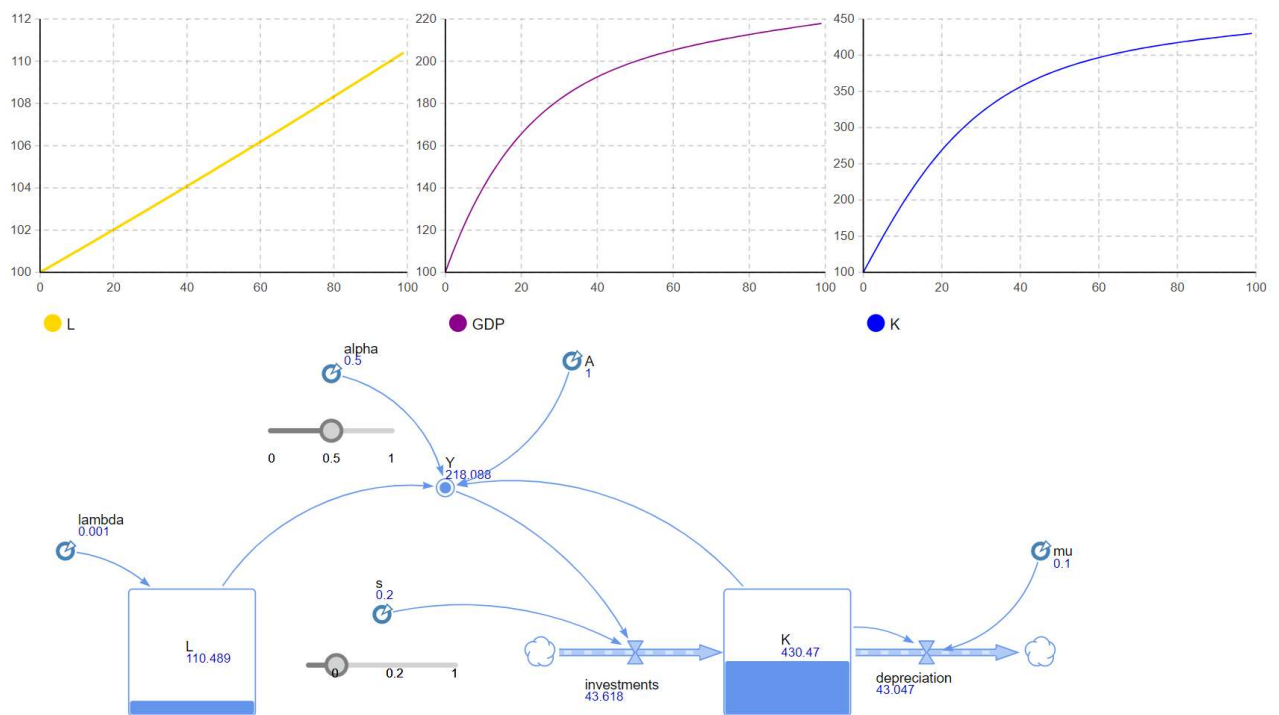


Рисунок 22 – Непрерывная модель Солоу в AnyLogic

Созданная модель реалистично описывает и наглядно демонстрирует развитие экономики, удовлетворяющей описанным ранее предположениям, особенно в долгосрочной перспективе. Кроме того, данная имитационная модель может быть использована для оценки эффективности тех или иных управленческих решений путём варьирования параметров.

Задание 3

Создать имитационную модель Солоу в абсолютных показателях методом дискретно-событийного моделирования в AnyLogic.

Реализация

Для управления изменением состоянием переменных был добавлен элемент «событие», действие которого заключалось в пересчёте значений Y, K, L на каждой итерации с периодичностью, равной единице модельного времени. Динамика $K(t)$ и $L(t)$, заданная формулами (18) - (22), была переписана в дискретной форме:

$$\begin{aligned} L(t+1) &= L(t) + \lambda \cdot L(t), & L(0) &= L_0 \\ K(t+1) &= K(t) - \mu \cdot K(t) + s \cdot Y(t), & K(0) &= K_0 \end{aligned} \quad (29)$$

Результат работы симуляции в течение первых 100 итераций отражён на рисунке 23.

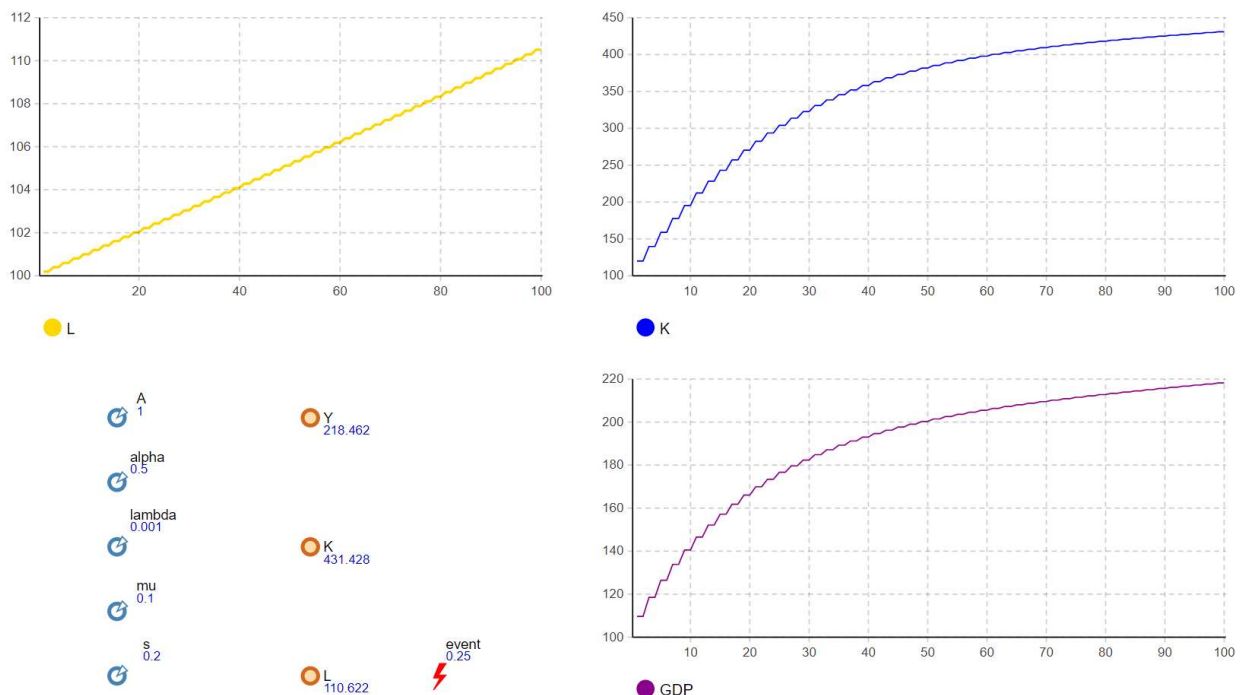


Рисунок 23 – Дискретная модель Солоу в AnyLogic

Можно заметить, что поведение дискретной модели аналогично непрерывной с теми же значениями параметров $A, \alpha, \lambda, \mu, s$.

2.2. Модели распространения инфекционных заболеваний

Рассматриваемые в данном разделе модели относятся к классу *компарментных* моделей (*compartmental*), принцип которых заключается в разделении популяции на изолированные группы (компарменты) согласно установленным признакам и описанию перемещения особей между ними.

Базовой компарментной моделью, используемой для описания динамики распространения эпидемий, является модель SIR.

2.2.1. Модель SIR

Описание и предположения модели:

Существует популяция фиксированной численности. Каждый её член может находиться в одном из трёх состояний: восприимчивый к заражению (*susceptible*), заражённый (*infectious*), выздоровевший (*recovered*). Изначально имеется некоторое количество заражённых особей и большинство восприимчиво к заболеванию, т. е. с заданной вероятностью могут заразиться при контакте с инфицированной особью или внешним источником заражения (при наличии такового). Инфицированные особи спустя некоторое время выздоравливают.

Математическая модель SIR:

$$\begin{aligned} N &= S + I + R \\ \frac{dS}{dt} &= -\alpha \cdot S - \frac{\beta \cdot I \cdot S}{N} \\ \frac{dI}{dt} &= \alpha \cdot S + \frac{\beta \cdot I \cdot S}{N} - \gamma \cdot I \\ \frac{dR}{dt} &= \gamma \cdot I \end{aligned} \tag{30}$$

где N – численность населения (популяции), $N = \text{const}$;

$S(t)$ – численность восприимчивых к инфекции, $S(0) > 0$;

$I(t)$ – численность инфицированных, $I(0) > 0$;

$R(t)$ – численность переболевших;

α – интенсивность заражения от внешнего источника, $0 \leq \alpha < 1$;

β – интенсивность контактов с последующим заражением, $0 < \beta < 1$;

γ – интенсивность выздоровления инфицированных, $0 < \gamma < 1$.

Задание 1

Создать непрерывную имитационную модель распространения эпидемий SIR в AnyLogic.

Реализация

Компартменты S, I, R были реализованы с помощью элементов системной динамики типа «накопитель», а для моделирования перемещения индивидов между ними были использованы потоки с заданными интенсивностями.

На рисунке 24 демонстрируется результат запущенного эксперимента.

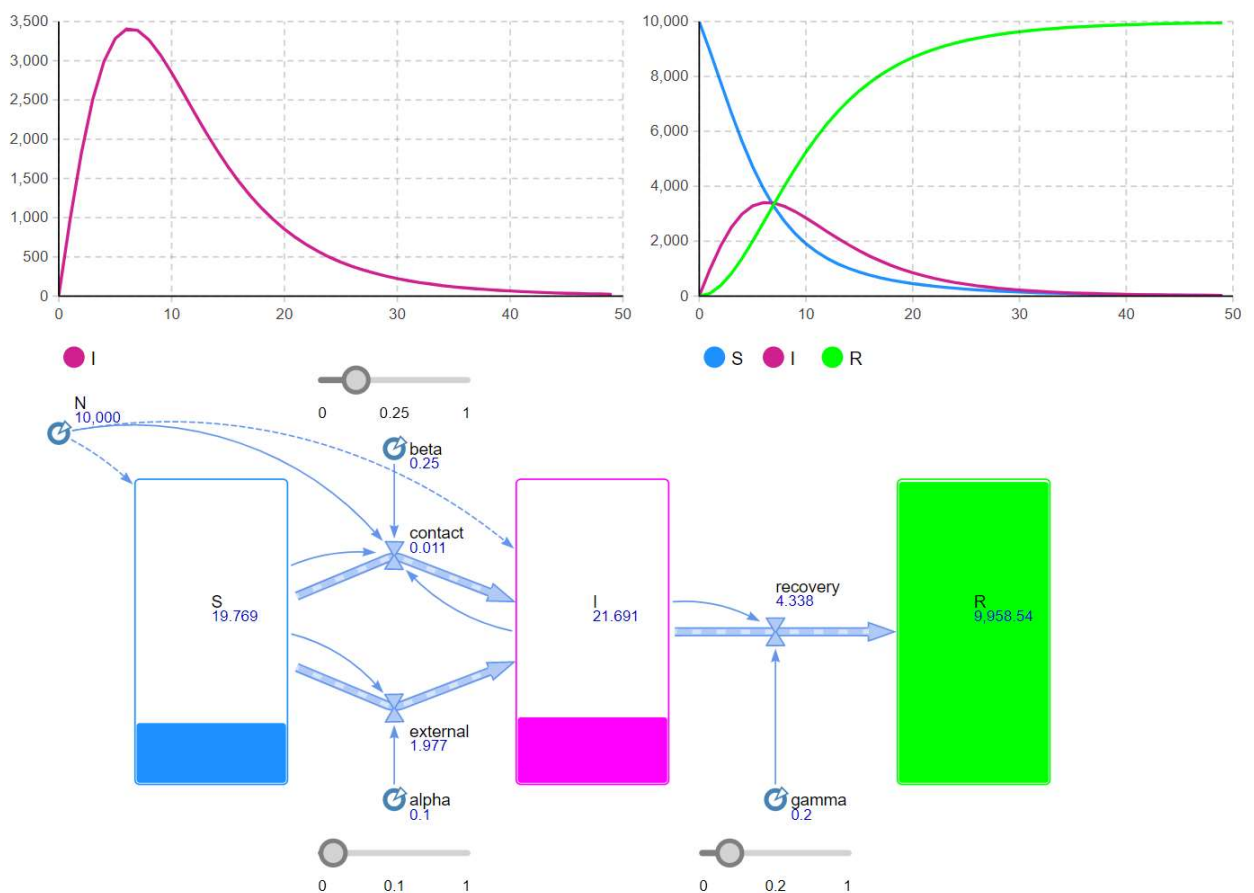


Рисунок 24 – Непрерывная модель SIR в AnyLogic

Как видно из приведённого рисунка, при выбранных значениях параметров N, α, β, γ для того, чтобы практически все индивиды приобрели иммунитет к инфекции вследствие перенесения заболевания, потребовалось 50 дней. При этом на пике эпидемии число заражённых не превысило 35% от общей численности населения.

Задание 2

Создать дискретную имитационную модель SIR в AnyLogic.

Реализация

Компартменты S, I, R были представлены в виде переменных, изменение значений которых происходило под действием циклического события согласно уравнениям динамики в дискретной форме:

$$\begin{aligned} N &= S(t) + I(t) + R(t) \quad \forall t \\ S(t+1) &= S(t) - \alpha \cdot S(t) - \frac{\beta \cdot I(t) \cdot S(t)}{N} \\ I(t+1) &= \alpha \cdot S(t) + \frac{\beta \cdot I(t) \cdot S(t)}{N} - \gamma \cdot I(t) \\ R(t+1) &= R(t) + \gamma \cdot I(t) \end{aligned} \quad (31)$$

При описании действия события было учтено последовательное обновление переменных, в связи с чем первыми рассчитывались новые значения R и I , после чего S находилось из первого уравнения (31). На рисунке 25 демонстрируется результат запущенного эксперимента.

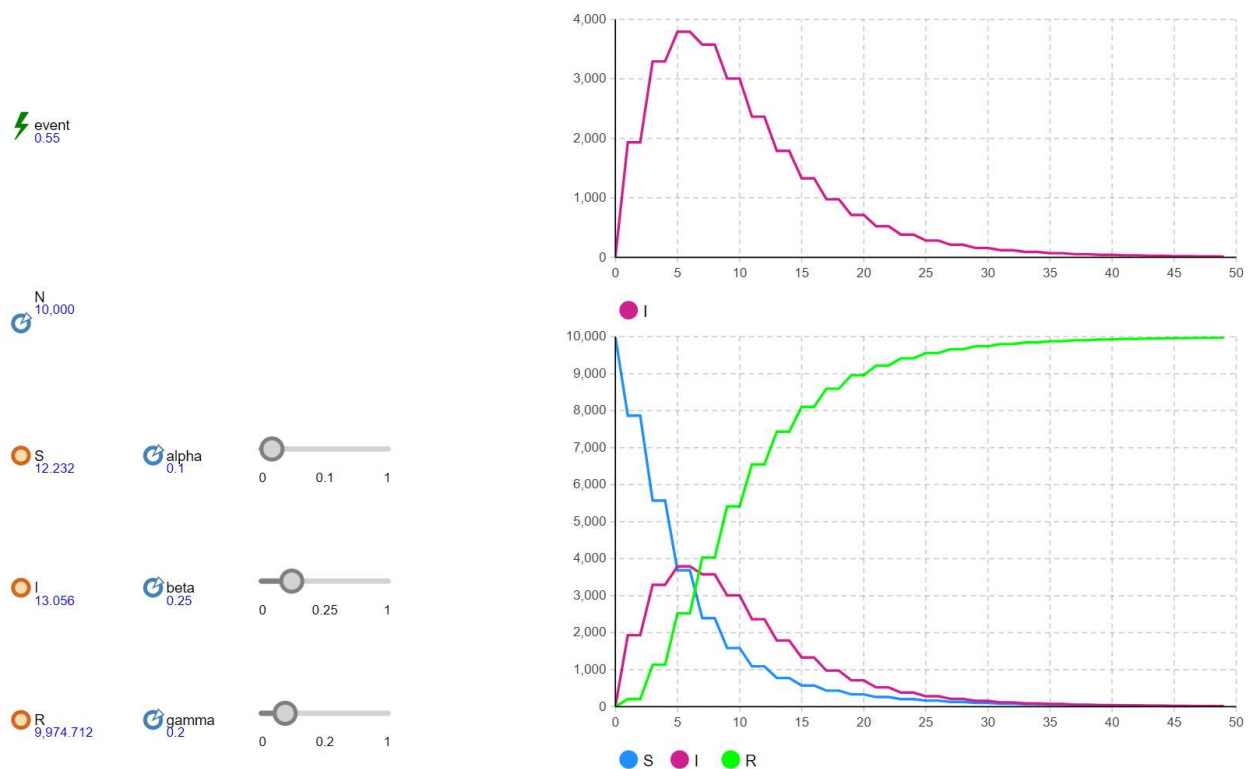


Рисунок 25 – Дискретная модель SIR в AnyLogic

2.2.2. Модель SEIR

Некоторые инфекции характеризуются значительным латентным периодом, в течение которого особи инфицированы, но не заразны. Для моделирования распространения таких заболеваний используется модификация модели SIR, включающая также компартмент E (*exposed*) для латентных, т. е. неактивных заражённых. В данном случае будет рассматриваться только способ передачи инфекции от одной особи к другой ($\alpha = 0$).

Математическая модель SEIR:

$$\begin{aligned} N &= S + E + I + R \\ \frac{dS}{dt} &= -\frac{\beta \cdot I \cdot S}{N} \\ \frac{dE}{dt} &= \frac{\beta \cdot I \cdot S}{N} - \delta \cdot E \\ \frac{dI}{dt} &= \delta \cdot E - \gamma \cdot I \\ \frac{dR}{dt} &= \gamma \cdot I \end{aligned} \tag{32}$$

где N – численность популяции, $N = \text{const}$;

$S(t)$ – численность восприимчивых к инфекции;

$E(t)$ – численность латентных заражённых;

$I(t)$ – численность активных заражённых;

$R(t)$ – численность переболевших;

β – интенсивность контактов с последующим заражением, $0 < \beta < 1$;

δ – интенсивность перехода заражённых из латентной стадии в активную – величина, обратная среднему латентному периоду, $0 < \delta < 1$;

γ – интенсивность выздоровления инфицированных – величина, обратная средней длительности заболевания в активной стадии, $0 < \gamma < 1$.

Задание

Создать непрерывную имитационную модель распространения эпидемий SEIR в AnyLogic.

Реализация

Имитационная модель SEIR была разработана на основе созданной ранее модели SIR. На рисунке 26 представлен результат одного из проведённых экспериментов.

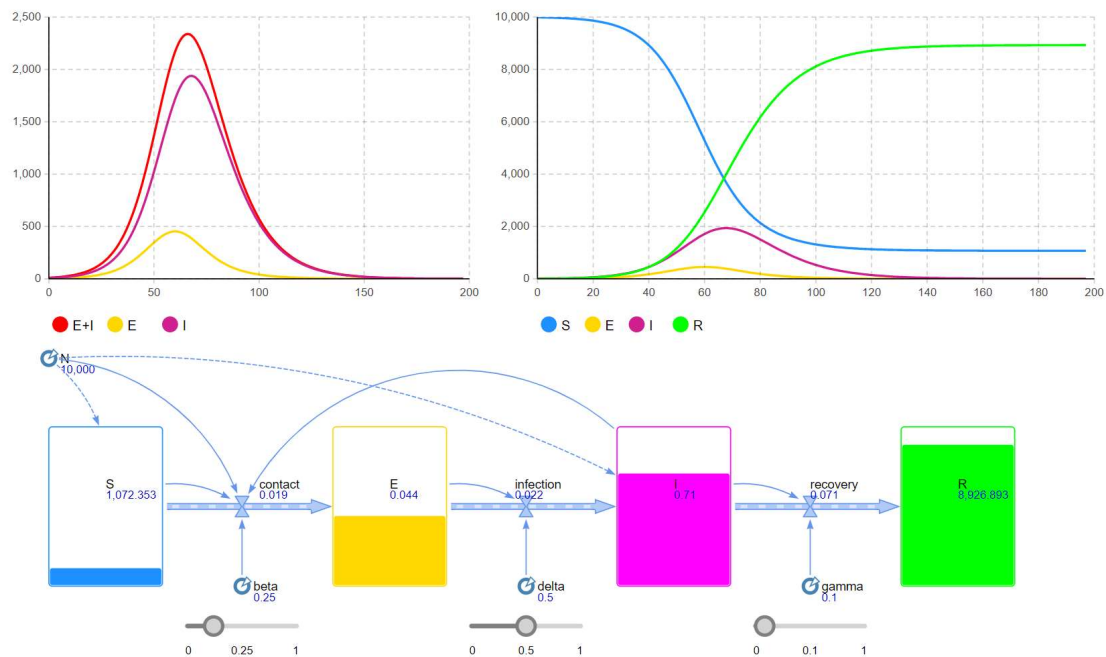


Рисунок 26 – Непрерывная модель SEIR в AnyLogic

Было замечено, что при определённых наборах значений параметров β, δ, γ эпидемия может затронуть совсем незначительную долю популяции и угаснуть. Это характерно, например, для инфекций с невысокой продолжительностью. Тем не менее, поскольку параметр δ целиком определяется свойствами самой инфекции, как и параметр γ , на который может оказать влияние только эффективность существующих методов лечения заболевания, единственным управляющим параметром, способным регулировать развитие эпидемии, остаётся β , который может быть уменьшен путём снижения вероятности контакта заразных особей с восприимчивыми к инфекции.

2.2.3. Модель SEIRD

Для некоторых заболеваний характерен определённый риск летального исхода. Модель SEIRD, включающая также компартмент D (*dead*), позволяет учитывать данное обстоятельство.

Математическая модель SEIRD:

$$\begin{aligned} N &= S + E + I + R + D \\ \frac{dS}{dt} &= -\frac{\beta \cdot I \cdot S}{N - D} \\ \frac{dE}{dt} &= \frac{\beta \cdot I \cdot S}{N - D} - \delta \cdot E \\ \frac{dI}{dt} &= \delta \cdot E - \gamma \cdot I - \mu \cdot I \\ \frac{dR}{dt} &= \gamma \cdot I \\ \frac{dD}{dt} &= \mu \cdot I \end{aligned} \tag{33}$$

где N – численность популяции в начальный момент времени;

$S(t)$ – численность восприимчивых к инфекции;

$E(t)$ – численность латентных заражённых;

$I(t)$ – численность активных заражённых;

$R(t)$ – численность переболевших;

$D(t)$ – численность погибших, $D(0) = 0$;

β – интенсивность контактов с последующим заражением, $0 < \beta < 1$;

δ – интенсивность перехода заражённых из латентной стадии в активную – величина, обратная среднему латентному периоду, $0 < \delta < 1$;

γ – интенсивность выздоровления инфицированных – величина, обратная средней длительности заболевания в активной стадии, $0 < \gamma < 1$;

μ – коэффициент летальности – величина равная отношению числа умерших от инфекции к общему числу подтверждённых диагнозов, $0 < \mu < 1$.

Задание

Создать непрерывную имитационную модель распространения эпидемий SEIRD в AnyLogic.

Реализация

Имитационная модель SEIRD была разработана на основе созданной ранее модели SEIR. На рисунке 27 демонстрируется попытка построения модели на наборе параметров $\{\beta, \delta, \gamma, \mu\}$, близких к оценкам, полученным для COVID-19. Принимая средние длительности латентного периода и активной стадии заболевания равными 5 дням, имеем $\delta = \gamma = 0,2$. Значения коэффициентов $\beta = 0,4$ (до введения ковидных ограничений) и $\mu = 2,64 \cdot 10^{-3}$ были взяты из открытых статистических данных и научно-исследовательских работ. Начальное распределение по компартментам было выбрано по собственному усмотрению.

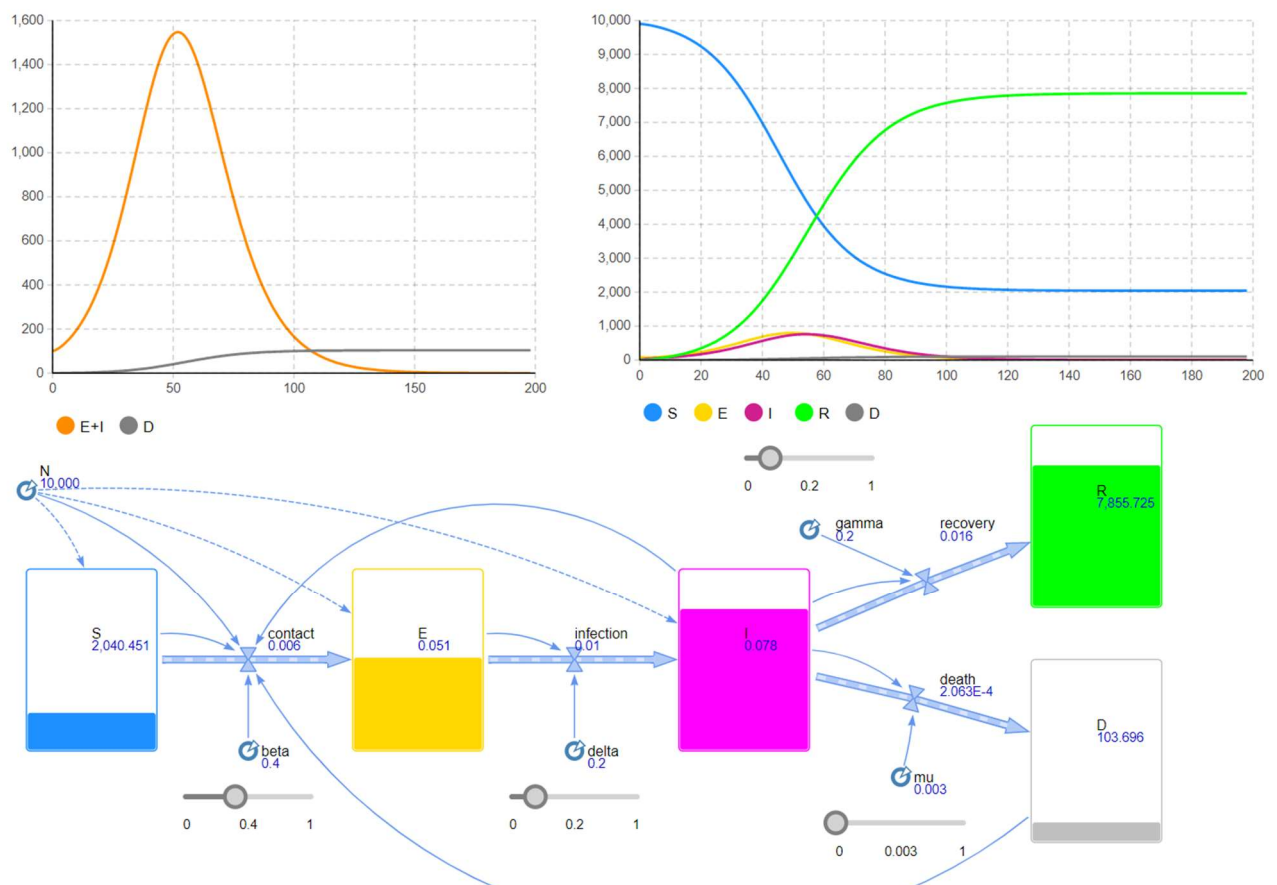


Рисунок 27 – Непрерывная модель SEIRD в AnyLogic

Стоит отметить, что в случае $\beta = 0,3$ (вариант действующих ограничений по распространению инфекции) при остальных неизменных параметрах эпидемия затронет не более 60% населения (и не более 13% при $\beta = 0,2$).

2.2.4. Модель SEIRD + вакцинация

Задание

Построить модификацию модели SEIRD, позволяющую оценить влияние вакцинации восприимчивого населения на развитие эпидемии. Реализовать созданную модель средствами AnyLogic.

Решение

Описание модификации:

Предположим, что вакцинация особей, восприимчивых к инфекции, происходит с постоянной интенсивностью. При этом в результате введения вакцины у особи вырабатывается иммунитет, в результате которого снижается риск инфицирования, а в случае заражения заболевание протекает в более лёгкой форме и характеризуется сниженной контагиозностью (*contagious*). В данной модели будет рассмотрен общий случай, предусматривающий отличия в степени и характере воздействия инфекции на вакцинированный и невакцинированный организмы во всех её ключевых проявлениях. Однако на практике количество расхождений в течении болезни не столь велико, что позволит снизить количество параметров модели.

Математическая модель SEIRD+V:

$$\begin{aligned} N &= S + V + E_S + E_V + I_S + I_V + R + D \\ \frac{dS}{dt} &= -v \cdot S - \frac{(\beta_{SS} \cdot I_S + \beta_{SV} \cdot I_V) \cdot S}{N - D} \\ \frac{dE_S}{dt} &= \frac{(\beta_{SS} \cdot I_S + \beta_{SV} \cdot I_V) \cdot S}{N - D} - \delta_S \cdot E_S \\ \frac{dE_V}{dt} &= \frac{(\beta_{VS} \cdot I_S + \beta_{VV} \cdot I_V) \cdot V}{N - D} - \delta_V \cdot E_V \\ \frac{dI_S}{dt} &= \delta_S \cdot E_S - \gamma_S \cdot I_S - \mu_S \cdot I_S \\ \frac{dI_V}{dt} &= \delta_V \cdot E_V - \gamma_V \cdot I_V - \mu_V \cdot I_V \\ \frac{dR}{dt} &= \gamma_S \cdot I_S + \gamma_V \cdot I_V \\ \frac{dD}{dt} &= \mu_S \cdot I_S + \mu_V \cdot I_V \\ \frac{dV}{dt} &= v \cdot S \end{aligned} \tag{34}$$

где N – численность популяции в начальный момент времени;

$S(t)$ – численность восприимчивых к инфекции;

$V(t)$ – численность вакцинированных неинфицированных;

$E_S(t)$ – численность невакцинированных латентных заражённых;

$E_V(t)$ – численность вакцинированных латентных заражённых;

$I_S(t)$ – численность невакцинированных активных заражённых;

$I_V(t)$ – численность вакцинированных активных заражённых;

$R(t)$ – численность переболевших;

$D(t)$ – численность погибших, $D(0) = 0$;

ν – интенсивность вакцинации, $0 < \nu < 1$;

β_{SS} – вероятность заражения восприимчивой особи при контакте с невакцинированной инфицированной, $0 < \beta_{SS} < 1$;

β_{SV} – вероятность заражения восприимчивой особи при контакте с вакцинированной инфицированной, $0 < \beta_{SV} < 1$;

β_{VS} – вероятность заражения вакцинированной особи при контакте с невакцинированной инфицированной, $0 < \beta_{VS} < 1$;

β_{VV} – вероятность заражения вакцинированной особи при контакте с вакцинированной инфицированной, $0 < \beta_{VV} < 1$;

δ_S – величина, обратная среднему латентному периоду для невакцинированных особей, $0 < \delta_S < 1$;

δ_V – величина, обратная среднему латентному периоду для вакцинированных особей, $0 < \delta_V < 1$;

γ_S – величина, обратная средней длительности заболевания в активной стадии для невакцинированных, $0 < \gamma_S < 1$;

γ_V – величина, обратная средней длительности заболевания в активной стадии для вакцинированных, $0 < \gamma_V < 1$;

μ_S – коэффициент летальности среди невакцинированных, $0 < \mu_S < 1$;

μ_V – коэффициент летальности среди вакцинированных, $0 < \mu_V < 1$.

Полученная модель может быть использована для оценки результата проводимых мероприятий по вакцинированию восприимчивого населения, а также для сравнения эффективности различных вакцин.

Стоит отметить, что данная модель, как и предыдущие, строится на предположении о том, что переболевшие особи не способны заразиться повторно. Кроме того, в большинстве случаев для выработки иммунитета к инфекции в результате вакцинации требуется определённое время, в течение которого заболевание протекает в лёгкой форме. Устранение этих недостатков, а также добавление динамики рождения и смерти позволит получать реалистичные результаты при моделировании распространения более широкого спектра инфекций.

2.3. Модель Лотки-Вольтерра

Модель Лотки-Вольтерра – модель взаимодействия двух видов типа «хищник - жертва».

В математической форме модель описывается системой нелинейных уравнений с двумя переменными:

$$\begin{cases} \frac{dx}{dt} = a \cdot x - b \cdot x \cdot y, \\ \frac{dy}{dt} = -c \cdot y + d \cdot x \cdot y. \end{cases} \quad (35)$$

где $x(t)$ – численность жертв в момент времени t , $x(t) \geq 0$;

$y(t)$ – численность хищников в момент времени t , $y(t) \geq 0$;

a – коэффициент прироста числа жертв, $a > 0$;

b – коэффициент истребления жертв хищником, $b > 0$;

c – коэффициент смертности хищника в условиях нехватки еды, $c > 0$;

d – коэффициент прироста числа хищников за счёт жертв, $d > 0$.

Данная система основана на следующих предположениях:

- в отсутствие хищников жертвы размножаются неограниченно согласно уравнению Мальтуса $\frac{dx}{dt} = a \cdot x$;
- в отсутствие жертв хищники вымирают со скоростью $\frac{dy}{dt} = -c \cdot y$;
- частота встреч хищников и жертв прямо пропорциональна $x \cdot y$;
- коэффициенты при $x \cdot y$ характеризуют превращение биомассы жертв в биомассу хищников.

Система (35) имеет два положения равновесия. Точка $(0, 0)$ (типа седло) не представляет существенного интереса, поскольку означает отсутствие представителей обоих видов. Таким образом, ключевая стационарная точка (типа центр), вокруг которой происходят колебания, определяется по формуле:

$$\begin{cases} x^* = \frac{c}{d}, \\ y^* = \frac{a}{b}. \end{cases} \quad (36)$$

Задание 1

Выполнить анализ модели Лотки-Вольтерра в AnyLogic.

Решение

Динамика численности жертв и хищников была реализована с помощью накопителей согласно формулам (35). Периодические колебания численности видов с течением времени, фазовая кривая и стационарная точка отражены на графиках, приведённых на рисунке 29.

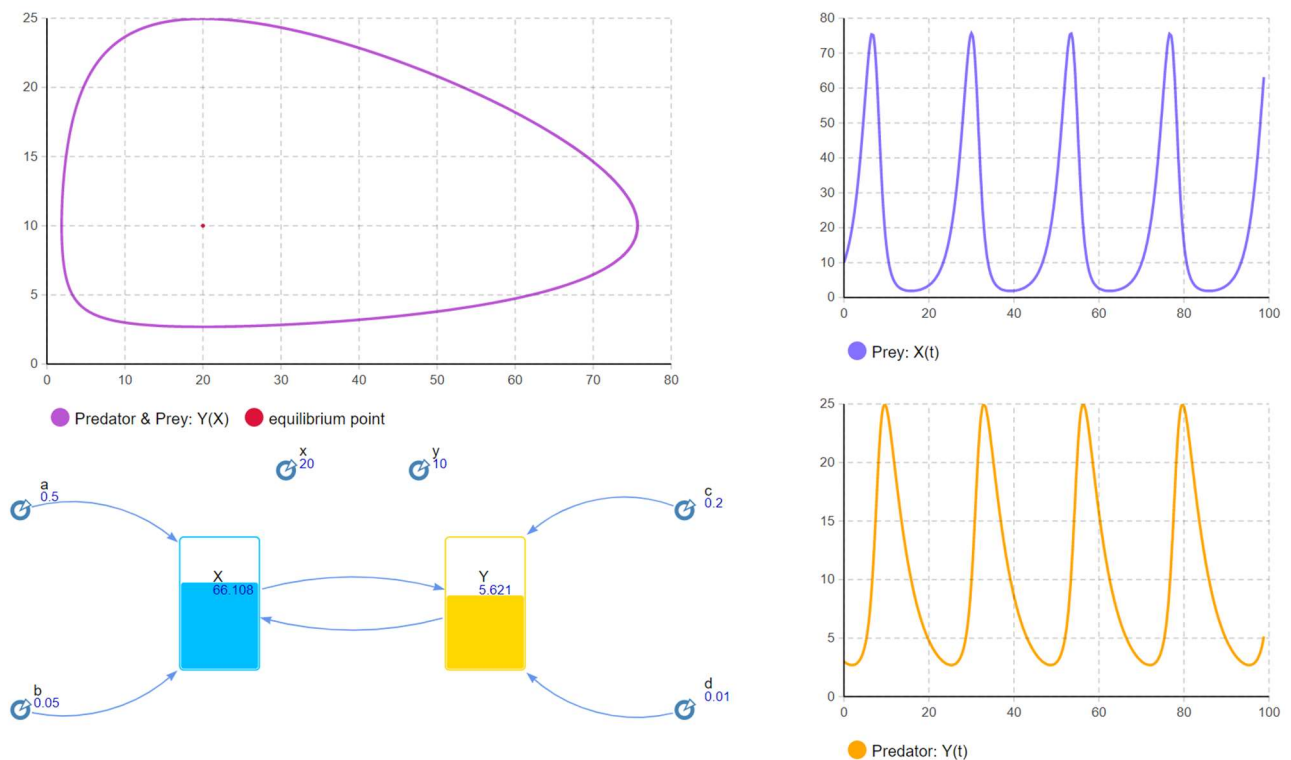


Рисунок 29 – Модель Лотки-Вольтерра в AnyLogic

Многократный запуск симуляции на различных наборах параметров и начальных данных позволяет также отметить неизменные вертикальные изоклины, которые выражаются уравнениями прямых:

$$x = 0$$
$$x = \frac{c}{d}$$

А также горизонтальные изоклины:

$$y = 0$$
$$y = \frac{a}{b}$$

Задание 2

Выполнить численный анализ модели Лотки-Вольтерра в Python.

Решение

На языке программирования Python была написана функция, позволяющая получить массивы данных о численности хищников и жертв в заданные моменты времени с постоянным периодом дискретизации на основе значений параметров модели Лотки-Вольтера и численности обоих видов в начальный момент времени. Уравнения динамики (35) были реализованы в виде вспомогательных функций. Исходный код функций приведён на рисунке 30.

```
import numpy as np
import matplotlib.pyplot as plt
from NSolveDE import RK4_auto_system

def dx(x, y, args):
    """ dx / dt = a * x - b * x * y """
    a = args[0]
    b = args[1]
    return a * x - b * x * y

def dy(x, y, args):
    """ dy / dt = d * x * y - c * y """
    c = args[0]
    d = args[1]
    return d * x * y - c * y

def Lotka_Volterra(a, b, c, d, x0, y0, dt, n: int):
    """ """
    x = np.empty(n)
    y = np.empty(n)
    x[0] = x0
    y[0] = y0

    for t in range(0, n - 1):
        x[t + 1], y[t + 1] = RK4_auto_system(dx, dy, x[t], y[t], dt, [a, b], [c, d])

    return x, y
```

Рисунок 30 – Реализация модели Лотки-Вольтерра в Python

Для численного решения автономной системы ОДУ (35) были написаны модификации методов Эйлера и Рунге-Кутты (IV порядка), как показано на рисунке 31.

```

def Euler_auto_system(f, g, x_t, y_t, dt, f_args, g_args):
    """
    Numerical solving of autonomous DE system by the Euler method
    :param f: function name  $f(x, y, *args) = dx/dt$ 
    :param g: function name  $g(x, y, *args) = dy/dt$ 
    :param x_t:  $x(t)$ 
    :param y_t:  $y(t)$ 
    :param dt:  $t\_step$ 
    :param f_args: additional positional arguments of the function  $f$ 
    :param g_args: additional positional arguments of the function  $g$ 
    :return: tuple  $(x(t+dt), y(t+dt))$ 
    """
    x_next = x_t + dt * f(x_t, y_t, f_args)
    y_next = y_t + dt * g(x_t, y_t, g_args)
    return x_next, y_next

def RK4_auto_system(f, g, x_t, y_t, dt, f_args, g_args):
    """
    . . .
    k1x = dt * f(x_t, y_t, f_args)
    k1y = dt * g(x_t, y_t, g_args)

    k2x = dt * f(x_t + k1x / 2, y_t + k1y / 2, f_args)
    k2y = dt * g(x_t + k1x / 2, y_t + k1y / 2, g_args)

    k3x = dt * f(x_t + k2x / 2, y_t + k2y / 2, f_args)
    k3y = dt * g(x_t + k2x / 2, y_t + k2y / 2, g_args)

    k4x = dt * f(x_t + k3x, y_t + k3y, f_args)
    k4y = dt * g(x_t + k3x, y_t + k3y, g_args)

    x_next = x_t + (k1x + 2 * k2x + 2 * k3x + k4x) / 6
    y_next = y_t + (k1y + 2 * k2y + 2 * k3y + k4y) / 6
    return x_next, y_next

```

Рисунок 31 – Численное решение автономной системы ОДУ методом Эйлера и методом Рунге-Кутты на Python

Значения параметров a, b, c, d были выбраны аналогично рисунку 29. Для получения фазового портрета были построены 6 траекторий, соответствующих различным парам начальных значений численности жертв и хищников. Также согласно формуле (36) было найдено положение равновесия системы. Значения параметров приведены на рисунке 32.

```

a = 0.5
b = 0.05
c = 0.2
d = 0.01

t0 = 0
T = 50
dt = 0.1
time = np.arange(t0, T, dt)

x0 = c/d
y0 = a/b

initial_points = [(100, 10), (10, 10), (5, 15), (30, 10), (5, 1), (20, 9)]

```

Рисунок 32 – Параметры модели и начальные данные

Численное интегрирование проводилось по методу Рунге-Кутты. Периодические колебания численности популяций хищников и жертв, а также построенные траектории на фазовой плоскости демонстрируются на рисунке 33.

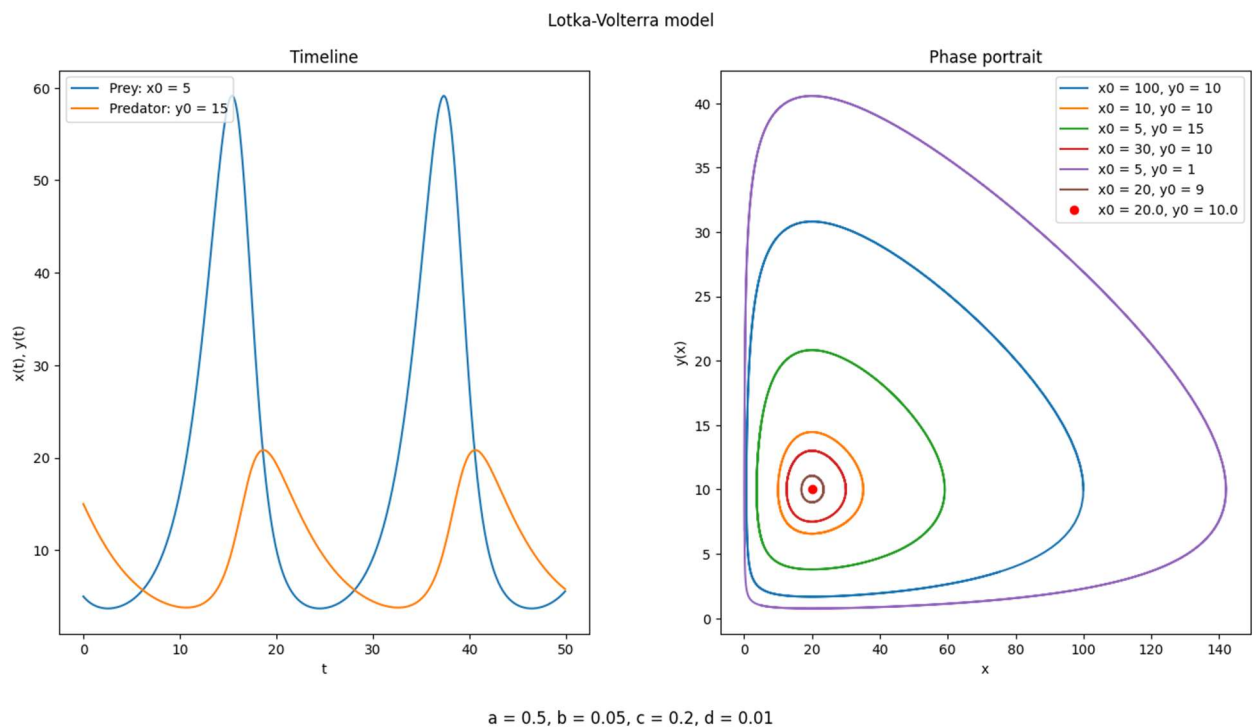


Рисунок 33 – Численный анализ модели Лотки-Вольтерра в Python

Из фазового портрета хорошо видно, что точка покоя имеет тип центр. Также можно заметить, что максимальная и минимальная численность каждого вида достигается при численности другого вида, равной «стационарной» (точки пересечения траектории с соответствующей изоклиной).

2.4. Модель взаимодействия двух конкурирующих видов

Двумерная модель конкуренции – модель, описывающая взаимодействие двух видов в условиях внутривидовой и межвидовой конкуренции.

Математическая модель взаимодействия двух конкурирующих видов выглядит следующим образом:

$$\begin{cases} \frac{dx_1}{dt} = a_1 \cdot x_1 - b_{12} \cdot x_1 \cdot x_2 - c_1 \cdot x_1^2, \\ \frac{dx_2}{dt} = a_2 \cdot x_2 - b_{21} \cdot x_1 \cdot x_2 - c_2 \cdot x_2^2. \end{cases} \quad (37)$$

где $x_1(t)$ – численность первого вида в момент времени t , $x_1(t) \geq 0$;

$x_2(t)$ – численность второго вида в момент времени t , $x_2(t) \geq 0$;

a_1, a_2 – коэффициенты собственной скорости роста видов, $a_1, a_2 > 0$;

b_{12}, b_{21} – коэффициенты межвидовой конкуренции, $b_{12}, b_{21} > 0$;

c_1, c_2 – коэффициенты внутривидовой конкуренции, $c_1, c_2 > 0$.

Задание 1

Выполнить качественный анализ двумерной модели конкуренции.

Решение

С помощью средств Wolfram Mathematica были найдены положения равновесия и матрица Якоби, как показано на рисунке 34.

```
In[*]:= G = {a1 x1 - b12 x1 x2 - c1 x1^2,
             a2 x2 - b21 x1 x2 - c2 x2^2};

In[*]:= points = Solve[G == 0, {x1, x2}]

Out[*]=
{{x1 -> 0, x2 -> 0}, {x1 -> 0, x2 -> a2/c2},
 {x1 -> - (a2 b12 + a1 c2) / (b12 b21 - c1 c2), x2 -> - (a1 b21 - a2 c1) / (-b12 b21 + c1 c2)}, {x1 -> a1/c1, x2 -> 0}}

In[*]:= A = Grad[G, {x1, x2}];
A // MatrixForm

Out[*]//MatrixForm=
( a1 - 2 c1 x1 - b12 x2      -b12 x1
  -b21 x2      a2 - b21 x1 - 2 c2 x2 )
```

Рисунок 34 – Положения равновесия системы (37) и матрица коэффициентов

Типы точек покоя, лежащих на осях, были определены путём оценки собственных чисел матриц коэффициентов линеаризованной системы в окрестностях этих точек, как показано на рисунке 35.

In[]:=* $A_1 = A /. \{x_1 \rightarrow 0, x_2 \rightarrow 0\};$

In[]:=* **Eigenvalues@A₁**

Out[]=*

$\{a_1, a_2\}$

In[]:=* **Eigenvectors@A₁**

Out[]=*

$\{\{1, 0\}, \{0, 1\}\}$

Вывод: (0, 0) - неустойчивый узел

In[]:=* $A_2 = A /. \{x_1 \rightarrow \frac{a_1}{c_1}, x_2 \rightarrow 0\};$

In[]:=* **Eigenvalues@A₂**

Out[]=*

$\left\{-a_1, \frac{-a_1 b_{21} + a_2 c_1}{c_1}\right\}$

In[]:=* **Eigenvectors@A₂**

Out[]=*

$\left\{\{1, 0\}, \left\{\frac{a_1 b_{12}}{a_1 b_{21} - a_1 c_1 - a_2 c_1}, 1\right\}\right\}$

Вывод: $\left(\frac{a_1}{c_1}, 0\right)$ - устойчивый узел при $a_1 b_{21} > a_2 c_1$ (1)

- седло при $a_1 b_{21} < a_2 c_1$ (2)

In[]:=* $A_3 = A /. \{x_1 \rightarrow 0, x_2 \rightarrow \frac{a_2}{c_2}\};$

In[]:=* **Eigenvalues@A₃**

Out[]=*

$\left\{-a_2, \frac{-a_2 b_{12} + a_1 c_2}{c_2}\right\}$

In[]:=* **Eigenvectors@A₃**

Out[]=*

$\left\{\{0, 1\}, \left\{-\frac{-a_2 b_{12} + a_1 c_2 + a_2 c_2}{a_2 b_{21}}, 1\right\}\right\}$

Вывод: $\left(0, \frac{a_2}{c_2}\right)$ - устойчивый узел при $a_2 b_{12} > a_1 c_2$ (3)

- седло при $a_2 b_{12} < a_1 c_2$ (4)

Рисунок 35 – Анализ стационарных точек, лежащих на осях

Для установления типа четвёртого положения равновесия были произведены оценки знака определителя и следа матрицы коэффициентов, а также дискриминанта характеристического уравнения. Результаты отражены на рисунке 36.

$$In[*]:= A_4 = A /. \left\{ x_1 \rightarrow \frac{a_2 b_{12} - a_1 c_2}{b_{12} b_{21} - c_1 c_2}, x_2 \rightarrow \frac{a_1 b_{21} - a_2 c_1}{b_{12} b_{21} - c_1 c_2} \right\};$$

При каких условиях $x_1^*, x_2^* > 0$?

(I): $(3) \wedge (1)$, т. к. $(3) * (1) \Rightarrow b_{12} b_{21} > c_1 c_2$

(II): $(4) \wedge (2)$, т. к. $(4) * (2) \Rightarrow b_{12} b_{21} < c_1 c_2$

$In[*]:= det = Simplify@Det@A_4$

$$Out[*]= \frac{(a_1 b_{21} - a_2 c_1) (-a_2 b_{12} + a_1 c_2)}{b_{12} b_{21} - c_1 c_2}$$

(I): $\det A_4 < 0$

(II): $\det A_4 > 0$

$In[*]:= trace = Simplify@Tr@A_4$

$$Out[*]= \frac{a_1 (-b_{21} + c_1) c_2 + a_2 c_1 (-b_{12} + c_2)}{b_{12} b_{21} - c_1 c_2}$$

$$trace = \frac{c_1 (a_1 c_2 - a_2 b_{12}) + c_2 (a_2 c_1 - a_1 b_{21})}{b_{12} b_{21} - c_1 c_2}$$

(II): $\text{tr } A_4 < 0$

$In[*]:= Reduce[trace^2 - 4 * det < 0 \&\&$
 $a_1 > 0 \&\& a_2 > 0 \&\& b_{12} > 0 \&\& b_{21} > 0 \&\& c_1 > 0 \&\& c_2 > 0 \&\&$
 $a_1 b_{21} < a_2 c_1 \&\& a_2 b_{12} < a_1 c_2,$
 $\{a_1, a_2, b_{12}, b_{21}, c_1, c_2\}]$

$Out[*]=$
False

$In[*]:= Reduce[trace^2 - 4 * det > 0 \&\&$
 $a_1 > 0 \&\& a_2 > 0 \&\& b_{12} > 0 \&\& b_{21} > 0 \&\& c_1 > 0 \&\& c_2 > 0 \&\&$
 $a_1 b_{21} < a_2 c_1 \&\& a_2 b_{12} < a_1 c_2,$
 $\{a_1, a_2, b_{12}, b_{21}, c_1, c_2\}]$

$$Out[*]= a_1 > 0 \&\& a_2 > 0 \&\& b_{12} > 0 \&\& b_{21} > 0 \&\& c_1 > \frac{a_1 b_{21}}{a_2} \&\& c_2 > \frac{a_2 b_{12}}{a_1}$$

(II): $(\text{tr } A_4)^2 - 4 \det A_4 > 0$

Вывод: $\left(\frac{a_2 b_{12} - a_1 c_2}{b_{12} b_{21} - c_1 c_2}, \frac{a_1 b_{21} - a_2 c_1}{b_{12} b_{21} - c_1 c_2} \right)$ - седло при $a_1 b_{21} > a_2 c_1$ (1) $\wedge$ $a_2 b_{12} > a_1 c_2$ (3)
- устойчивый узел при $a_1 b_{21} < a_2 c_1$ (2) $\wedge$ $a_2 b_{12} < a_1 c_2$ (4)

Рисунок 36 – Анализ четвёртой стационарной точки

Варианты фазового портрета, при которых все стационарные точки лежат в первой четверти, приведены в таблице 9.

Таблица 9 – Варианты фазового портрета с четырьмя точками покоя

Стационарная точка	Тип точки и характер устойчивости	
	при $\begin{cases} a_1 \cdot b_{21} > a_2 \cdot c_1, \\ a_2 \cdot b_{12} > a_1 \cdot c_2. \end{cases}$	при $\begin{cases} a_1 \cdot b_{21} < a_2 \cdot c_1, \\ a_2 \cdot b_{12} < a_1 \cdot c_2. \end{cases}$
$(0, 0)$	неустойчивый узел	неустойчивый узел
$\left(\frac{a_1}{c_1}, 0\right)$	устойчивый узел	седло
$\left(0, \frac{a_2}{c_2}\right)$	устойчивый узел	седло
$\left(\frac{a_2 \cdot b_{12} - a_1 \cdot c_2}{b_{12} \cdot b_{21} - c_1 \cdot c_2}, \frac{a_1 \cdot b_{21} - a_2 \cdot c_1}{b_{12} \cdot b_{21} - c_1 \cdot c_2}\right)$	седло	устойчивый узел

На рисунке 37 демонстрируется вид фазового портрета системы в описанных случаях. Графики получены с помощью функции StreamPlot в Wolfram.

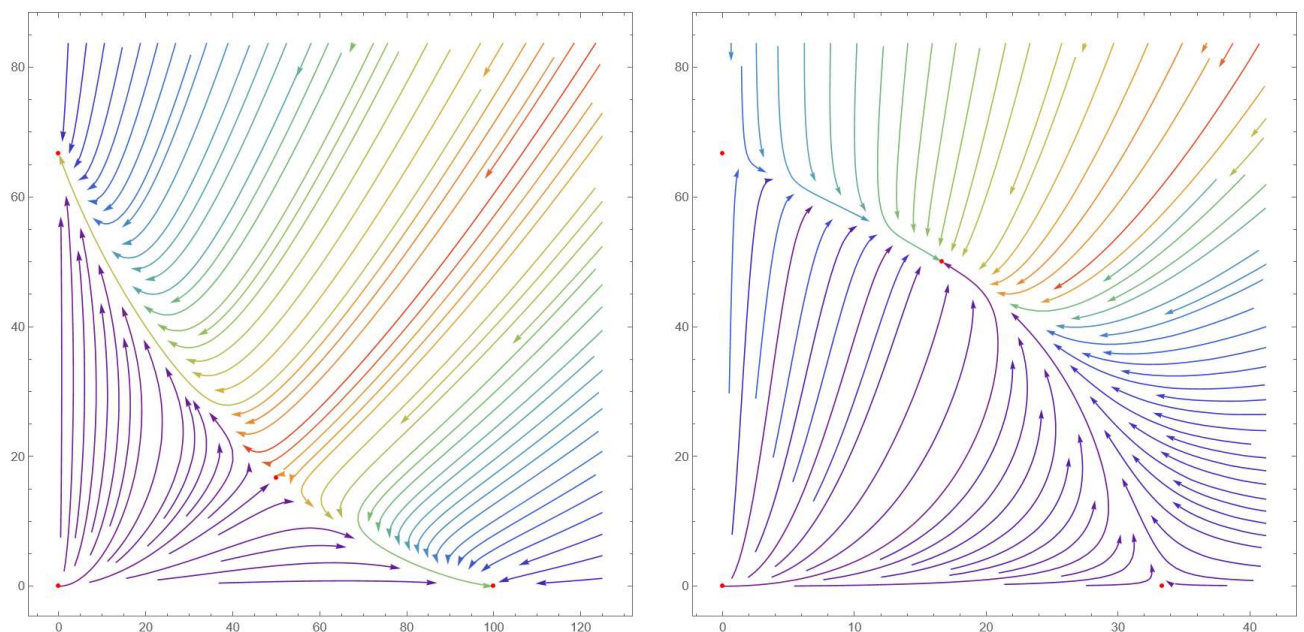


Рисунок 37 – Фазовые портреты системы, включающие четыре особые точки

При других соотношениях параметров в области допустимых решений системы будут находиться три положения равновесия. Тип каждой точки покоя будет определяться из условий, представленных на рисунке 35 (в случае невырожденной линеаризованной матрицы). Вариации представлены на рисунке 38.

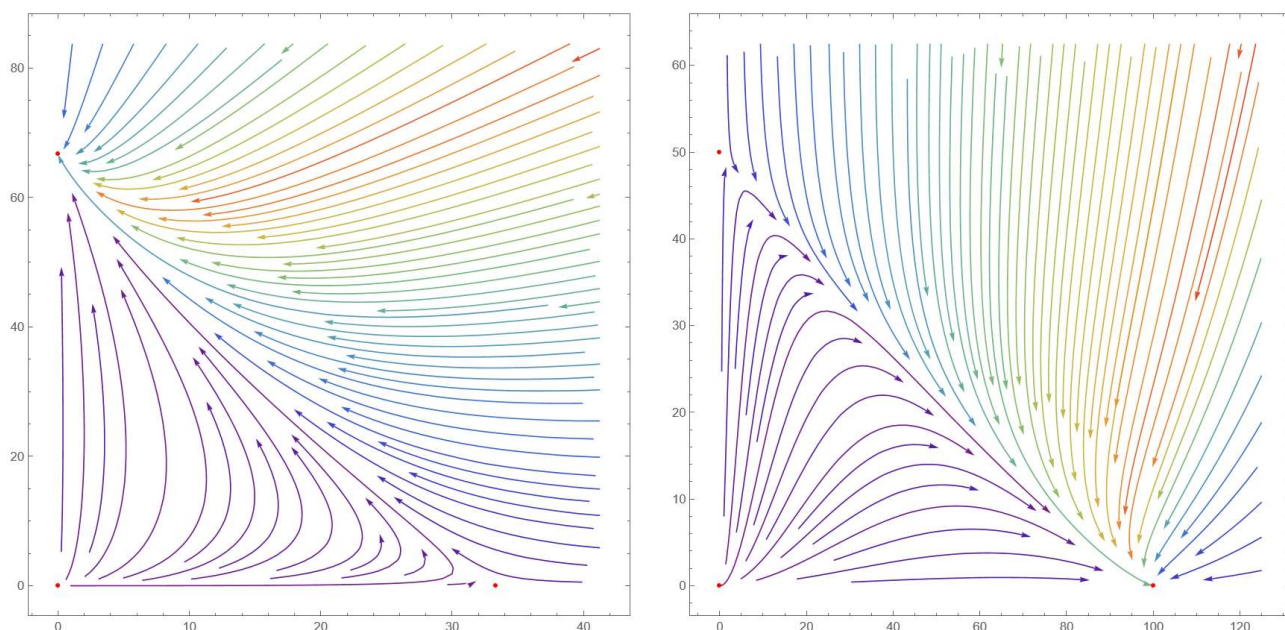


Рисунок 38 – Фазовые портреты системы, включающие три особые точки

Случаи $a_1 \cdot b_{21} = a_2 \cdot c_1$ и $a_2 \cdot b_{12} = a_1 \cdot c_2$, требуют отдельного рассмотрения, поскольку тогда матрица линеаризации некоторых особых точек имеет отрицательное и нулевое собственные значения. Для линейной системы это означало бы бесконечное множество устойчивых точек покоя, лежащих на прямой, однако будет неверно для исходной нелинейной. Фазовые портреты системы (37) при данных соотношениях параметров приведены на рисунке 39.

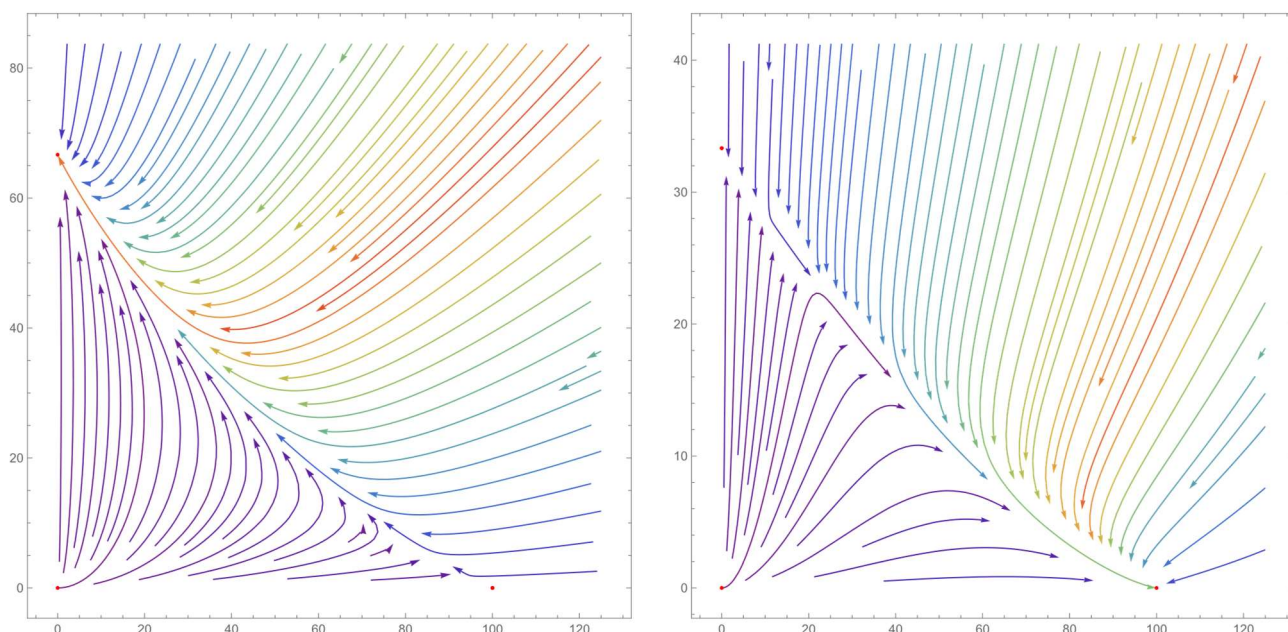


Рисунок 39 – Фазовые портреты системы при условиях
 $a_1 \cdot b_{21} = a_2 \cdot c_1$ (слева) и $a_2 \cdot b_{12} = a_1 \cdot c_2$ (справа)

Задание 2

Выполнить численный анализ двумерной модели конкуренции.

Решение

Для численного анализа модели средствами библиотеки `matplotlib` на различных наборах параметров $\{a_1, a_2, b_{12}, b_{21}, c_1, c_2\}$ были построены векторные поля, отражающие фазовый портрет системы. Также с помощью реализованной ранее функции для численного решения автономной системы ОДУ (см. Рисунок 31 –) для различных пар начальных значений $\{x_1(0), x_2(0)\}$ были получены интегральные кривые, построенные в осях $x_1(t)$, $x_2(t)$ и на фазовой плоскости. Стоит отметить, что для численного анализа были выбраны те же наборы параметров, которые были использованы ранее для построения графиков, приведённых на рисунках 37 - 39. Начальные точки для построения интегральных кривых в фазовой плоскости были выбраны равномерно.

Первыми среди ключевых вариантов фазового портрета были рассмотрены случаи, описанные в таблице 9. На рисунке 40 приведены графики, иллюстрирующие поведение динамической системы при соотношении параметров $a_1 \cdot b_{21} > a_2 \cdot c_1$ и $a_2 \cdot b_{12} > a_1 \cdot c_2$.

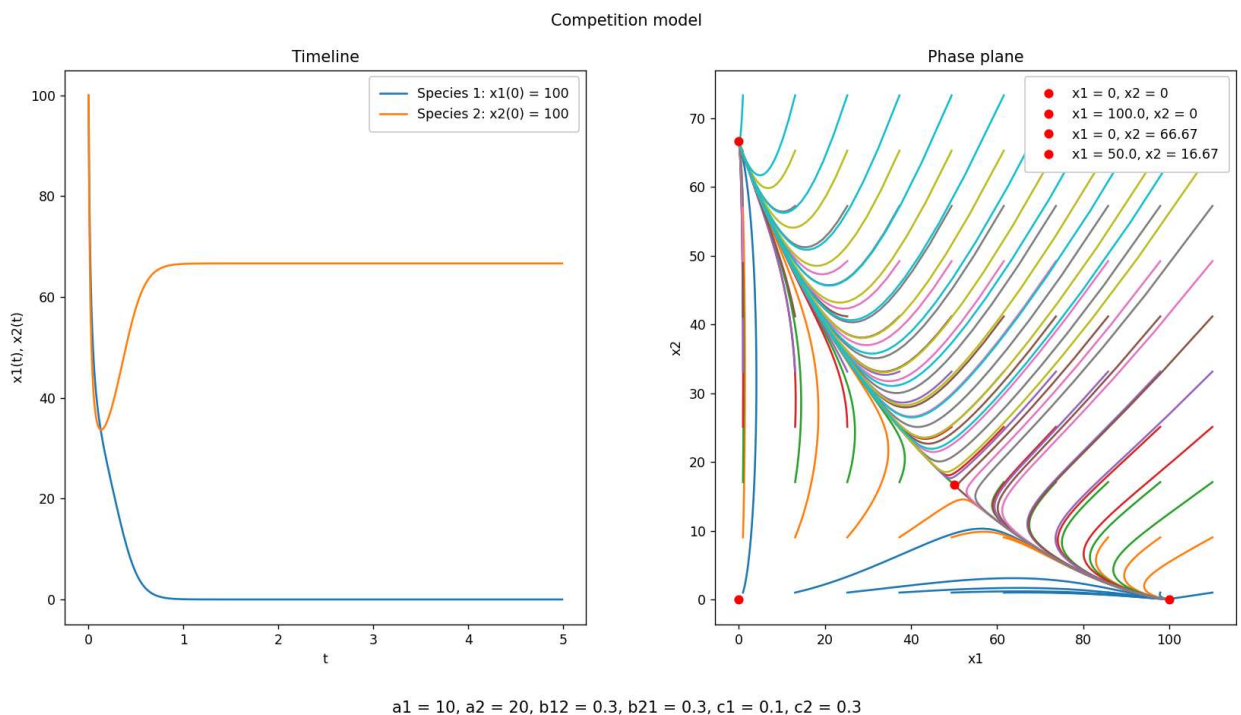


Рисунок 40 – Проекция интегральных кривых для случая $\frac{c_1}{b_{21}} < \frac{a_1}{a_2} < \frac{b_{12}}{c_2}$

Из графиков на рисунке выше, можно заметить, что в рассматриваемом случае при любых начальных значениях x_1, x_2 , не совпадающих с особыми, равновесие установится в одной из точек $\left(\frac{a_1}{c_1}, 0\right), \left(0, \frac{a_2}{c_2}\right)$, что означает вымирание одного вида. Единственным вариантом сосуществования обоих видов при данных соотношениях параметров является стационарная точка $\left(\frac{a_2 \cdot b_{12} - a_1 \cdot c_2}{b_{12} \cdot b_{21} - c_1 \cdot c_2}, \frac{a_1 \cdot b_{21} - a_2 \cdot c_1}{b_{12} \cdot b_{21} - c_1 \cdot c_2}\right)$.

Анализ системы при $a_1 \cdot b_{21} < a_2 \cdot c_1$ и $a_2 \cdot b_{12} < a_1 \cdot c_2$ демонстрируется на рисунке 41.

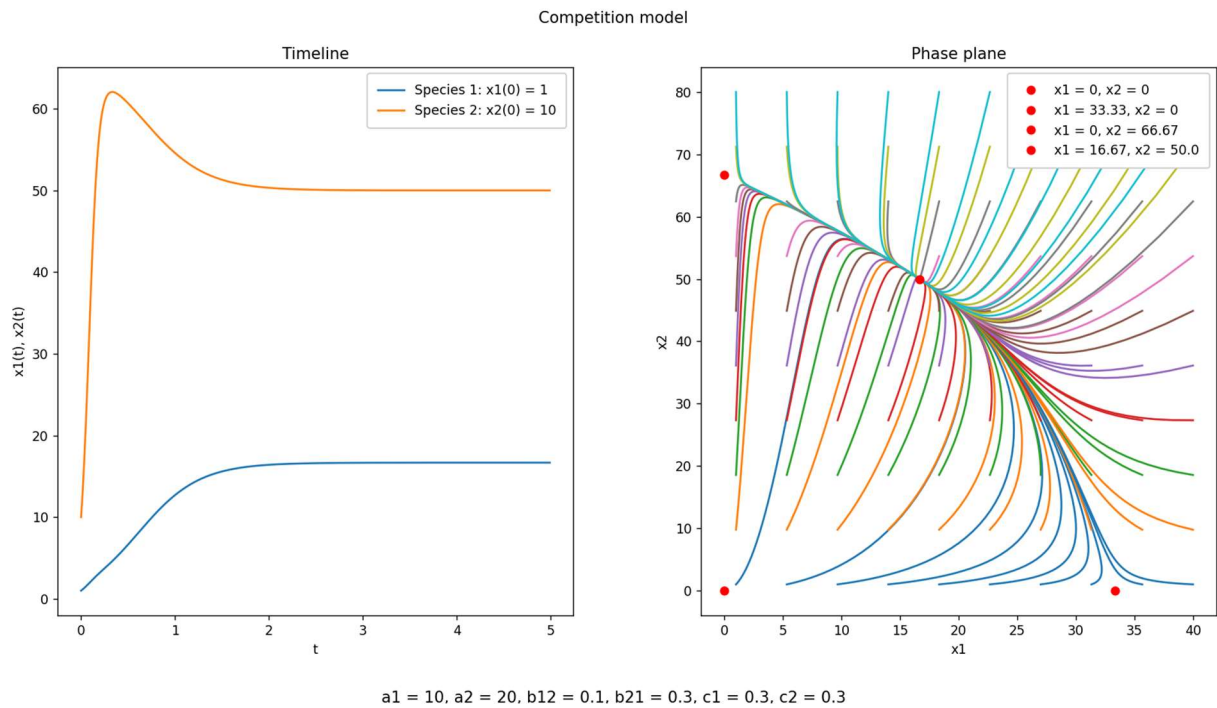


Рисунок 41 – Проекция интегральных кривых для случая $\frac{b_{12}}{c_2} < \frac{a_1}{a_2} < \frac{c_1}{b_{21}}$

В представленном случае ситуация оказывается обратной: при любых начальных значениях x_1, x_2 , не совпадающих с особыми, система стремится к положению равновесия $\left(\frac{a_2 \cdot b_{12} - a_1 \cdot c_2}{b_{12} \cdot b_{21} - c_1 \cdot c_2}, \frac{a_1 \cdot b_{21} - a_2 \cdot c_1}{b_{12} \cdot b_{21} - c_1 \cdot c_2}\right)$, при котором численность обоих видов положительна и неизменна.

Затем были рассмотрены варианты фазового портрета с тремя стационарными точками, расположенными на координатных осях. Вид траекторий системы в условиях $a_1 \cdot b_{21} < a_2 \cdot c_1$ и $a_2 \cdot b_{12} > a_1 \cdot c_2$ отражён на рисунке 42.

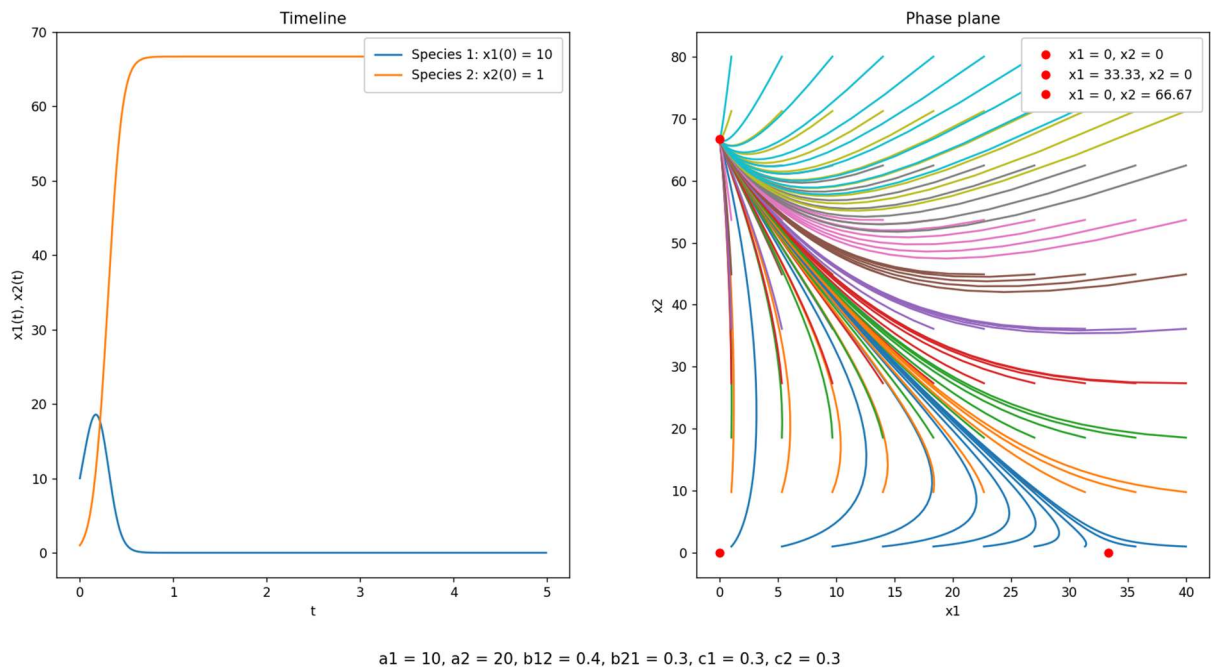


Рисунок 42 – Проекция интегральных кривых для случая $\frac{b_{12}}{c_2} > \frac{a_1}{a_2} < \frac{c_1}{b_{21}}$

Для данного случая характерно схождение траекторий в точке $(0, \frac{a_2}{c_2})$, что означает гарантированное вымирание первого вида. Исключение составляет лишь точка $(\frac{a_1}{c_1}, 0)$, в которой сохраняется неизменная положительная численность первого вида за счёт отсутствия представителей второго.

Ситуация при $a_1 \cdot b_{21} > a_2 \cdot c_1$ и $a_2 \cdot b_{12} < a_1 \cdot c_2$ показана на рисунке 43.

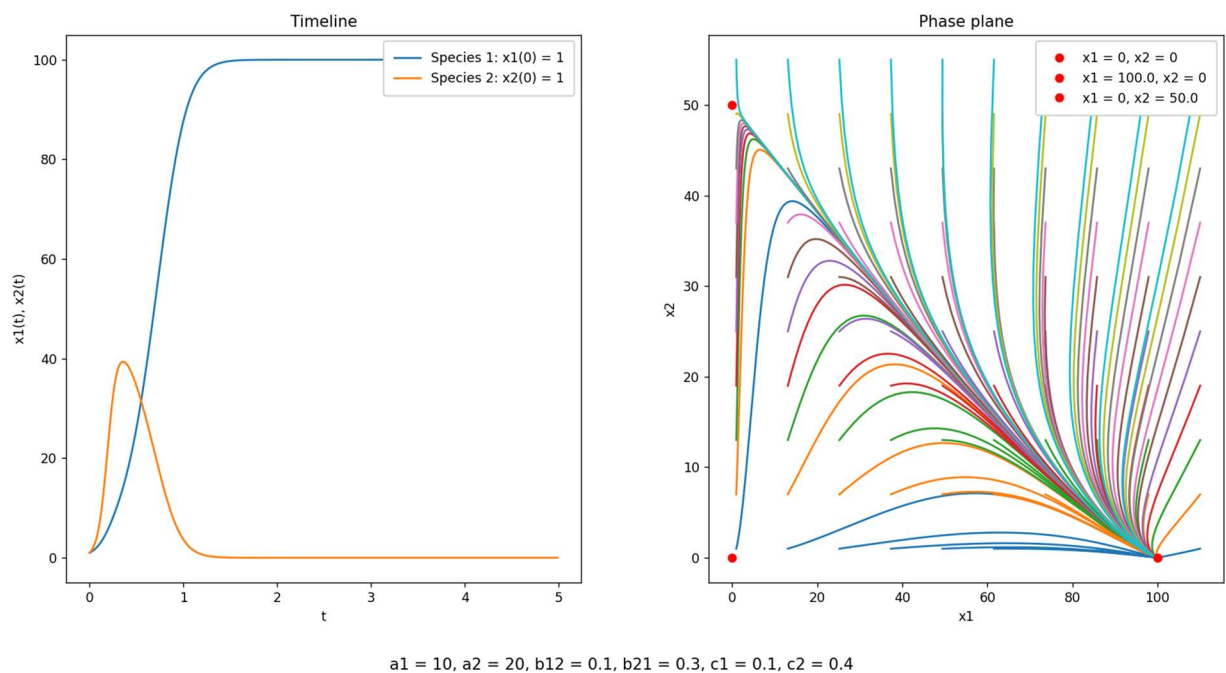


Рисунок 43 – Проекция интегральных кривых для случая $\frac{b_{12}}{c_2} < \frac{a_1}{a_2} > \frac{c_1}{b_{21}}$

Рассматриваемый вариант является симметрично-противоположным предыдущему: из любых начальных условий (кроме особых) система приходит к равновесию в устойчивом узле $\left(\frac{a_1}{c_1}, 0\right)$ – состоянии, равносильном гибели второго вида. Сохранение второго вида за счёт первого в точке $\left(0, \frac{a_2}{c_2}\right)$ возможно только при такой начальной численности.

При выполнении только одного равенства из двух ($a_1 \cdot b_{21} = a_2 \cdot c_1$ или $a_2 \cdot b_{12} = a_1 \cdot c_2$) фазовый портрет вблизи особой точки, матрица линеаризации которой оказывается вырожденной, немного напоминает прямую устойчивых точек покоя, хотя в остальном сохраняется картина устойчивого узла или седла. В ряде случаев отмечено изменение скорости достижения равновесного состояния. На рисунке 44 демонстрируется один из таких вариантов.

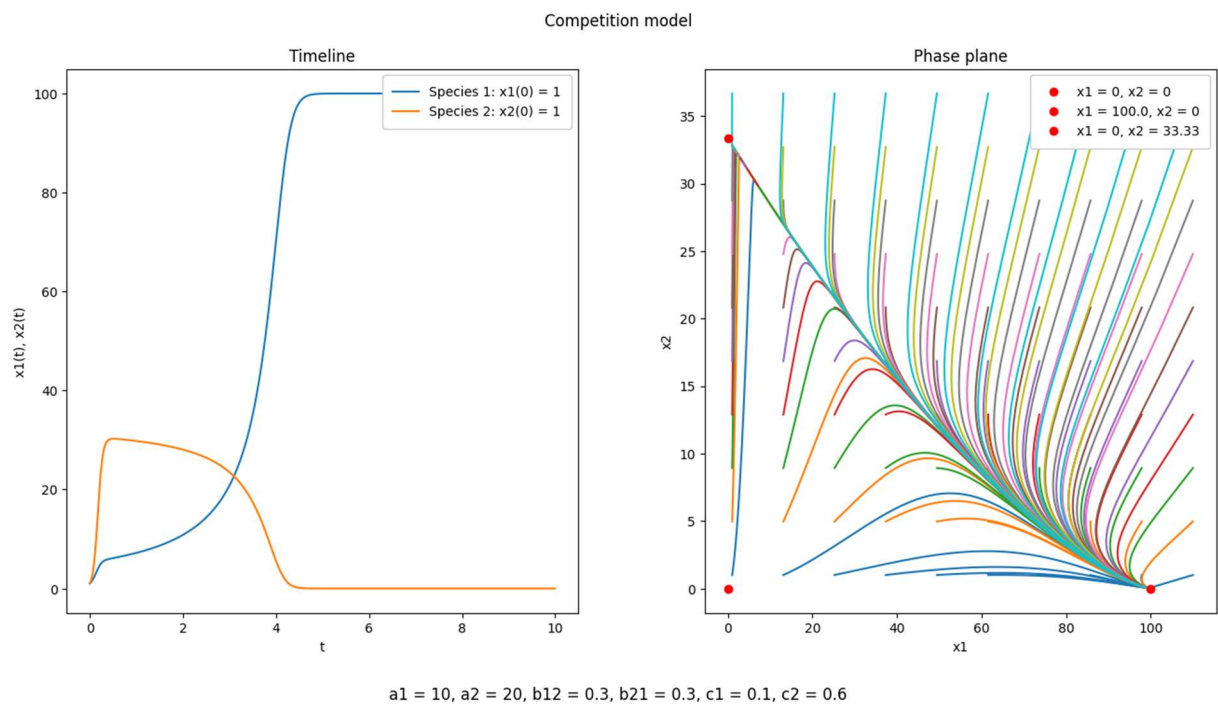


Рисунок 44 – Проекция интегральных кривых для случая $\frac{b_{12}}{c_2} = \frac{a_1}{a_2} > \frac{c_1}{b_{21}}$

Реализованные в процессе работы функции для построения и визуализации интегральных кривых были оформлены как методы класса в целях удобства использования для анализа модели конкуренции. Фрагмент исходного кода, содержащий инициализацию объекта класса, а также методы для описания динамики и построения интегральных кривых, приведён на рисунке 45.


```

import numpy as np
import matplotlib.pyplot as plt
from itertools import product
from my_tools import operator
from NSolveDE import RK4_auto_system, Euler_auto_system

class CompetitionModel(object):
    def __init__(self, a1, a2, b12, b21, c1, c2):
        if (a1 * b21 == a2 * c1) and (a2 * b12 == a1 * c2):
            raise ValueError('Invalid argument values: '
                             'a1 * b21 == a2 * c1 and a2 * b12 == a1 * c2')

        self.a1 = a1
        self.a2 = a2
        self.b12 = b12
        self.b21 = b21
        self.c1 = c1
        self.c2 = c2

        self.case1 = np.sign(a1 * b21 - a2 * c1)
        self.case2 = np.sign(a2 * b12 - a1 * c2)

        self.equilibrium_points = [(0, 0),
                                   (a1 / c1, 0),
                                   (0, a2 / c2)]

        if b12 * b21 != c1 * c2 and self.case1 * self.case2 > 0:
            self.equilibrium_points.append(((a2 * b12 - a1 * c2) / (b12 * b21 - c1 * c2),
                                             (a1 * b21 - a2 * c1) / (b12 * b21 - c1 * c2)))

    def dx1(self, x1, x2, args):
        return self.a1 * x1 - self.b12 * x1 * x2 - self.c1 * x1 ** 2

    def dx2(self, x1, x2, args):
        return self.a2 * x2 - self.b21 * x1 * x2 - self.c2 * x2 ** 2

    def get_integral_curves(self, x1_0, x2_0, t0, T, dt, method='RK4'):
        time = np.arange(t0, T, dt)

        n = time.shape[0]
        x1 = np.empty(n)
        x2 = np.empty(n)
        x1[0] = x1_0
        x2[0] = x2_0

        if method == 'RK4':
            get_next = RK4_auto_system
        elif method == 'Euler':
            get_next = Euler_auto_system
        else:
            print('Invalid method value. RK4 method used instead. '
                  'Available options are "RK4" and "Euler"')
            get_next = RK4_auto_system

        for t in range(0, n - 1):
            x1[t + 1], x2[t + 1] = get_next(self.dx1, self.dx2, x1[t], x2[t], dt, None, None)

        return time, x1, x2

```

Рисунок 45 – Инициализация и ключевые методы класса CompetitionModel

На рисунке 46 приведён исходный код метода класса, используемого для визуализации проекций интегральных кривых.

```
def plot(self,
        initial_point=(1, 1),
        x1_min=1,
        x2_min=1,
        x1_max=None,
        x2_max=None,
        zoom=1.2,
        density=10
    ):
    if x1_max is None:
        x1_max = zoom * max([point[0] for point in self.equilibrium_points])
    if x2_max is None:
        x2_max = zoom * max([point[1] for point in self.equilibrium_points])

    x1_coords = np.linspace(x1_min, x1_max, density)
    x2_coords = np.linspace(x2_min, x2_max, density)

    initial_points = list(product(*[x1_coords, x2_coords]))

    # Visualization
    fig, axes = plt.subplots(nrows=1, ncols=2)
    fig.tight_layout(pad=3, w_pad=1)
    fig.suptitle('Competition model')
    fig.supxlabel(f'a1 = {self.a1}, a2 = {self.a2}, '
                  f'b12 = {self.b12}, b21 = {self.b21}, '
                  f'c1 = {self.c1}, c2 = {self.c2} \n\n'
                  f'a1 * b21 {operator(self.case1)} a2 * c1, '
                  f'a2 * b12 {operator(self.case2)} a1 * c2')

    # Timeline
    time, X1, X2 = self.get_integral_curves(initial_point[0],
                                             initial_point[1],
                                             t0, T, dt, 'RK4')

    axes[0].plot(time, X1, label=f'Species 1: x1(0) = {initial_point[0]}')
    axes[0].plot(time, X2, label=f'Species 2: x2(0) = {initial_point[1]}')

    axes[0].set_xlabel('t')
    axes[0].set_ylabel('x1(t), x2(t)')
    axes[0].set_title('Timeline')
    axes[0].legend(loc='upper right', framealpha=1, borderpad=0.8)

    # Phase plane
    for (x1_0, x2_0) in initial_points:
        time, X1, X2 = self.get_integral_curves(x1_0, x2_0, t0, T, dt, 'RK4')
        axes[1].plot(X1, X2)

    for (x1_0, x2_0) in self.equilibrium_points:
        axes[1].plot(x1_0, x2_0, 'ro', label=f'x1 = {round(x1_0, 2)}, x2 = {round(x2_0, 2)}')

    axes[1].set_xlabel('x1')
    axes[1].set_ylabel('x2')
    axes[1].set_title('Phase plane')
    axes[1].legend(loc='upper right', framealpha=1, borderpad=0.8)

    plt.show()
```

Рисунок 46 – Визуализация интегральных кривых

Исходный код метода класса для построения фазового портрета с помощью функции `streamplot` библиотеки `matplotlib` представлен на рисунке 47.

```

def streamplot(self,
                x1_min=1,
                x2_min=1,
                x1_max=None,
                x2_max=None,
                zoom=1.5,
                ls_density=10,
                sp_density=2
                ):
    if x1_max is None:
        x1_max = zoom * max([point[0] for point in self.equilibrium_points])
    if x2_max is None:
        x2_max = zoom * max([point[1] for point in self.equilibrium_points])

    x = np.linspace(x1_min, x1_max, ls_density)
    y = np.linspace(x2_min, x2_max, ls_density)
    X, Y = np.meshgrid(x, y)
    U_, V_ = np.array([self.dx1(X, Y, None),
                       self.dx2(X, Y, None)])

    fig, axes = plt.subplots(nrows=1, ncols=1)
    fig.tight_layout()
    fig.subplots_adjust(bottom=0.175, top=0.93, right=0.8, left=0.2)

    fig.suptitle('Competition model')
    fig.supxlabel(f'a1 = {self.a1}, a2 = {self.a2}, '
                 f'b12 = {self.b12}, b21 = {self.b21}, '
                 f'c1 = {self.c1}, c2 = {self.c2} \n\n'
                 f'a1 * b21 {operator(self.case1)} a2 * c1, '
                 f'a2 * b12 {operator(self.case2)} a1 * c2')

    axes.axis((x1_min, x1_max, x2_min, x2_max))
    axes.set_xlabel('x1')
    axes.set_ylabel('x2')

    axes.streamplot(x, y, U_, V_,
                    density=sp_density,
                    linewidth=1,
                    arrowstyle="->")

    for point in self.equilibrium_points:
        axes.plot(*point,
                  'bo',
                  label=f'{{round(point[0], 2), round(point[1], 2)}}')

    axes.legend(loc='upper right', framealpha=1, borderpad=0.8)

    plt.show()

```

Рисунок 47 – Исходный код метода для визуализации векторного поля

Стоит отметить, что модель конкуренции (2.4), как и модель «хищник-жертва» (2.3), является частным случаем модели взаимодействия двух видов, построенной на общих гипотезах Вольтерра и включающей помимо вышеуказанных такие типы взаимодействия, как симбиоз, комменсализм, аменсализм и нейтрализм.

2.5. Аттрактор Лоренца

Модель Лоренца – стохастическая модель, основанная на упрощённых гидродинамических уравнениях Навье-Стокса.

Математически модель описывается трёхмерной нелинейной автономной системой дифференциальных уравнений следующего вида:

$$\begin{cases} \frac{dx}{dt} = \sigma \cdot (y - x), \\ \frac{dy}{dt} = x \cdot (r - z) - y, \\ \frac{dz}{dt} = x \cdot y - b \cdot z. \end{cases} \quad (38)$$

где σ, r, b – параметры системы, $\sigma, r, b > 0$.

Модель Лоренца является реальным физическим примером динамических систем с хаотическим поведением. Система (38) демонстрирует большое разнообразие качественно различных динамических режимов, в том числе сосуществование периодических аттракторов и переход к хаосу через удвоение периода. При $\sigma = 10$, $b = \frac{8}{3}$ и произвольном r , система является детерминированной.

Аттрактор – компактное инвариантное подмножество фазового пространства динамической системы, все траектории из некоторой окрестности которого стремятся к нему с течением времени.

Аттрактор Лоренца – странный аттрактор квазигиперболического типа, возникающий в системе (38) при $\sigma = 10$, $b = \frac{8}{3}$ и $r = 28$ (классическое значение параметра r) независимо от выбора начальных условий.

При других значениях r система имеет не менее интересные решения, однако детально в данной работе они рассмотрены не будут.

Задание 1

Реализовать имитационную модель Лоренца в AnyLogic. Проанализировать поведение системы (38) при различных значениях параметра r . Построить фазовые портреты в плоскостях XY , YZ , XZ , а также графики $x(t)$, $y(t)$, $z(t)$.

Решение

Значения x, y, z были реализованы как накопители с динамикой, задаваемой соответствующим уравнением системы (38). На рисунке 48 демонстрируется запуск симуляции с классическими значениями параметров, при которых возникает аттрактор Лоренца, из начальной точки с координатами $(1, 1, 20)$.

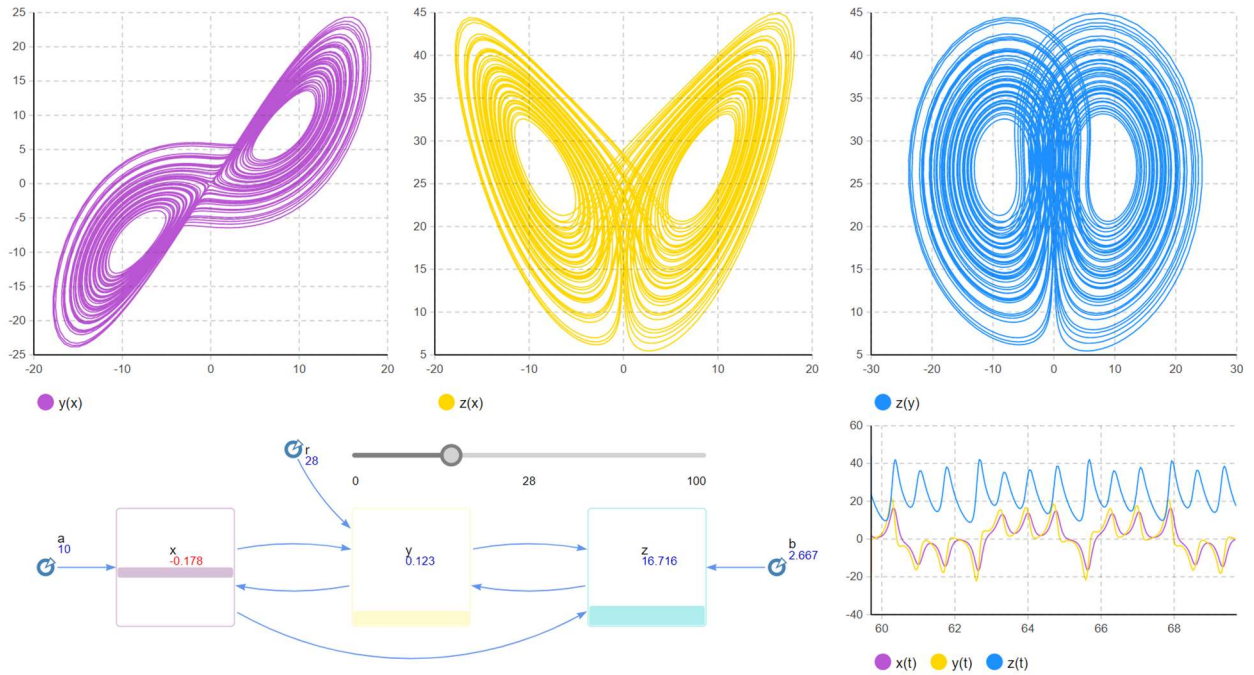


Рисунок 48 – Классический аттрактор Лоренца при $r = 28$

Один из устойчивых фокусов при $r = 12$ отражён на рисунке ниже.

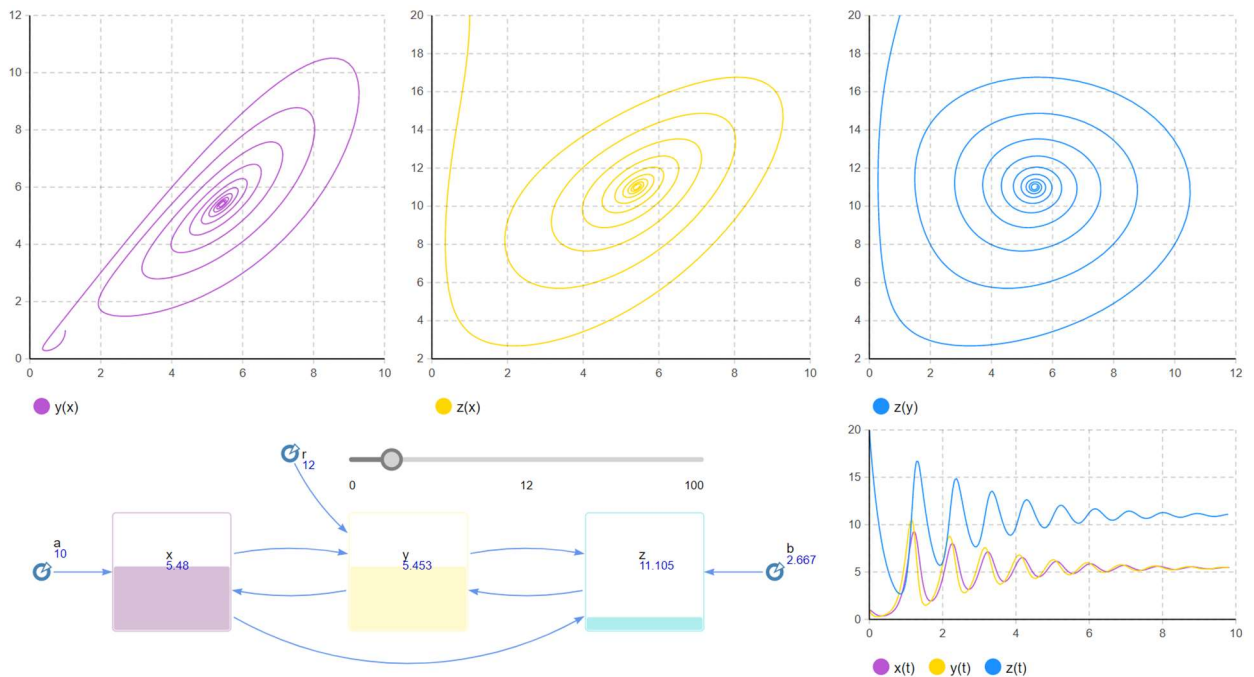


Рисунок 49 – Устойчивый фокус при $r = 12$

Приблизиться к получению гомоклинической петли удалось при значении $r = 13,879472$ и начальных условиях $x(0) = 10^{-5}$, $y(0) = z(0) = 0$. Результат отражён на рисунке 50.

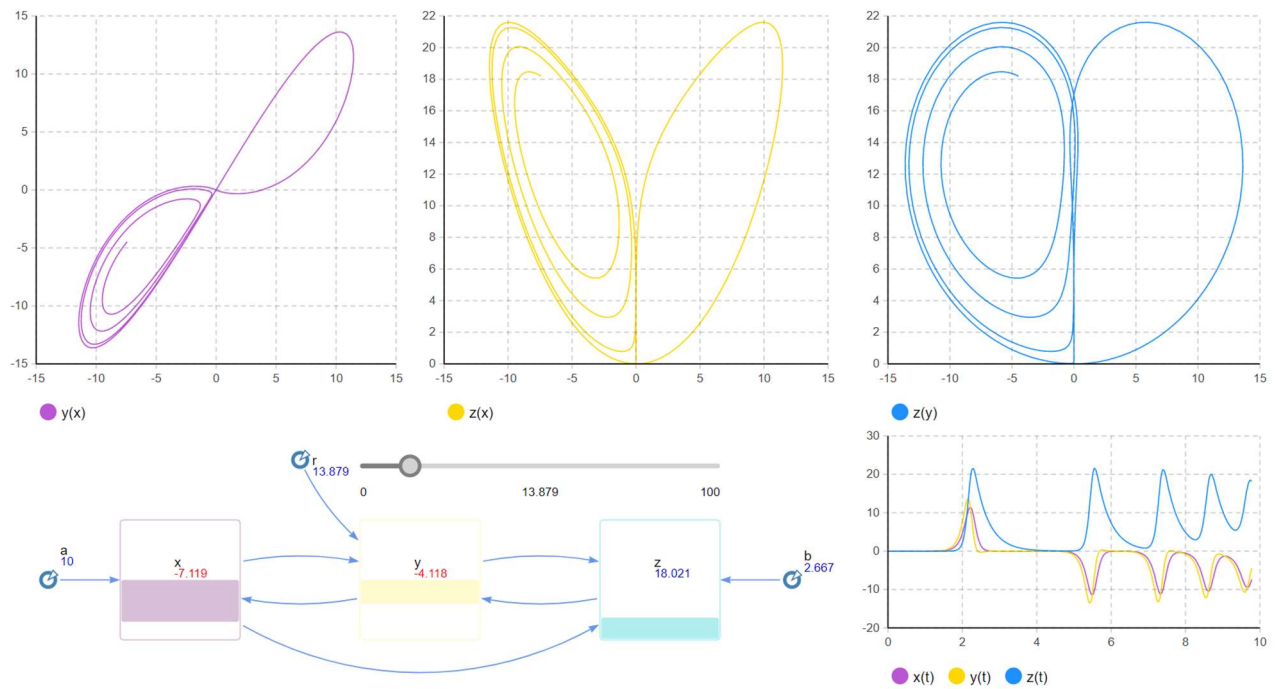


Рисунок 50 – Попытка построения гомоклинической траектории

На рисунке 51 приведён предельный цикл, полученный при $r \approx 23,6$. С этого момента траектории перестают сходиться к одному из фокусов.

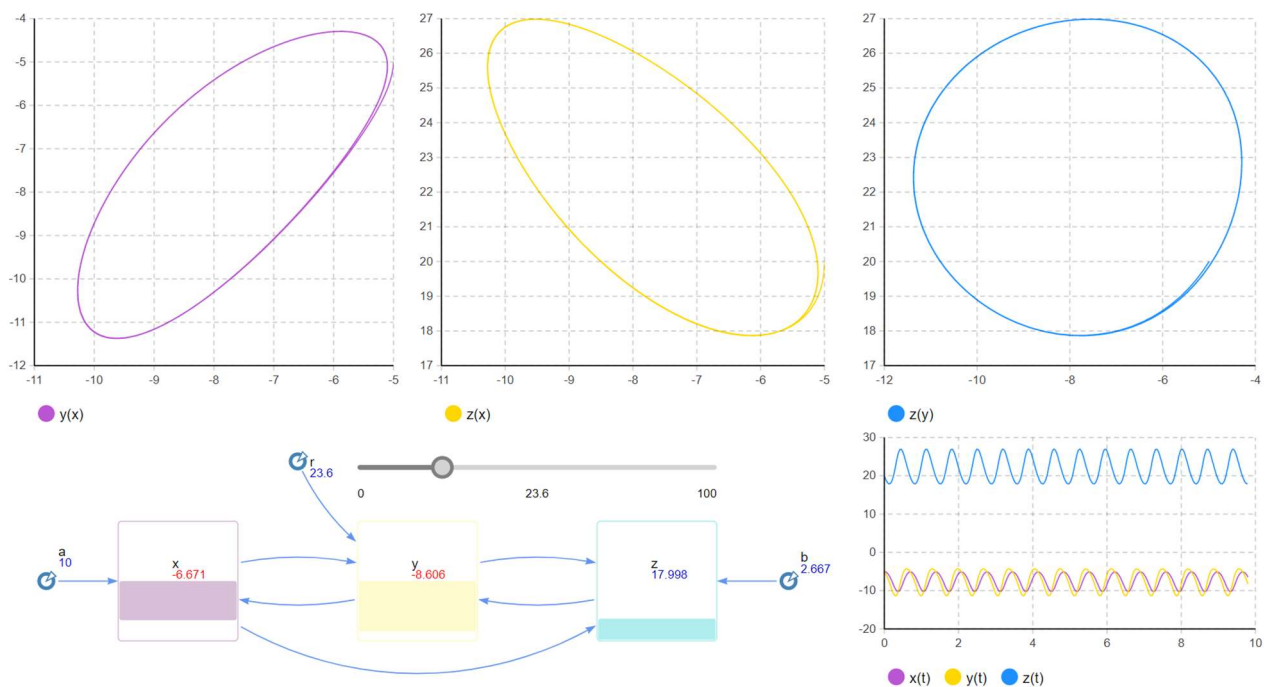


Рисунок 51 – Предельный цикл

Из рисунка 52 видно, что при больших значениях параметра снижается хаотичность и возникают несимметричные периодические решения.

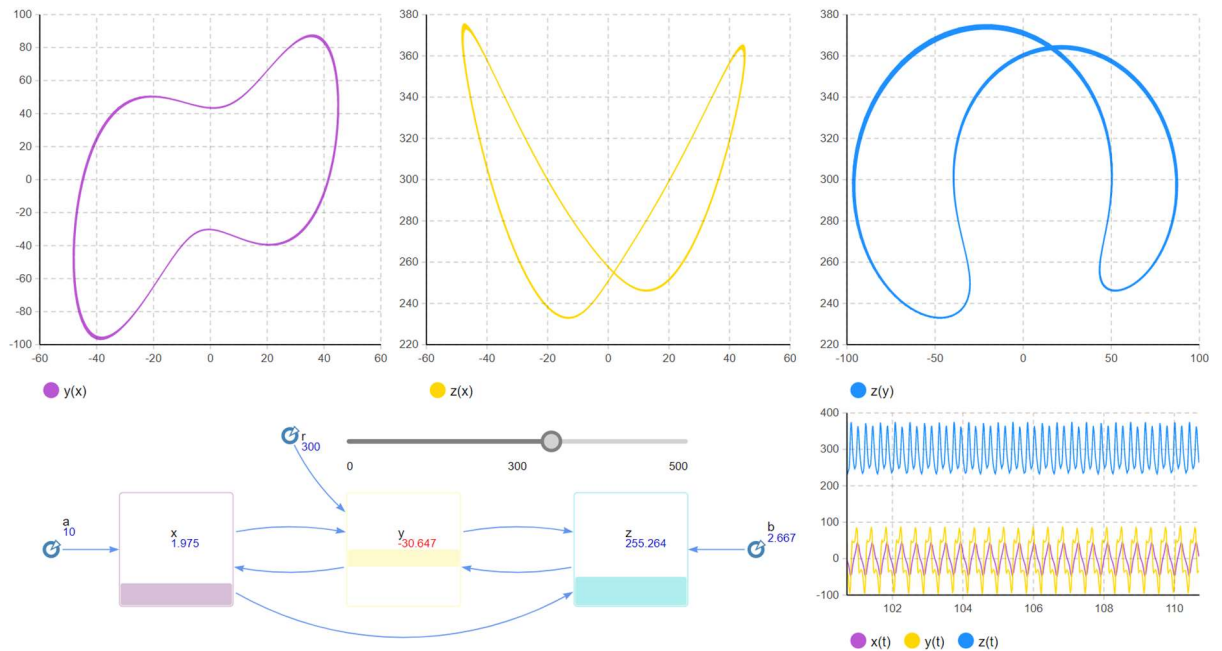


Рисунок 52 – Запуск симуляции на $r = 300$

С приближением r к бесконечности замкнутая интегральная кривая становится всё более симметричной и похожей на объёмный полумесяц или подкову. В ряде случаев отмечено стремление формы к плоскому кругу.

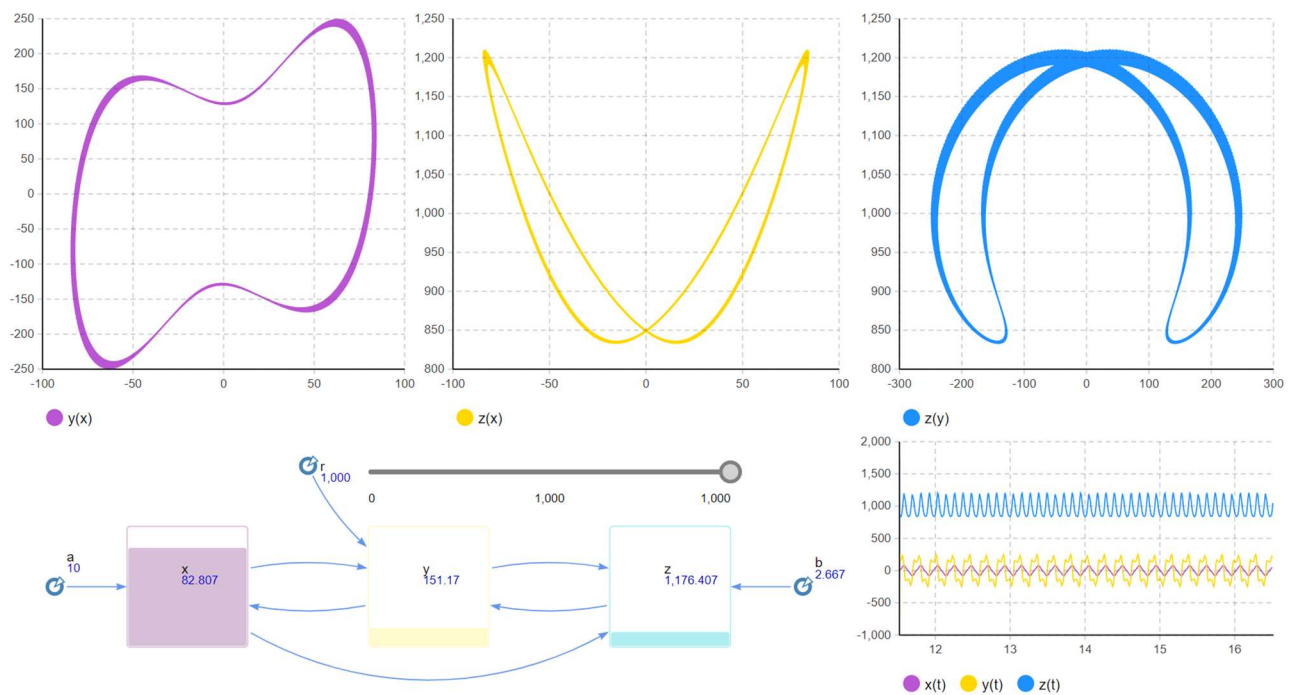


Рисунок 53 – Поведение системы при $r = 1000$

Задание 2

Численный анализ аттрактора Лоренца в Python.

Решение

Для численного решения системы (38) реализованные ранее методы были модифицированы для трёхмерного случая, как показано на рисунке 54.

```
def Euler_auto_system_XYZ(f, g, h, x_t, y_t, z_t, dt, f_args, g_args, h_args):
    """
    Numerical solving of a three-dimensional autonomous DE system by the Euler method
    :param f: function name  $f(x, y, z, *args) = dx/dt$ 
    :param g: function name  $g(x, y, z, *args) = dy/dt$ 
    :param h: function name  $h(x, y, z, *args) = dz/dt$ 
    :param x_t:  $x(t)$ 
    :param y_t:  $y(t)$ 
    :param z_t:  $z(t)$ 
    :param dt:  $t\_step$ 
    :param f_args: additional positional arguments of the function  $f$  (iterable object)
    :param g_args: additional positional arguments of the function  $g$  (iterable object)
    :param h_args: additional positional arguments of the function  $h$  (iterable object)
    :return: tuple  $(x(t+dt), y(t+dt), z(t+dt))$ 
    """
    x_next = x_t + dt * f(x_t, y_t, z_t, f_args)
    y_next = y_t + dt * g(x_t, y_t, z_t, g_args)
    z_next = z_t + dt * h(x_t, y_t, z_t, h_args)

    return x_next, y_next, z_next


def RK4_auto_system_XYZ(f, g, h, x_t, y_t, z_t, dt, f_args, g_args, h_args):
    """
    Numerical solving of a three-dimensional autonomous DE system by the Runge-Kutta method
    :param f: function name  $f(x, y, z, *args) = dx/dt$ 
    :param g: function name  $g(x, y, z, *args) = dy/dt$ 
    :param h: function name  $h(x, y, z, *args) = dz/dt$ 
    :param x_t:  $x(t)$ 
    :param y_t:  $y(t)$ 
    :param z_t:  $z(t)$ 
    :param dt:  $t\_step$ 
    :param f_args: additional positional arguments of the function  $f$  (iterable object)
    :param g_args: additional positional arguments of the function  $g$  (iterable object)
    :param h_args: additional positional arguments of the function  $h$  (iterable object)
    :return: tuple  $(x(t+dt), y(t+dt), z(t+dt))$ 
    """
    k1x = dt * f(x_t, y_t, z_t, f_args)
    k1y = dt * g(x_t, y_t, z_t, g_args)
    k1z = dt * h(x_t, y_t, z_t, h_args)

    k2x = dt * f(x_t + k1x / 2, y_t + k1y / 2, z_t + k1z / 2, f_args)
    k2y = dt * g(x_t + k1x / 2, y_t + k1y / 2, z_t + k1z / 2, g_args)
    k2z = dt * h(x_t + k1x / 2, y_t + k1y / 2, z_t + k1z / 2, h_args)

    k3x = dt * f(x_t + k2x / 2, y_t + k2y / 2, z_t + k2z / 2, f_args)
    k3y = dt * g(x_t + k2x / 2, y_t + k2y / 2, z_t + k2z / 2, g_args)
    k3z = dt * h(x_t + k2x / 2, y_t + k2y / 2, z_t + k2z / 2, h_args)

    k4x = dt * f(x_t + k3x, y_t + k3y, z_t + k3z, f_args)
    k4y = dt * g(x_t + k3x, y_t + k3y, z_t + k3z, g_args)
    k4z = dt * h(x_t + k3x, y_t + k3y, z_t + k3z, h_args)

    x_next = x_t + (k1x + 2 * k2x + 2 * k3x + k4x) / 6
    y_next = y_t + (k1y + 2 * k2y + 2 * k3y + k4y) / 6
    z_next = z_t + (k1z + 2 * k2z + 2 * k3z + k4z) / 6

    return x_next, y_next, z_next
```

Рисунок 54 – Численное решение трёхмерной автономной системы ОДУ методами Эйлера и Рунге-Кутты на Python

Модель Лоренца была представлена в виде класса, атрибутами которого являются параметры системы. Исходный код инициализации объекта класса, а также методов, предназначенных для задания динамики согласно уравнениям (38) и численного интегрирования, приведён на рисунке 55.

```
class LorenzAttractor(object):
    def __init__(self, sigma=10.0, r=28.0, b=8 / 3):
        self.sigma = sigma
        self.r = r
        self.b = b

        if self.r > 1:
            xy = np.sqrt(self.b * (self.r - 1))
            self.special_points = [(xy, xy, self.r - 1),
                                   (-xy, -xy, self.r - 1)]
        else:
            self.special_points = [(0, 0, 0)]

    def dx(self, x, y, z, args=None):
        return self.sigma * (y - x)

    def dy(self, x, y, z, args=None):
        return x * (self.r - z) - y

    def dz(self, x, y, z, args=None):
        return x * y - self.b * z

    def get_integral_curve(self, x0, y0, z0, t0, T, dt, method='RK4'):
        time = np.arange(t0, T, dt)

        n = time.shape[0]
        x = np.empty(n)
        y = np.empty(n)
        z = np.empty(n)

        x[0] = x0
        y[0] = y0
        z[0] = z0

        if method == 'RK4':
            get_next = RK4_auto_system_XYZ
        elif method == 'Euler':
            get_next = Euler_auto_system_XYZ
        else:
            print('Invalid method value. RK4 method used instead. '
                  'Available options are "RK4" and "Euler"')
            get_next = RK4_auto_system_XYZ

        for t in range(0, n - 1):
            x[t + 1], y[t + 1], z[t + 1] = get_next(self.dx, self.dy, self.dz,
                                                    x[t], y[t], z[t],
                                                    dt,
                                                    None, None, None)

        return time, x, y, z
```

Рисунок 55 – Инициализация и ключевые методы класса LorenzAttractor

С помощью инструментов библиотеки matplotlib была реализована визуализация полученной интегральной кривой в трёхмерном пространстве. Также на графике были отмечены особые точки для случая $r > 1$ согласно формулам:

$$\begin{aligned}(x_1^*, y_1^*, z_1^*) &= (\sqrt{b \cdot (r - 1)}, \sqrt{b \cdot (r - 1)}, r - 1) \\(x_2^*, y_2^*, z_2^*) &= (-\sqrt{b \cdot (r - 1)}, -\sqrt{b \cdot (r - 1)}, r - 1)\end{aligned}\tag{39}$$

Исходный код метода представлен на рисунке 56.

```
def plot3d(self, t0, T, dt, initial_point=(1, 1, 1), cut=0):

    time, x, y, z = self.get_integral_curve(initial_point[0],
                                            initial_point[1],
                                            initial_point[2],
                                            t0, T, dt, 'RK4')

    fig = plt.figure()
    fig.suptitle(f'Lorenz attractor (r = {self.r}), t = [{t0}, {T}])')

    axes = fig.add_subplot(1, 1, 1, projection='3d')
    axes.set_xlabel("x")
    axes.set_ylabel("y")
    axes.set_zlabel("z")

    axes.plot(x[cut:], y[cut:], z[cut:], lw=0.5, color='mediumslateblue')
    for point in self.special_points:
        axes.scatter(*point, marker='o', color='magenta')

    plt.show()
```

Рисунок 56 – Функция для построения трёхмерного графика

Изображение, полученное для случая $r = 24$, приведено на рисунке 57.

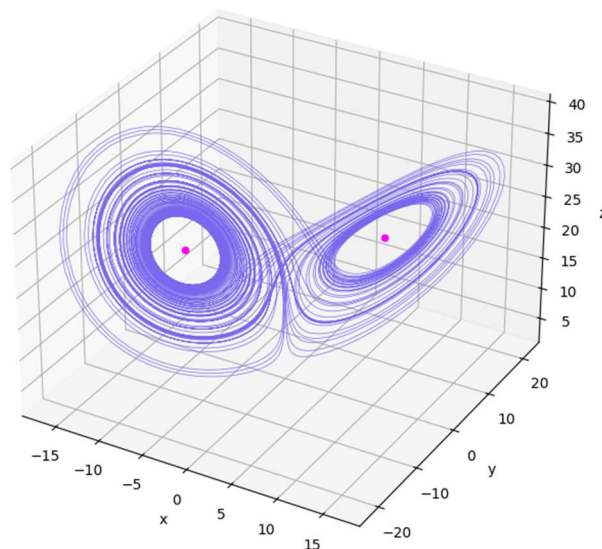


Рисунок 57 – Трёхмерное изображение аттрактора Лоренца при $r = 24$

Реализация метода для отображения графиков $x(t), y(t), z(t)$, а также фазовых портретов в плоскостях XY, XZ, YZ демонстрируется на рисунке 58.

```
def plot2d(self, t0, T, dt, initial_point=(1., 1., 1.), cut=0):

    time, x, y, z = self.get_integral_curve(initial_point[0],
                                            initial_point[1],
                                            initial_point[2],
                                            t0, T, dt, 'RK4')

    time_cut = time[cut:]
    x_cut = x[cut:]
    y_cut = y[cut:]
    z_cut = z[cut:]
    tpc = int(50/dt)

    fig = plt.figure()
    fig.suptitle(f'Lorenz attractor (r = {self.r}, t = [{t0}, {T}])')

    # Timeline
    axes3 = fig.add_subplot(2, 2, 3)
    axes3.plot(time_cut[:tpc], x_cut[:tpc], lw=0.7, label='x(t)', color='purple')
    axes3.plot(time_cut[:tpc], y_cut[:tpc], lw=0.7, label='y(t)', color='gold')
    axes3.plot(time_cut[:tpc], z_cut[:tpc], lw=0.7, label='z(t)', color='dodgerblue')
    axes3.set_xlabel("t")
    axes3.set_ylabel("x, y, z")
    axes3.legend(loc='upper right', framealpha=1)

    axes1 = fig.add_subplot(2, 2, 1)
    axes1.plot(x_cut, y_cut, lw=0.7, label='y(x)', color='mediumorchid')
    # axes1.set_title('y(x)')
    axes1.set_xlabel("x")
    axes1.set_ylabel("y")
    axes1.legend()

    axes2 = fig.add_subplot(2, 2, 2)
    axes2.plot(x_cut, z_cut, lw=0.7, label='z(x)', color='gold')
    # axes2.set_title('z(x)')
    axes2.set_xlabel("x")
    axes2.set_ylabel("z")
    axes2.legend()

    axes4 = fig.add_subplot(2, 2, 4)
    axes4.plot(y_cut, z_cut, lw=0.7, label='z(y)', color='dodgerblue')
    # axes2.set_title('z(y)')
    axes4.set_xlabel("y")
    axes4.set_ylabel("z")
    axes4.legend()

    plt.show()
```

Рисунок 58 – Функция для построения двумерных графиков

Стоит отметить, что отличное от нуля значение аргумента `cut` позволяет при необходимости убрать соответствующее количество первых точек решения, чтобы при построении проекций интегральной кривой скрыть начальный участок, который может находиться далеко от аттрактора и влиять на масштаб.

Графики, полученные в результате вызова метода у объекта модели со значением параметра $r = 24$ на временном интервале $[0, 100]$ с рангом разбиения 0,01, приведены на рисунке 59.

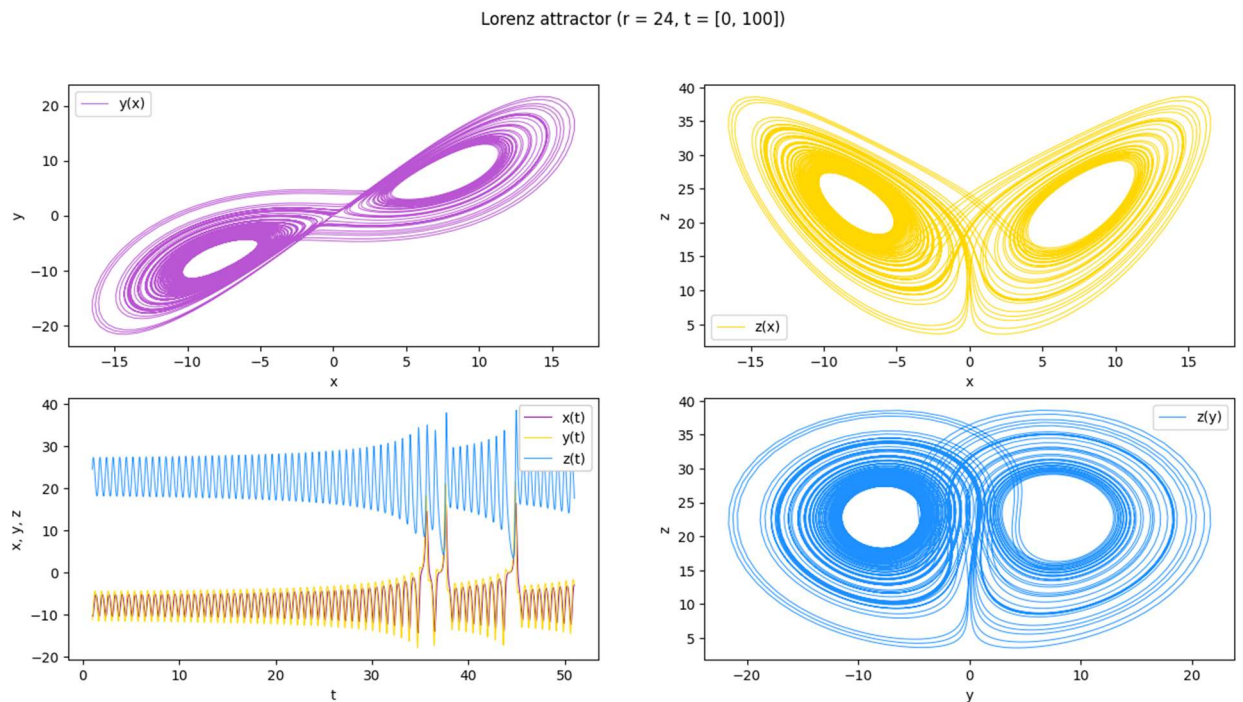


Рисунок 59 – Проекция траектории аттрактора Лоренца при $r = 24$

Примечательный вид решения системы был получен при $r = 100$. Проекция интегральной кривой демонстрируются на рисунке 60.

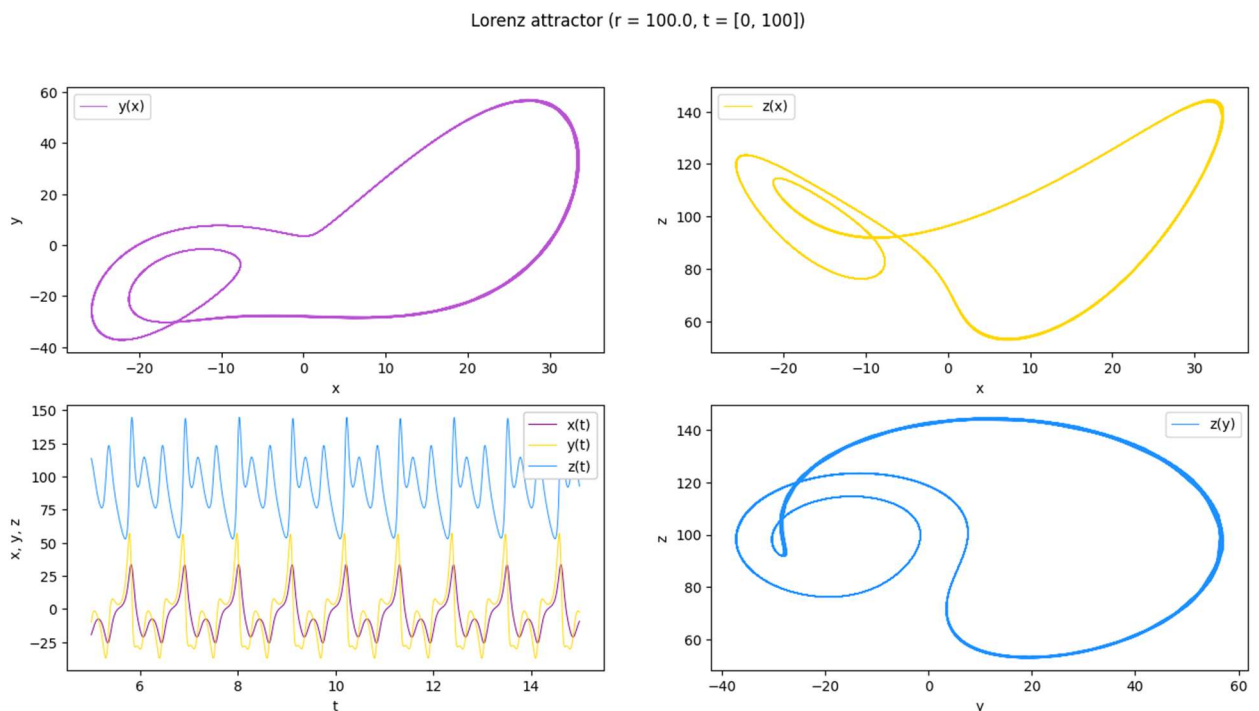


Рисунок 60 – Проекция интегральной кривой при $r = 100$

На рисунке 61 представлено объёмное изображение полученного решения.

Lorenz attractor ($r = 100.0$), $t = [0, 200]$)

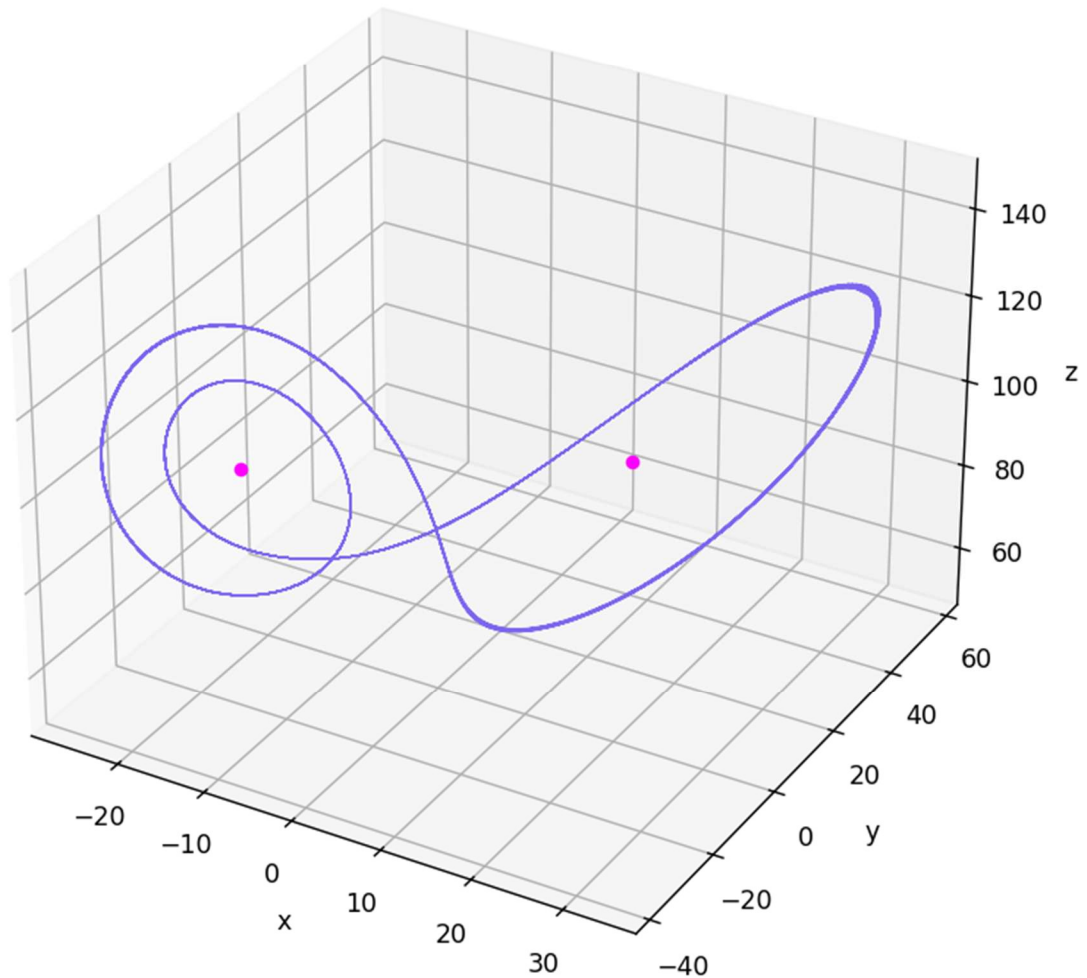


Рисунок 61 – Трёхмерное изображение траектории при $r = 100$

Стоит отметить, что при $r = 100 \pm 0.1$ траектория принимала вид, более характерный для хаотического странного аттрактора.

Поскольку методы, реализованные в Python (см. рисунок 54), не позволяли получить решение для больших значений параметра r в связи с возникающим переполнением, трёхмерные изображения для анализа $r > 1000$ были построены с помощью средств Wolfram Mathematica.

3. МОДЕЛИРОВАНИЕ ФИЗИЧЕСКИХ СИСТЕМ

3.1. Двойной математический маятник

Маятник – твёрдое тело, совершающее колебательное движение около неподвижной точки или оси.

Математический маятник – материальная точка, подвешенная на лёгкой нерастяжимой нити или на невесомом жёстком стержне.

Двойной маятник – механическая система из двух связанных между собой маятников.

В данном разделе будет рассмотрена модель *двойного математического маятника*.

Описание модели:

Первый математический маятник подвешен на невесомом жёстком стержне к неподвижной точке, а второй прикреплён с помощью другого невесомого жёсткого стержня к его концу (центру масс).

Схематичное изображение модели представлено на рисунке 62.

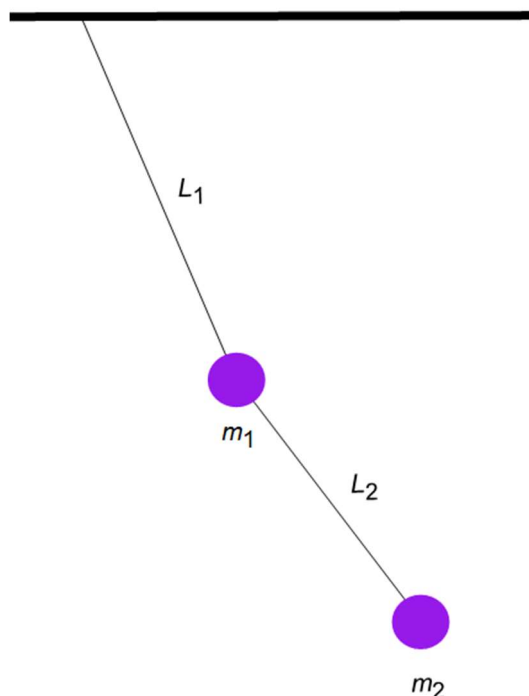


Рисунок 62 – Схема двойного математического маятника

Обозначения:

m_1 – масса первого маятника;

m_2 – масса второго маятника;

L_1 – длина стержня первого маятника;

L_2 – длина стержня второго маятника;

(x_1, y_1) – координаты центра масс первого маятника;

(x_2, y_2) – координаты центра масс второго маятника;

θ_1 – угол отклонения первого маятника от оси Oy ;

θ_2 – угол отклонения второго маятника от оси Oy .

Существует несколько способов получить уравнения движения двойного маятника: через рассмотрение сил, моментов сил или энергий. В механике для получения требуемых выражений чаще всего прибегают к составлению лагранжиана и решению уравнений Эйлера-Лагранжа. Однако наиболее простым для понимания методом является анализ сил, показанный на рисунке 63.

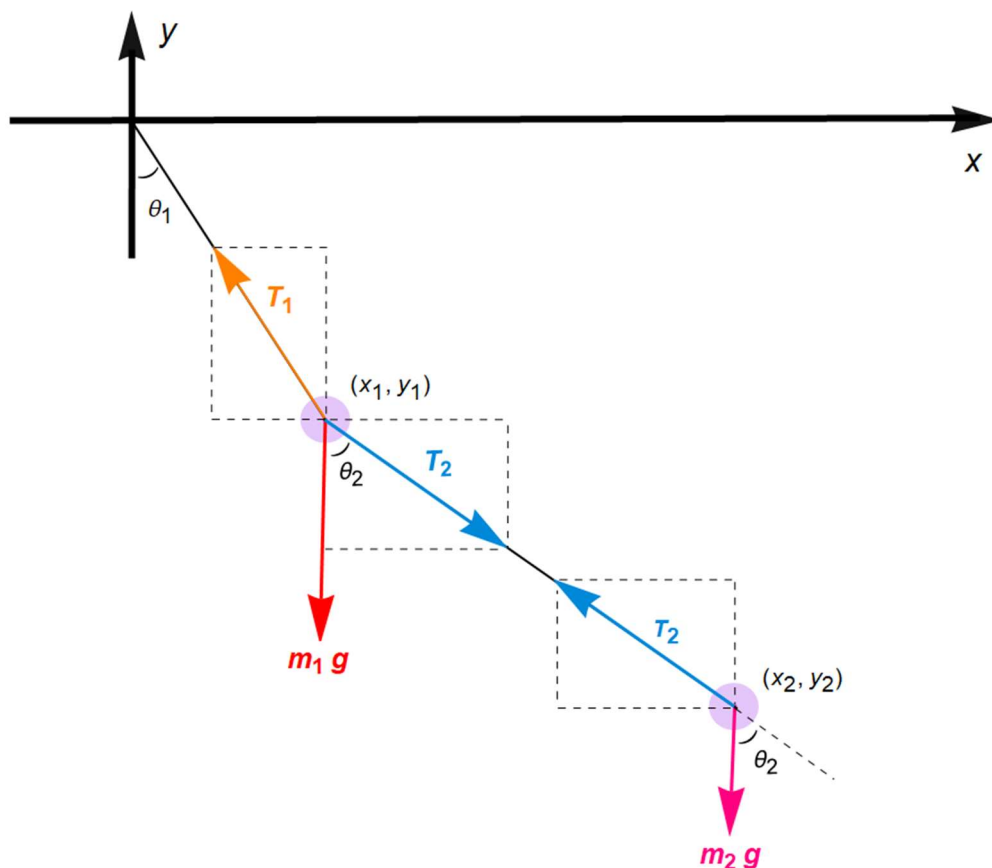


Рисунок 63 – Анализ сил

Запишем II закон Ньютона (для проекций):

$$\begin{cases} m_1 \ddot{x}_1 = -T_1 \sin \theta_1 + T_2 \sin \theta_2, \\ m_1 \ddot{y}_1 = T_1 \cos \theta_1 - T_2 \cos \theta_2 - m_1 g, \\ m_2 \ddot{x}_2 = -T_2 \sin \theta_2, \\ m_2 \ddot{y}_2 = T_2 \cos \theta_2 - m_2 g. \end{cases} \quad (40)$$

Избавимся от сил упругости T_1, T_2 :

$$\begin{cases} -\cos \theta_1 \cdot (m_1 \ddot{x}_1 + m_2 \ddot{x}_2) = \sin \theta_1 \cdot (m_1 \ddot{y}_1 + m_2 \ddot{y}_2 + m_1 g + m_2 g), \\ -\cos \theta_2 \cdot (m_2 \ddot{x}_2) = \sin \theta_2 \cdot (m_2 \ddot{y}_2 + m_2 g). \end{cases} \quad (41)$$

Затем можно домножить уравнения на соответствующие длины L_1, L_2 (см. рисунок 62) и получить систему из двух дифференциальных уравнений второго порядка относительно координат x_1, y_1, x_2, y_2 . Однако для того, чтобы система имела единственное решение, необходимо добавить алгебраические ограничения, связывающие координаты с длиной стержней (по теореме Пифагора).

$$\begin{cases} y_1 \cdot (m_1 \ddot{x}_1 + m_2 \ddot{x}_2) = x_1 \cdot (m_1 \ddot{y}_1 + m_2 \ddot{y}_2 + m_1 g + m_2 g), \\ (y_2 - y_1) \cdot m_2 \ddot{x}_2 = (x_2 - x_1) \cdot (m_2 \ddot{y}_2 + m_2 g), \\ x_1^2 + y_1^2 = L_1^2, \\ (x_2 - x_1)^2 + (y_2 - y_1)^2 = L_2^2. \end{cases} \quad (42)$$

Полученную систему дифференциально-алгебраических уравнений можно использовать для нахождения численного решения при заданных начальных условиях. Тем не менее, избыток искомых функций: $x_1(t), y_1(t), x_2(t), y_2(t)$ –, отсутствие явных выражений для производных, а также наличие алгебраических ограничений усложняет задачу и не позволяет использовать простые методы.

Важно отметить, что система (42) имеет лишь 2 степени свободы. Это особенно заметно, если от рассмотрения координат перейти к рассмотрению углов отклонения от оси Oy . Действительно, при заданных длинах стержней $L_1,$

L_2 положение центров масс однозначно определяется углами θ_1, θ_2 . Таким образом, следующим этапом станет «понижение размерности» системы (41) путём выражения координат x_1, y_1, x_2, y_2 через углы θ_1, θ_2 . Для этого воспользуемся формулами кинематики, тригонометрическими тождествами и правилами дифференцирования. Схематичная иллюстрация для соответствующих преобразований приведена на рисунке 64.

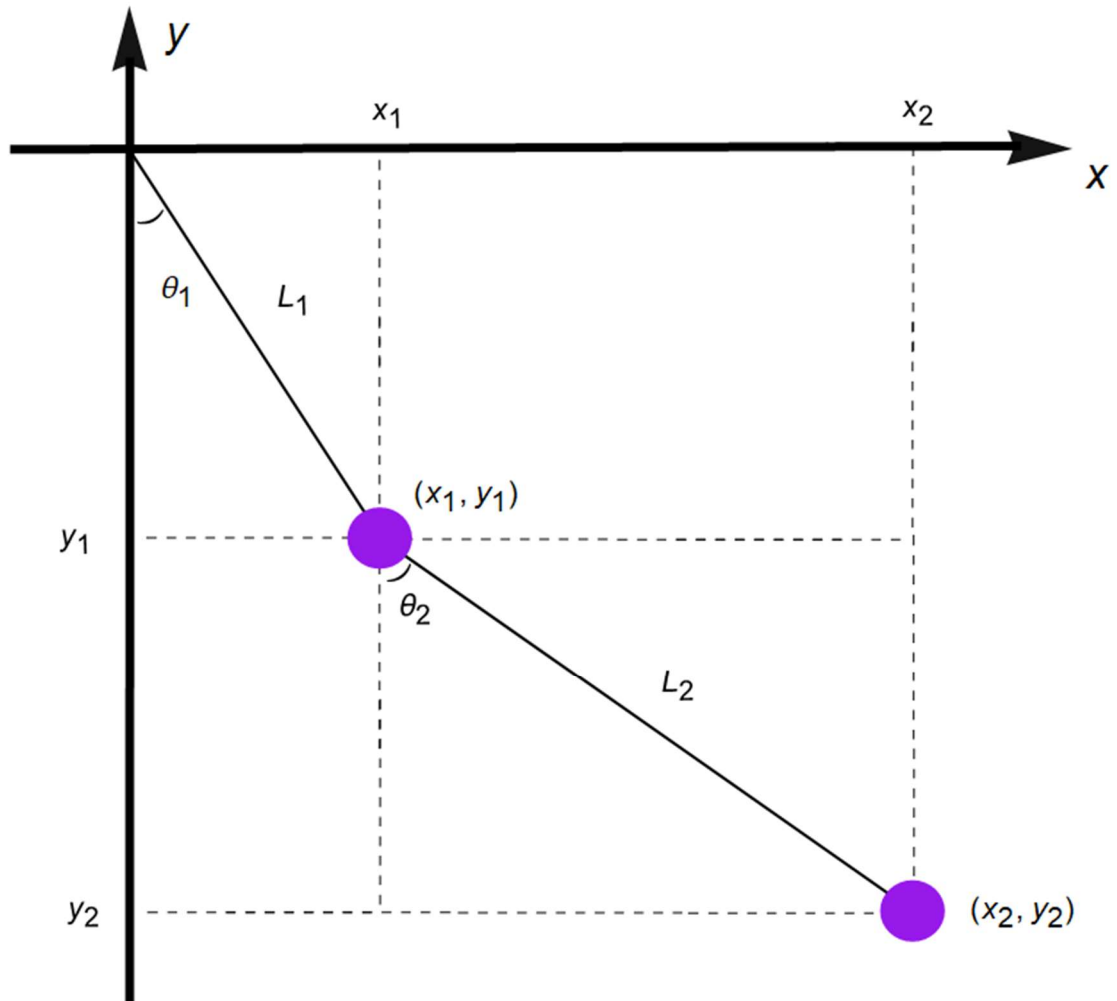


Рисунок 64 – Кинематика движения маятника

Выразим координаты через длины стержней и углы отклонения:

$$\begin{aligned}
 x_1 &= L_1 \sin \theta_1, \\
 y_1 &= -L_1 \cos \theta_1, \\
 x_2 &= x_1 + L_2 \sin \theta_2, \\
 y_2 &= y_1 - L_2 \cos \theta_2.
 \end{aligned}
 \tag{43}$$

Запишем выражения для производных первого порядка:

$$\begin{aligned}\dot{x}_1 &= \dot{\theta}_1 L_1 \cos \theta_1, \\ \dot{y}_1 &= \dot{\theta}_1 L_1 \sin \theta_1, \\ \dot{x}_2 &= \dot{x}_1 + \dot{\theta}_2 L_2 \cos \theta_2, \\ \dot{y}_2 &= \dot{y}_1 + \dot{\theta}_2 L_2 \sin \theta_2.\end{aligned}\tag{44}$$

И для производных второго порядка:

$$\begin{aligned}\ddot{x}_1 &= \ddot{\theta}_1 L_1 \cos \theta_1 - \dot{\theta}_1^2 L_1 \sin \theta_1, \\ \ddot{y}_1 &= \ddot{\theta}_1 L_1 \sin \theta_1 + \dot{\theta}_1^2 L_1 \cos \theta_1, \\ \ddot{x}_2 &= \ddot{x}_1 + \ddot{\theta}_2 L_2 \cos \theta_2 - \dot{\theta}_2^2 L_2 \sin \theta_2, \\ \ddot{y}_2 &= \ddot{y}_1 + \ddot{\theta}_2 L_2 \sin \theta_2 + \dot{\theta}_2^2 L_2 \cos \theta_2.\end{aligned}\tag{45}$$

Подставим выражения (45) в систему (41). После некоторых преобразований получим следующую систему:

$$\begin{cases} \ddot{\theta}_1 L_1 (m_1 + m_2) + \ddot{\theta}_2 L_2 m_2 \cos(\theta_1 - \theta_2) = \\ \quad = -\dot{\theta}_2^2 L_2 m_2 \sin(\theta_1 - \theta_2) - (m_1 + m_2)g \sin \theta_1, \\ \ddot{\theta}_1 L_1 m_2 \cos(\theta_1 - \theta_2) + \ddot{\theta}_2 L_2 m_2 = \dot{\theta}_1^2 L_1 m_2 \sin(\theta_1 - \theta_2) - m_2 g \sin \theta_2. \end{cases}\tag{46}$$

Найдём выражения для $\ddot{\theta}_1$ и $\ddot{\theta}_2$:

$$\begin{cases} \ddot{\theta}_1 = -\frac{m_1 g \sin \theta_1 + m_2 \sin(\theta_1 - \theta_2) (\dot{\theta}_1^2 L_1 \cos(\theta_1 - \theta_2) + \dot{\theta}_2^2 L_2 + g \cos \theta_2)}{L_1 (m_1 + m_2 \sin^2(\theta_1 - \theta_2))}, \\ \ddot{\theta}_2 = \frac{\sin(\theta_1 - \theta_2) ((m_1 + m_2)(g \cos \theta_1 + \dot{\theta}_1^2 L_1)) + \dot{\theta}_2^2 L_2 m_2 \cos(\theta_1 - \theta_2)}{L_2 (m_1 + m_2 \sin^2(\theta_1 - \theta_2))}. \end{cases}\tag{47}$$

Полученная система (47) представляет собой уравнения движения двойного математического маятника. К сожалению, интегрировать данные дифференциальные уравнения аналитически, чтобы явно найти $\theta_1(t)$ и $\theta_2(t)$, не представляется возможным. Тем не менее, можно воспользоваться одним из множества численных методов для нахождения приближённого решения системы. Для этого целесообразно понизить порядок системы.

Приведём систему (47) из двух ДУ второго порядка к системе из четырёх ДУ первого порядка:

$$\begin{cases} \dot{\theta}_1 = \omega_1, \\ \dot{\theta}_2 = \omega_2, \\ \dot{\omega}_1 = -\frac{m_1 g \sin \theta_1 + m_2 \sin(\theta_1 - \theta_2) (\omega_1^2 L_1 \cos(\theta_1 - \theta_2) + \omega_2^2 L_2 + g \cos \theta_2)}{L_1(m_1 + m_2 \sin^2(\theta_1 - \theta_2))}, \\ \dot{\omega}_2 = \frac{\sin(\theta_1 - \theta_2) ((m_1 + m_2)(g \cos \theta_1 + \omega_1^2 L_1)) + \omega_2^2 L_2 m_2 \cos(\theta_1 - \theta_2)}{L_2(m_1 + m_2 \sin^2(\theta_1 - \theta_2))}. \end{cases} \quad (48)$$

Стоит отметить, что введённые дополнительные переменные ω_1 , ω_2 имеют смысл угловых скоростей.

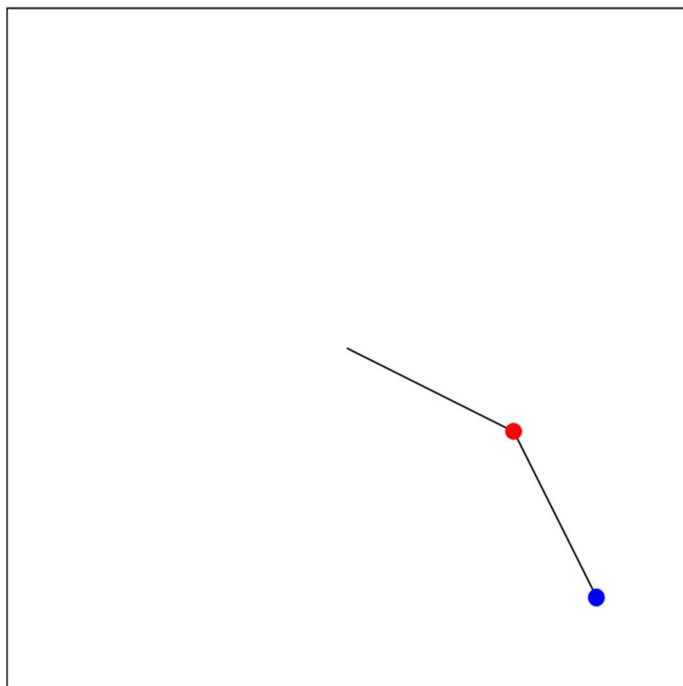
Каждая из полученных систем (42) и (47)-(48) описывает движение двойного математического маятника и представляет собой его математическую модель. Использование того или иного вида системы зависит исключительно от поставленных задач и выбранного метода численного решения задачи Коши.

Программная реализация в Wolfram Mathematica

На основе демонстрационного проекта Wolfram была реализована интерактивная версия модели двойного математического маятника, позволяющая по заданным пользователем параметрам m_1, m_2 и начальным условиям $x_1(0)$, $y_1(0)$, $x_2(0)$, $y_2(0)$ визуализировать временную эволюцию системы и траекторию движения центра масс «нижнего» маятника. Для описания движения двойного маятника были использованы системы дифференциально-алгебраических уравнений (40) и (42). Кроме того, был разработан аналогичный программный модуль, в котором движение двойного маятника задаётся системой (47). Преимущество данной реализации заключается в отсутствии алгебраических ограничений, что позволяет использовать простые численные методы.

Задание начального положения двойного маятника доступно путём перемещения центров масс с помощью мыши, а также путём задания координат с помощью бегунков с редактируемыми метками. Поле для задания параметров и исходного состояния двойного маятника демонстрируется на рисунке 65.

setStartPos[]



x1	1
y1	-0.5
x2	1.5
y2	-1.5
$\theta 1$	1.10715
$\theta 2$	0.463648
L1	1.11803
L2	1.11803
m1	1
m2	1

Рисунок 65 – Задание параметров и начального положения двойного маятника

Результат одного из запусков симуляции приведён на рисунке 66.

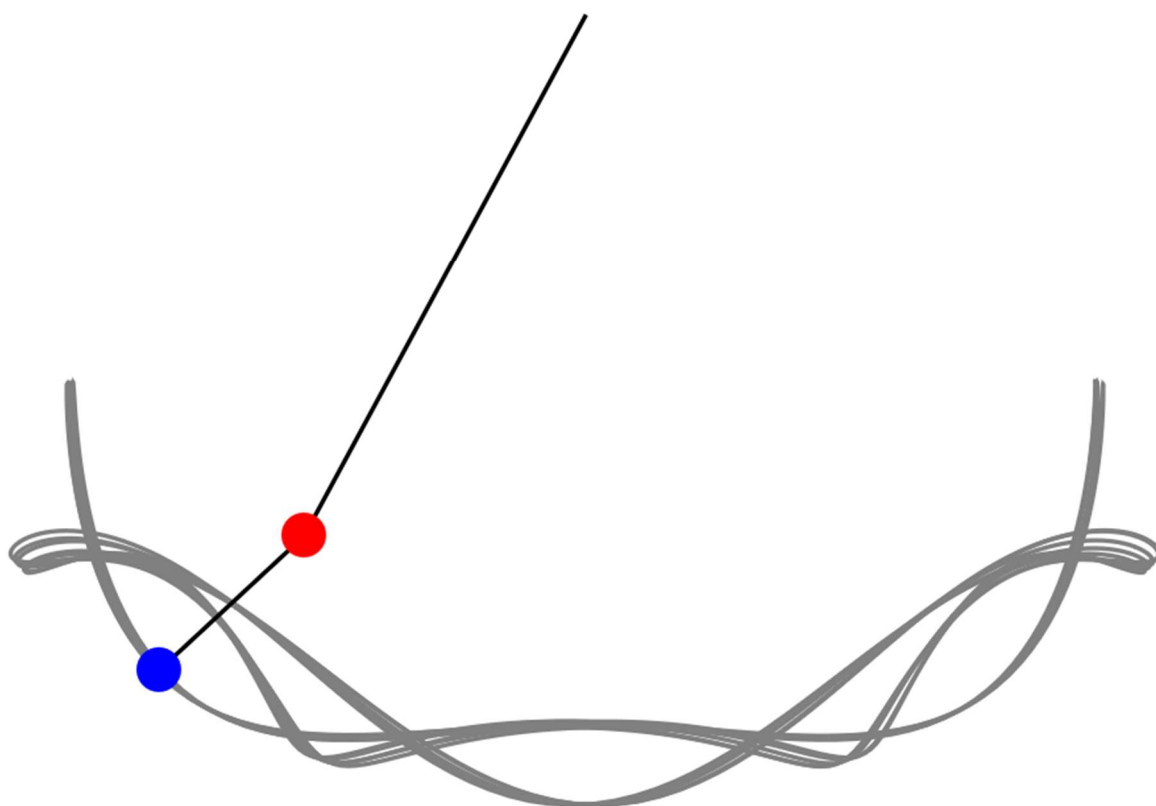


Рисунок 66 – Траектория движения двойного маятника

Из рисунка 67 можно заметить, что при значительной начальной энергии системы движение носит более хаотический характер.

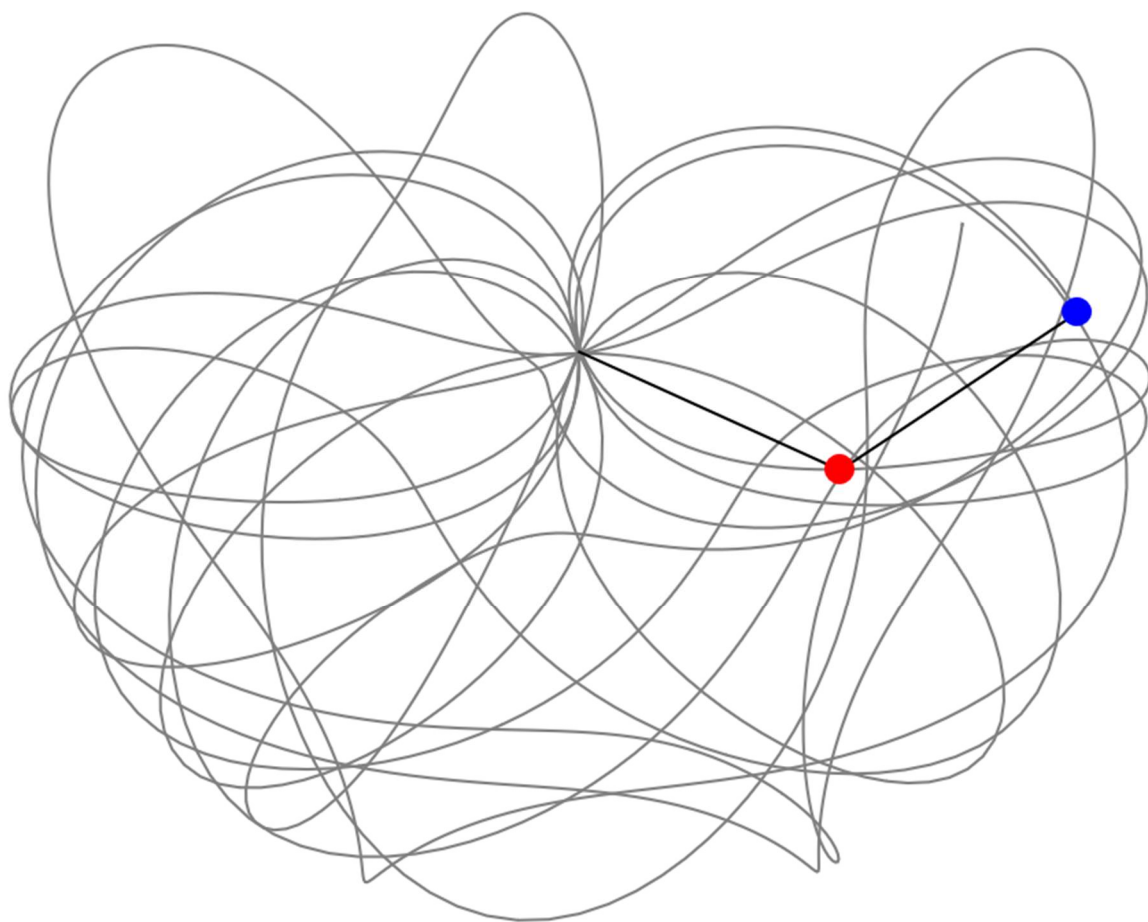


Рисунок 67 – Хаотическое движение двойного маятника

Программные модули, разработанные в системе Wolfram Mathematica, предпочтительно применять в целях визуализации движения двойного маятника, начальная энергия которого не слишком мала. Для исследования траекторий системы в различных фазовых плоскостях была создана имитационная модель в AnyLogic.

Программная реализация в AnyLogic

Для описания движения двойного маятника в AnyLogic была выбрана система обыкновенных дифференциальных уравнений первого порядка (48). Значения параметров m_1, m_2, L_1, L_2 , а также начальные условия $\theta_1(0), \theta_2(0), \omega_1(0), \omega_2(0)$ задаются с помощью соответствующих переменных.

Пример запуска созданной имитационной модели представлен на рисунке 68.

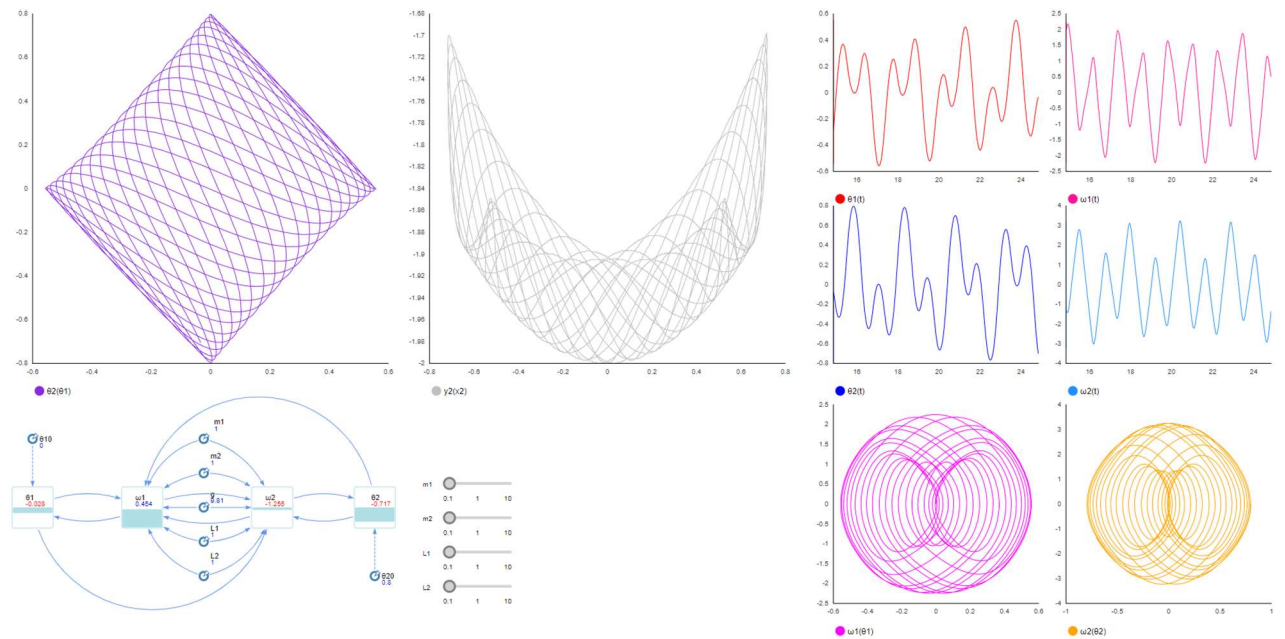


Рисунок 68 – Пример запуска модели в AnyLogic

Ещё один любопытный результат эксперимента демонстрируется на рисунке 69.

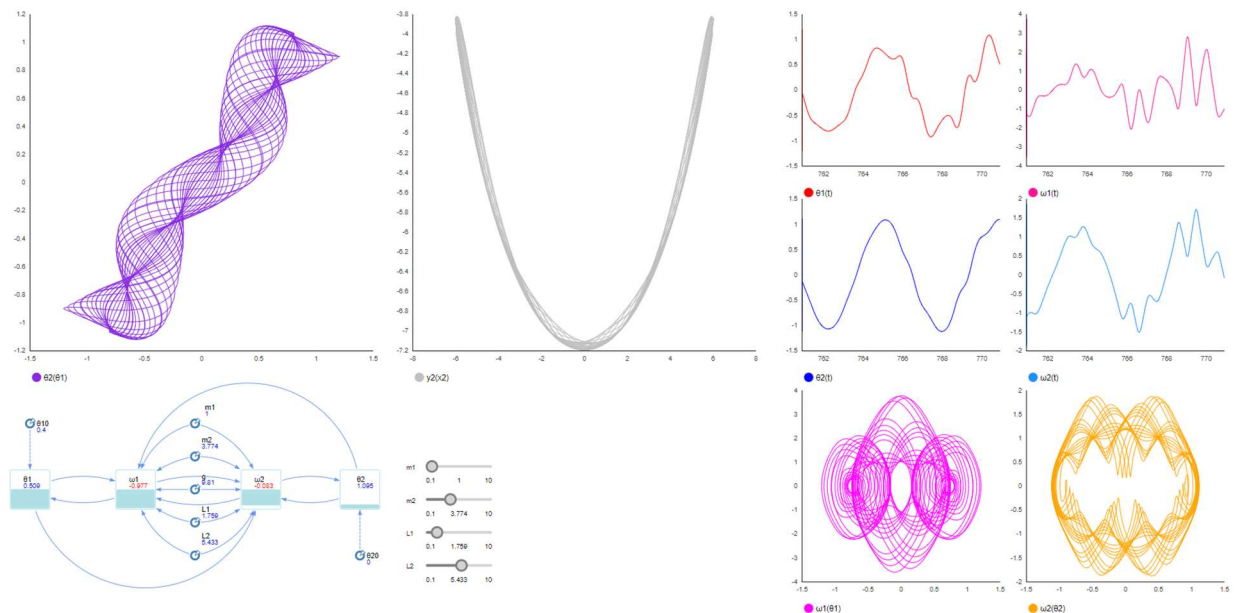


Рисунок 69 – Пример запуска модели в AnyLogic

На рисунке 70 показан вариант движения двойного маятника, близкий к детерминированному.

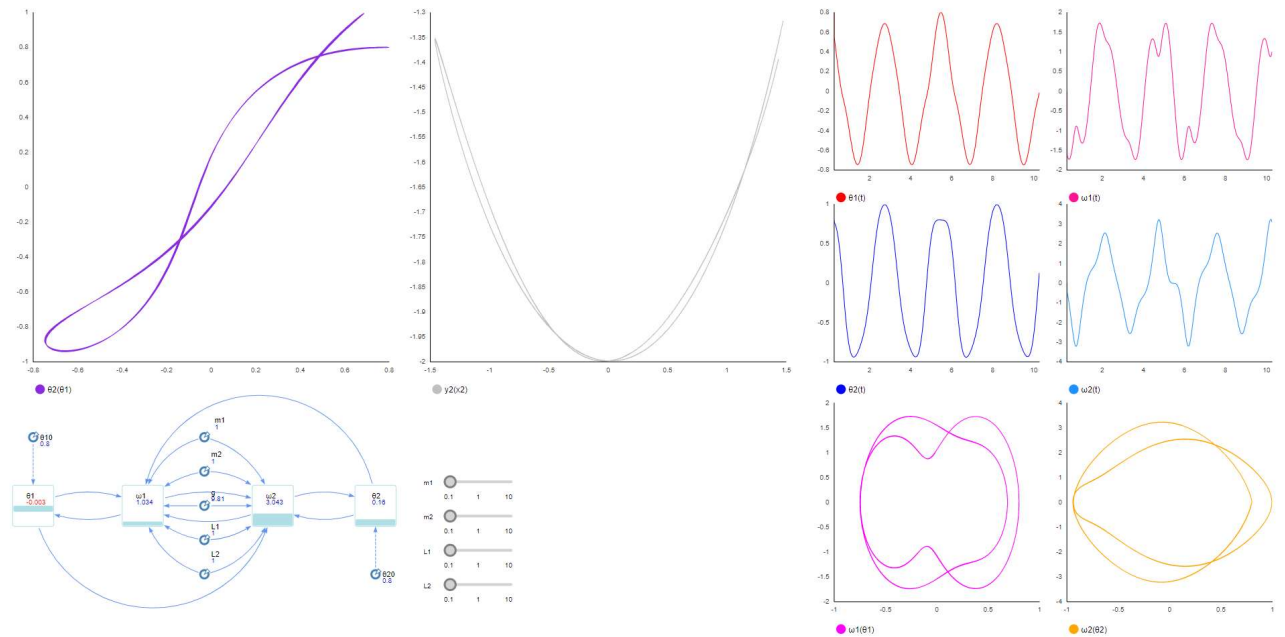


Рисунок 70 – Пример запуска модели в AnyLogic

Ещё один яркий пример временной эволюции двойного математического маятника отражён на рисунке 71.

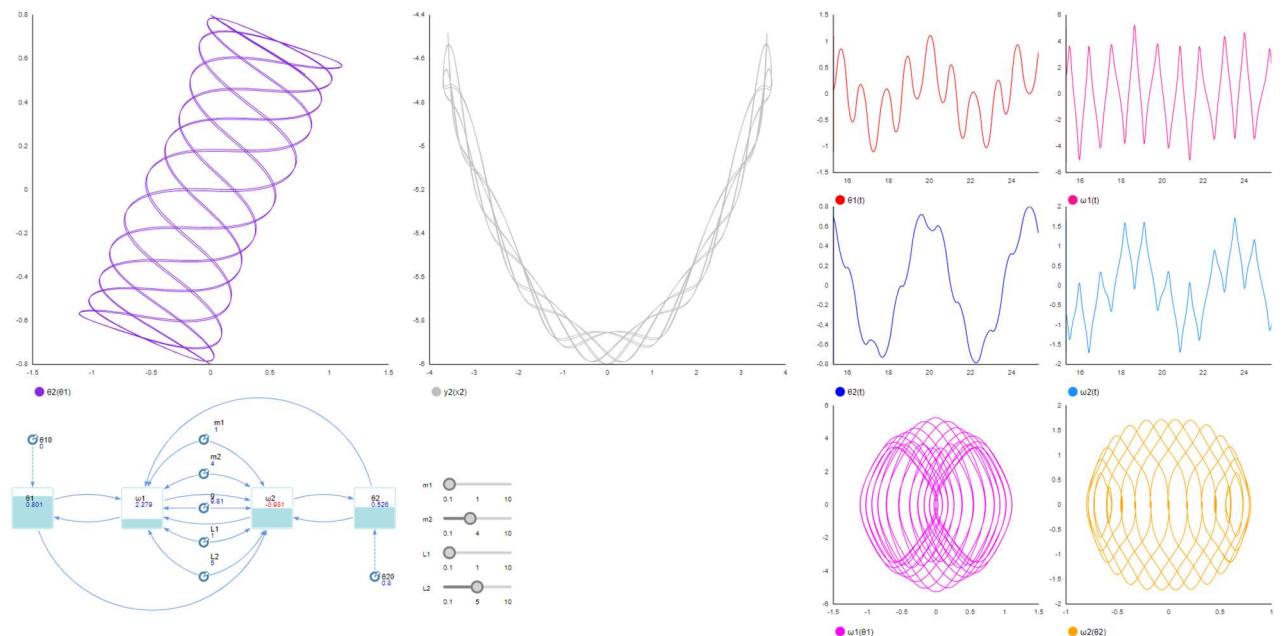


Рисунок 71 – Пример запуска модели в AnyLogic

Из приведённых выше рисунков можно заметить, что в ряде случаев проекция траектории на фазовую плоскость $\theta_2(\theta_1)$ напоминает *кривую Лиссажу*.

Кривые Лиссажу

Кривые Лиссажу (*Lissajous curves*) – это семейство кривых, описываемых системой параметрических уравнений вида:

$$\begin{cases} x(t) = A \cdot \cos(\omega_x t - \delta_x), \\ y(t) = B \cdot \cos(\omega_y t - \delta_y). \end{cases} \quad (49)$$

На рисунке 72 приведены несколько кривых Лиссажу в качестве примера.

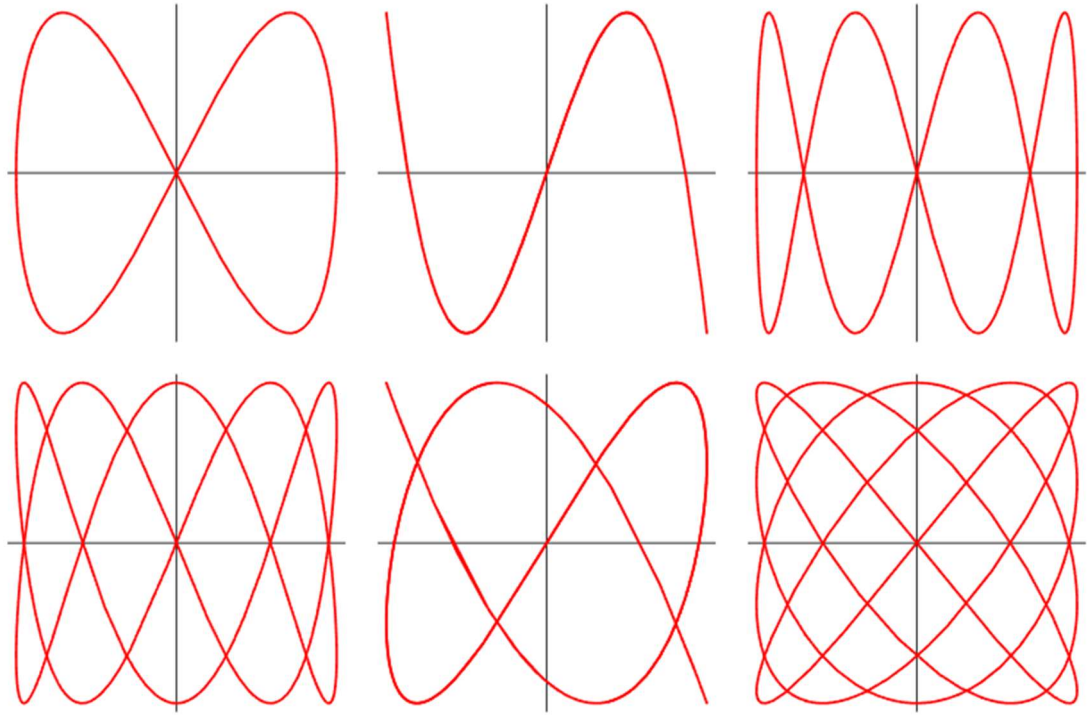


Рисунок 72 – Кривые Лиссажу

Выясним, как связаны кривые Лиссажу и ДУ 2 порядка. Для этого дважды продифференцируем уравнения (49).

$$\begin{cases} \ddot{x} = -\omega_x^2 \cdot A \cdot \cos(\omega_x t - \delta_x), \\ \ddot{y} = -\omega_y^2 \cdot B \cdot \cos(\omega_y t - \delta_y). \end{cases} \quad (50)$$

Заметим, что в правых частях стоят выражения для x и y из (49). Таким образом, получаем простую систему дифференциальных уравнений второго порядка, где ω_x , ω_y – частоты колебаний:

$$\begin{cases} \ddot{x} + \omega_x^2 \cdot x = 0, \\ \ddot{y} + \omega_y^2 \cdot y = 0. \end{cases} \quad (51)$$

Малые колебания. Линеаризация

Рассмотрим систему (46) в предположении малых колебаний: $\theta_1, \theta_2 \approx 0$, разностью $\theta_1 - \theta_2$ пренебрежём. Произведём замены по следующим правилам:

$$\begin{aligned}\sin \theta_1 &\rightarrow \theta_1, \\ \sin \theta_2 &\rightarrow \theta_2, \\ \sin(\theta_1 - \theta_2) &\rightarrow 0, \\ \cos(\theta_1 - \theta_2) &\rightarrow 1.\end{aligned}\tag{52}$$

В результате система примет вид:

$$\begin{cases} \ddot{\theta}_1 L_1 (m_1 + m_2) + \ddot{\theta}_2 L_2 m_2 = -(m_1 + m_2) g \theta_1, \\ \ddot{\theta}_1 L_1 m_2 + \ddot{\theta}_2 L_2 m_2 = -m_2 g \theta_2. \end{cases}\tag{53}$$

Выполним возможные преобразования:

$$\begin{cases} \ddot{\theta}_1 = \frac{g}{L_1 m_1} \cdot (m_2 \theta_2 - (m_1 + m_2) \cdot \theta_1), \\ \ddot{\theta}_2 = \frac{g}{L_2 m_1} \cdot (m_1 + m_2) \cdot (\theta_1 - \theta_2). \end{cases}\tag{54}$$

Система (54) легла в основу имитационной модели, реализованной в AnyLogic для исследования движения двойного маятника в случае малых колебаний. На рисунке 73 демонстрируется один из запусков симуляции.

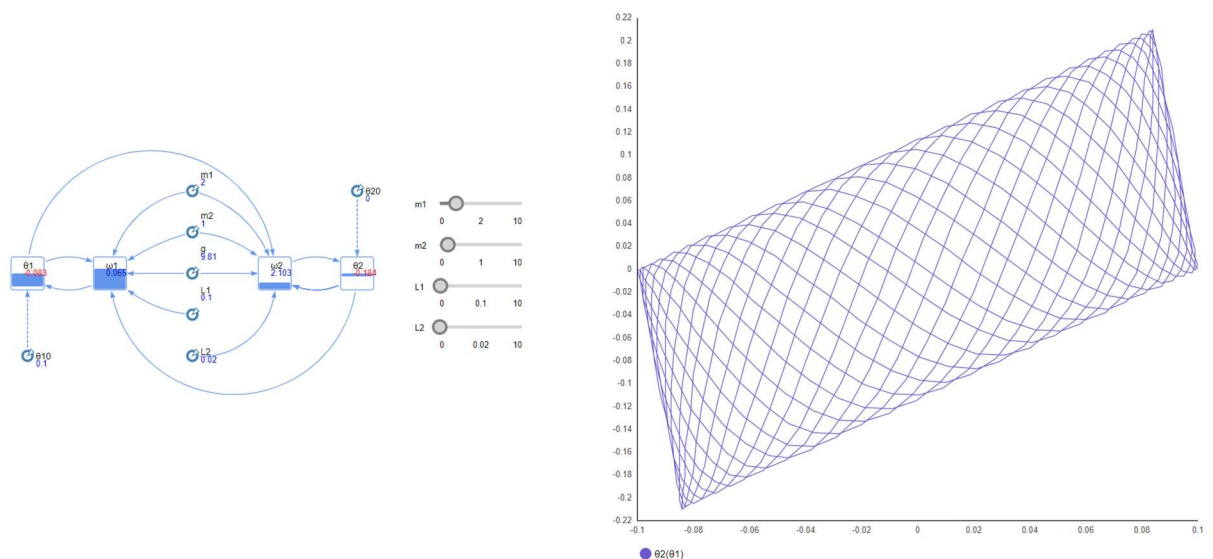


Рисунок 73 – Малые колебания двойного маятника

3.2. Связанные пружинные маятники

Описание модели (случай $N \geq 2$):

N идентичных брусков, расположенных в ряд на гладкой горизонтальной поверхности, соединены с соседними одинаковыми пружинками. Два крайние бруска прикреплены к вертикальным опорам пружинками, отличными от тех, которыми бруски соединены между собой. Движение происходит вдоль оси Ox , сопротивлением воздуха и трением пренебрегаем.

Схематичное изображение модели представлено на рисунке 74.



Рисунок 74 – Связанные пружинные маятники

Обозначения:

m – масса бруска;

K – коэффициент жёсткости пружины, соединяющей два бруска;

K_w – коэффициент жёсткости пружины, соединяющей брусок и опору;

x_i – смещение i -го бруска относительно своего положения покоя;

$v_i = \dot{x}_i$ – скорость движения i -го бруска.

Из II закона Ньютона для проекций на ось Ox получаем уравнения динамики системы:

$$\begin{cases} \ddot{x}_1 = k \cdot (x_2 - x_1) - k_w \cdot x_1, \\ \ddot{x}_n = k \cdot (x_{n+1} - x_n) - k \cdot (x_n - x_{n-1}), & 2 \leq n \leq N-1 \\ \ddot{x}_N = k \cdot (x_{N-1} - x_N) - k_w \cdot x_N. \end{cases} \quad (55)$$

где $k = \frac{K}{m}$;

$k_w = \frac{K_w}{m}$.

Для численного интегрирования целесообразно понизить порядок системы путём введения замены $v_i = \dot{x}_i$.

Задание 1

Реализовать имитационную модель двух связанных пружинных маятников средствами системной динамики AnyLogic. Построить графики $x_1(t)$, $x_2(t)$, а также фазовые портреты в плоскостях (x_1, x_2) , (x_1, v_1) , (x_2, v_2) . Реализовать анимацию движения маятников. Выполнить численный анализ на входных данных, приведённых в таблице 10.

Таблица 10 – Исходные данные для численного анализа

	1	2	3
$x_1(0)$	0,5	0,5	-0,5
$x_2(0)$	0	0,5	0,5
$v_1(0)$	0	0	0
$v_2(0)$	0	0	0
k	1	1	0,1
k_w	1	0,5	0,2

Реализация

В случае $N = 2$ система (55) примет следующий вид:

$$\begin{cases} \dot{x}_1 = v_1, \\ \dot{x}_2 = v_2, \\ \dot{v}_1 = k \cdot (x_2 - x_1) - k_w \cdot x_1, \\ \dot{v}_2 = k \cdot (x_1 - x_2) - k_w \cdot x_2. \end{cases} \quad (56)$$

Полученные уравнения были использованы для задания динамики соответствующих элементов имитационной модели. Для визуализации движения были дополнительно введены две переменные, содержащие начальные координаты центров масс. На рисунке 75 отражён запуск симуляции на первом наборе данных из приведённых в таблице 10.

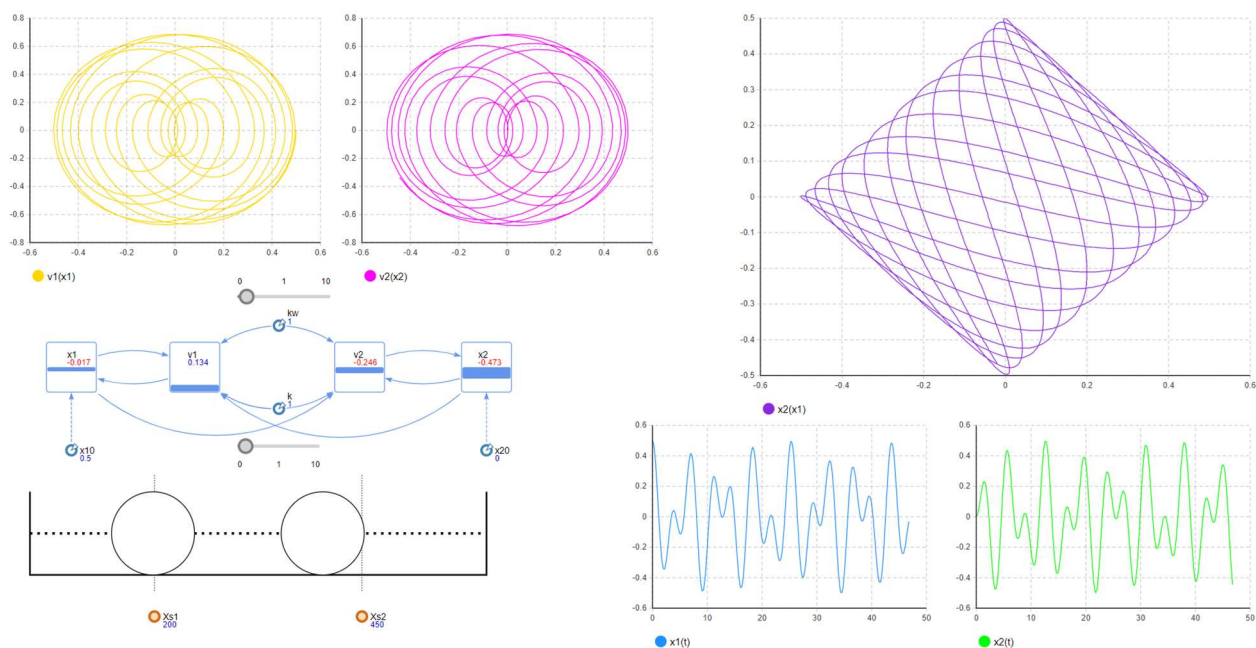


Рисунок 75 – Численный анализ на первом наборе данных

Любопытные результаты, отражённые на рисунке 76, были получены во время проведения эксперимента на втором наборе данных.

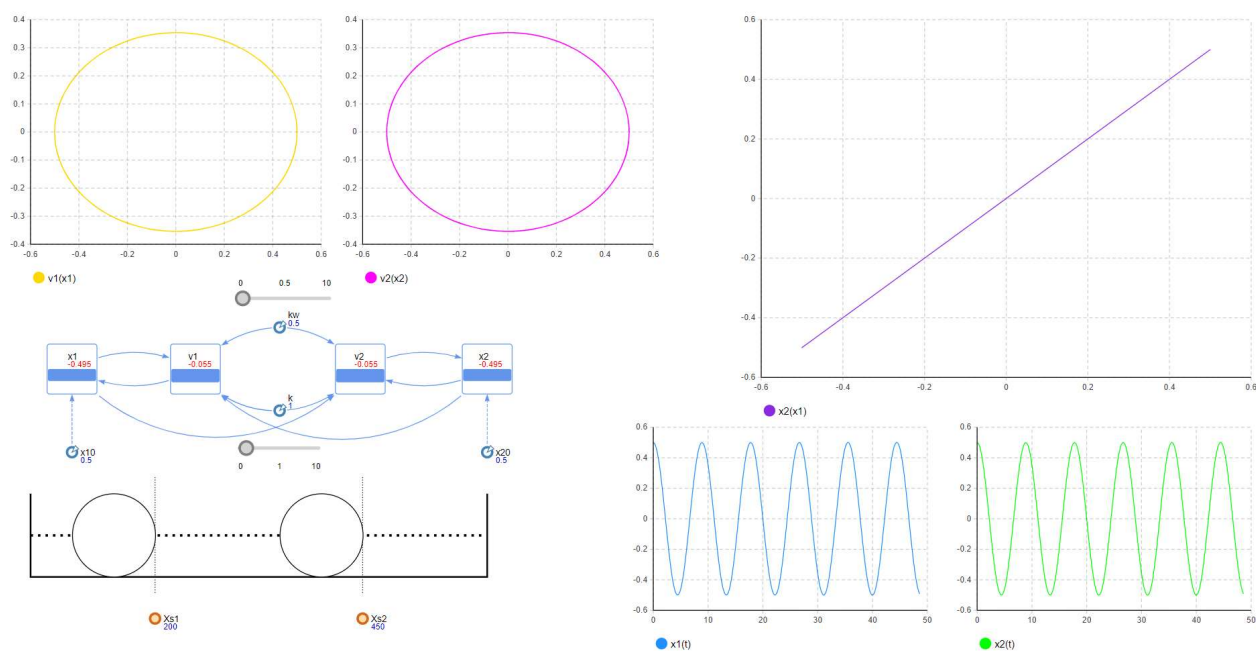


Рисунок 76 – Численный анализ на втором наборе данных

Из рисунка можно заметить, что движение центров масс было синхронным. В данном случае пружина, соединяющая грузики, в любой момент времени находилась в недеформированном состоянии и не оказывала воздействия на движение центров масс, как если бы она отсутствовала. Таким образом, значение k при прочих неизменных параметрах и начальных данных не играет роли.

На третьем наборе данных движение центров масс в любой момент времени было симметричным относительно центра пружины, их соединяющей, что отражено на графиках, приведённых на рисунке 77.

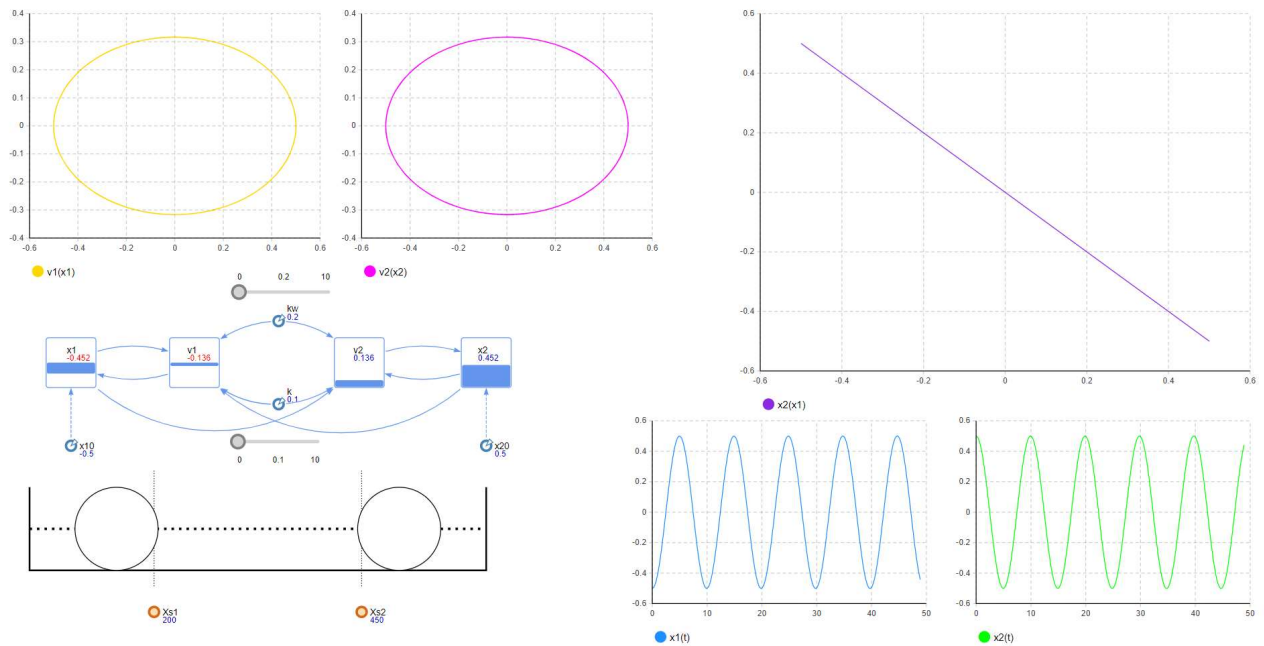


Рисунок 77 – Численный анализ на третьем наборе данных

В общем случае фазовый портрет Лиссажу в плоскости (x_1, x_2) напоминает кривую Лиссажу.

Задание 2

Реализовать модель в общем случае для $N \geq 3$ связанных пружинных маятников с помощью языка программирования Python. Создать анимацию движения центров масс.

Реализация

Модель N связанных пружинных маятников была реализована в виде класса, атрибутами которого являются количество грузиков, а также параметры k и k_w . Функции динамики были заданы согласно уравнениям системы (55) как методы класса. Исходный код инициализации объекта класса, а также вышеописанных методов представлен на рисунке 78.


```

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from NSolveDE import RK4_auto_system_XY, Euler_auto_system_XY

class SpringMassSystem(object):
    def __init__(self, n=3, k=1, kw=1):
        if n < 3:
            raise ValueError('Invalid argument values: n < 3')
        self.n = n
        self.k = k
        self.kw = kw

    def dx(self, x, v, args):
        return v

    def dv(self, center, v, args):
        left = args[0]
        right = args[1]
        i = args[2]
        if i == 1:
            return self.k * (right - center) - self.kw * center
        elif 1 < i < self.n:
            return self.k * (right - center) - self.k * (center - left)
        elif i == self.n:
            return self.k * (left - center) - self.kw * center
        else:
            raise ValueError('Invalid argument values: "i" out of bounds [1, n]')

    def get_integral_curves(self, x0, v0, t0, T, dt, method='RK4'):
        if method == 'RK4':
            get_next = RK4_auto_system_XY
        elif method == 'Euler':
            get_next = Euler_auto_system_XY
        else:
            print('Invalid method value. RK4 method used instead. Available options are "RK4" and "Euler"')
            get_next = RK4_auto_system_XY

        time = np.arange(t0, T, dt)
        m = time.shape[0]
        x = np.empty([self.n, m])
        v = np.empty([self.n, m])
        for i in range(self.n):
            x[i][0] = x0[i]
            v[i][0] = v0[i]

        for t in range(m - 1):
            x[0][t + 1], v[0][t + 1] = get_next(self.dx, self.dv,
                                                x[0][t], v[0][t],
                                                dt,
                                                [v[0][t]],
                                                [None, x[1][t], 1])

            for i in range(1, self.n - 1):
                x[i][t + 1], v[i][t + 1] = get_next(self.dx, self.dv,
                                                    x[i][t], v[i][t],
                                                    dt,
                                                    [v[i][t]],
                                                    [x[i - 1][t], x[i + 1][t], i + 1])

            x[self.n - 1][t + 1], v[self.n - 1][t + 1] = get_next(self.dx, self.dv,
                                                                x[self.n - 1][t], v[self.n - 1][t],
                                                                dt,
                                                                [v[self.n - 1][t]],
                                                                [x[self.n - 2][t], None, self.n])

        return time, x, v

```

Рисунок 78 – Инициализация и ключевые методы класса

Программный код метода, отвечающего за отрисовку анимации движения центров масс, приведён на рисунке 79.

```
def plot_animation(self, x0, v0, t0=0.0, T=10.0, dt=0.01, method='RK4', x_max=10):
    time, x, v = self.get_integral_curves(x0, v0, t0, T, dt, method)
    fig, ax = plt.subplots()
    ax.set_xlim([0, x_max])
    start = np.linspace(0, x_max, self.n + 2)[1:-1]
    txc_array = x.transpose() + start
    ax.plot((0, x_max), (0, 0), color='black', linestyle='dashed', linewidth=1, zorder=-1)
    scat = ax.scatter(txc_array[0], (0,) * self.n, s=20 ** 2, c=np.random.rand(self.n), alpha=1)

    def animate(i):
        current_coordinates = [(x, 0) for x in txc_array[i]]
        scat.set_offsets(current_coordinates)
        return (scat,)

    ani = animation.FuncAnimation(fig, animate, repeat=False, frames=time.shape[0]-1, interval=50)
    plt.show()
    return x

n = 7
x0 = 10 * np.random.rand(n)
v0 = [0] * n
model = SpringMassSystem(n=n, k=1, kw=2)
model.plot_animation(x0=x0, v0=v0, t0=0, T=25, dt=0.1, method='RK4', x_max=200)
```

Рисунок 79 – Отрисовка анимации и пример создания объекта класса

На рисунке 80 демонстрируется запуск программы для случая $N = 7, k = 1, k_w = 2$. Начальные смещения $x_i(0)$ были заданы случайным образом, а скорости приняты равными нулю.

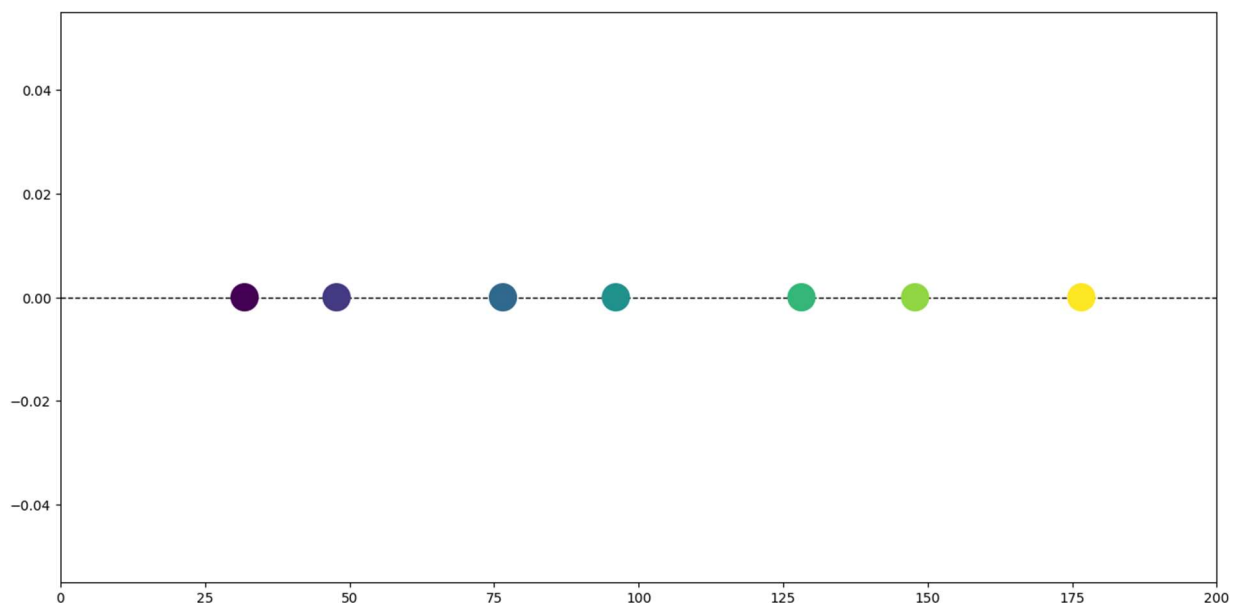


Рисунок 80 – Пример запуска программы

Стоит отметить, что реализация в Python является более универсальной, поскольку позволяет исследовать модель при любых значениях N .

3.3. Маятник Фуко

Маятник Фуко – математический маятник, предназначенный для экспериментальной демонстрации вращения Земли вокруг своей оси. Представляет собой тело массой до нескольких десятков килограммов, подвешенное на нерастяжимой нити или невесомом стержне длиной до нескольких десятков метров.

Движение центра масс маятника Фуко описывается следующей системой ОДУ первого порядка:

$$\begin{cases} \dot{x} = v_x, \\ \dot{y} = v_y, \\ \dot{v}_x = 2v_y\omega + \omega^2x - g\frac{x}{L}, \\ \dot{v}_y = 2v_x\omega + \omega^2y - g\frac{y}{L}. \end{cases} \quad (57)$$

где $\omega = 2\Omega \sin \lambda$ – вертикальная составляющая угловой скорости Земли;

$\Omega = \frac{2\pi}{86400}$ – угловая скорость вращения Земли вокруг своей оси;

λ – географическая широта (в радианах);

g – ускорение свободного падения;

L – длина подвеса.

Задание

Реализовать имитационную модель маятника Фуко средствами системной динамики AnyLogic. Отобразить графики $x(t), y(t)$, а также фазовые портреты в плоскостях $(x, v_x), (y, v_y), (x, y)$. Выполнить численный анализ модели с параметрами $\omega = 0.04, L = 100$.

Реализация

Переменные x, y, v_x, v_y были реализованы как накопители с динамикой, задаваемой соответствующими уравнениями системы (57). На рисунке 81 отражено состояние симуляции по прошествии 180 единиц модельного времени после запуска с начальными данными $x(0) = 10, y(0) = v_x(0) = v_y(0) = 0$.

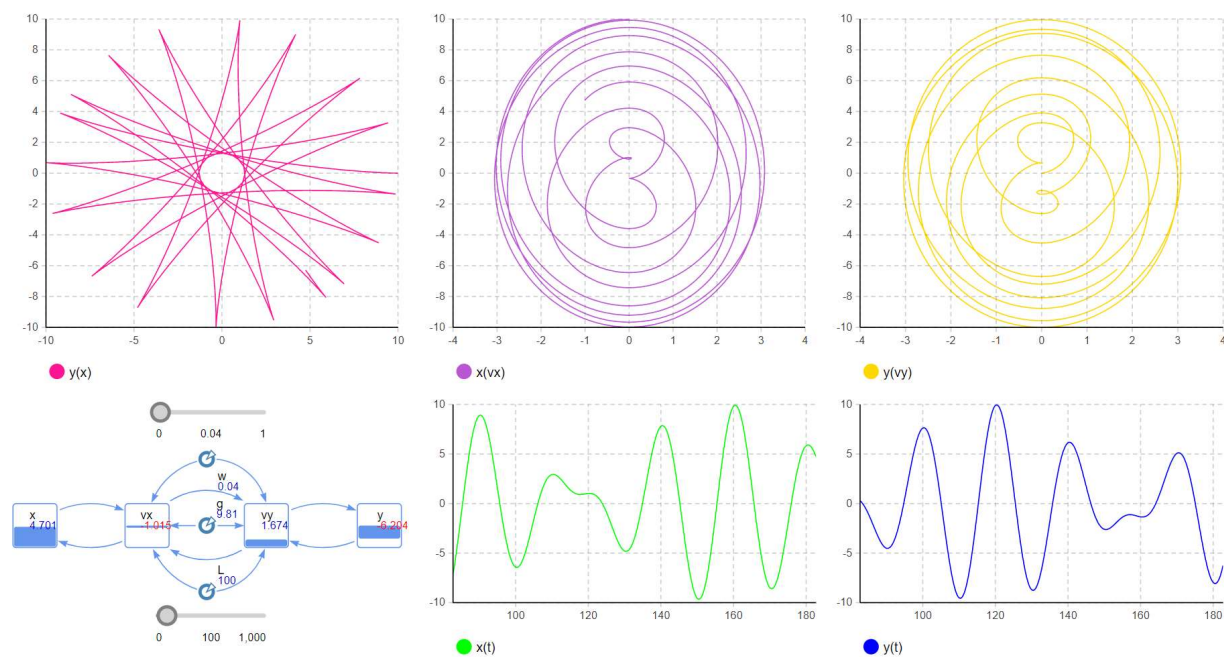


Рисунок 81 – Маятник Фуко в AnyLogic ($t = 180$)

На рисунке 82 демонстрируется состояние модели в режиме виртуального времени при отображении 1000 последних значений.

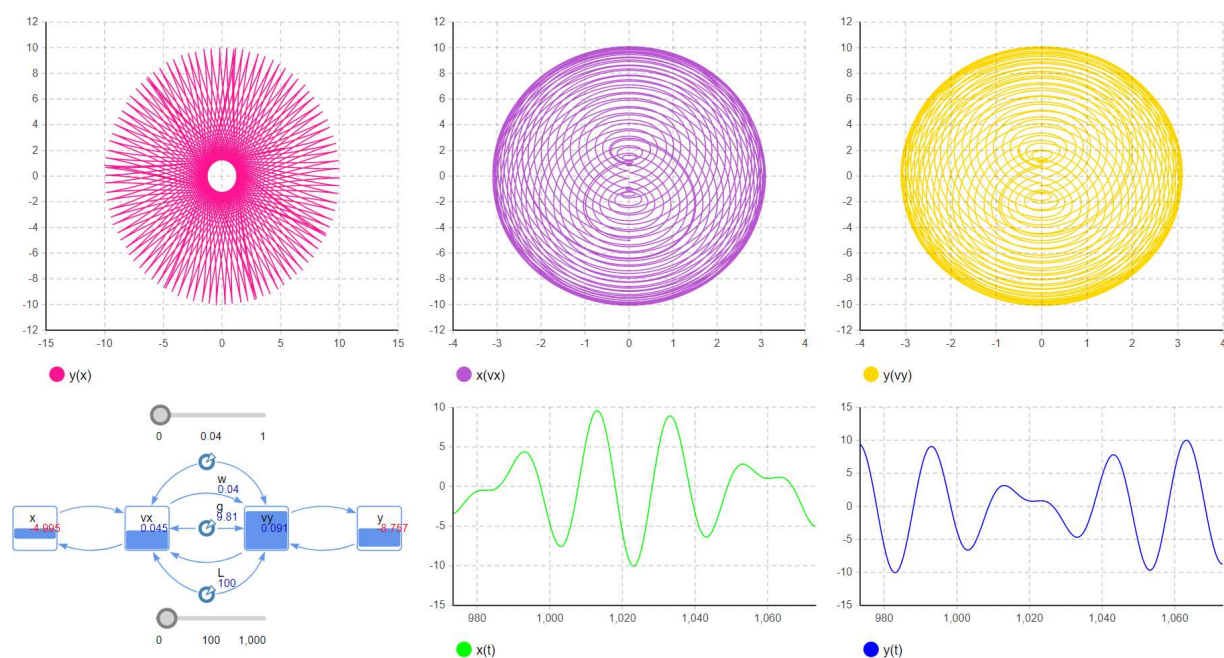


Рисунок 82 – Маятник Фуко в AnyLogic ($t = 1000$)

Можно заметить, что при $\omega = 0$ маятник не отклоняется от траектории, проекция которой на плоскость XU имеет вид отрезка.

3.4. Симпатические маятники

Симпатические маятники – колебательная система, представляющая собой два математических маятника, соединённых пружиной.

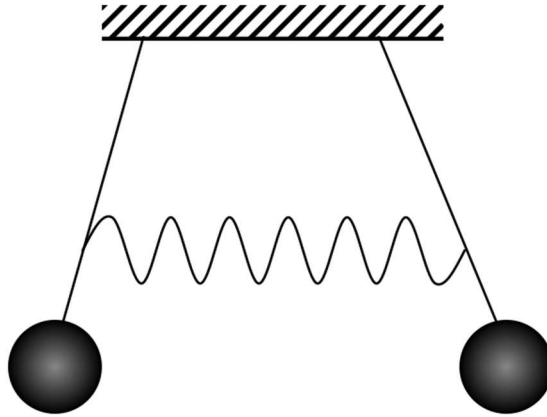


Рисунок 83 – Схема симпатических маятников

Описание и предположения модели:

Пусть маятники не идентичны (расстроены): массы неодинаковы, а длины стержней отличаются незначительно. Движение происходит в одной плоскости, сопротивлением воздуха и потерей энергии системы пренебрегаем. Рассмотрим систему в предположении малых колебаний, когда углы отклонения маятников от положения равновесия достаточно малы ($\theta_1, \theta_2 \approx 0$). Тогда имеют место следующие допущения, которые значительно упрощают вывод уравнений движения из II закона Ньютона: $\sin \theta_1 \rightarrow \theta_1, \sin \theta_2 \rightarrow \theta_2, \cos \theta_1 \rightarrow 1, \cos \theta_2 \rightarrow 1$.

В общем случае движение центров масс задаётся системой ОДУ первого порядка, приведённой ниже:

$$\begin{cases} \dot{x}_1 = v_1, \\ \dot{x}_2 = v_2, \\ \dot{v}_1 = -\omega_1^2 \cdot x_1 - k_1 \cdot (x_1 - x_2), \\ \dot{v}_2 = -\omega_2^2 \cdot x_2 - k_2 \cdot (x_2 - x_1). \end{cases} \quad (58)$$

где $\omega_i = \sqrt{\frac{g}{L_i}}$ – угловая скорость i -го маятника;

$k_i = \frac{K}{m_i}$ – отношение коэффициента жёсткости пружины K к массе маятника m_i .

Задание

Выполнить численный анализ симпатических маятников с параметрами $\omega_1 = 1.1$, $\omega_2 = 1.09$, $k_1 = k_2 = k = 0.1$ и начальными данными $x_1(0) = 0$, $x_2(0) = 1.5$, $v_1(0) = v_2(0) = 0$. Построить временные графики $x_1(t)$, $x_2(t)$ и фазовый портрет в плоскости (x_1, x_2) .

Реализация

Модель расстроенных симпатических маятников с заданными параметрами была построена средствами системной динамики AnyLogic. На рисунке 84 демонстрируется временной график, полученный в результате эксперимента с начальными значениями согласно условию задания.

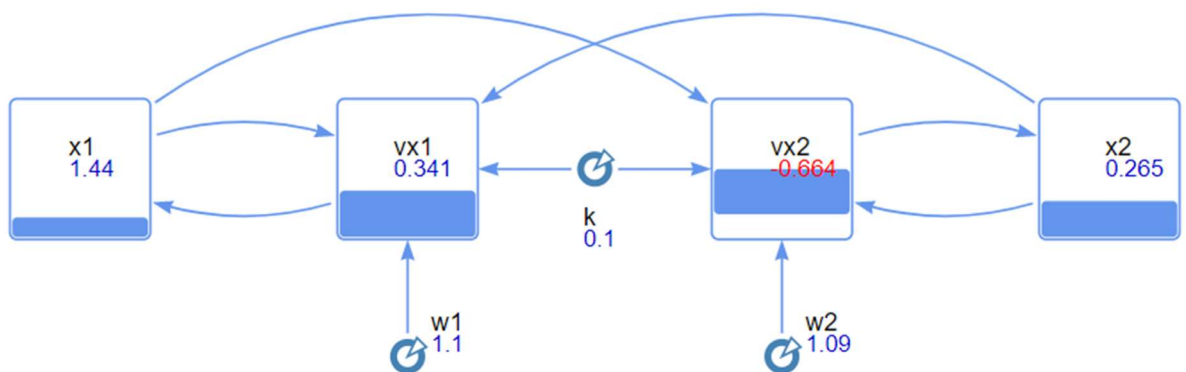
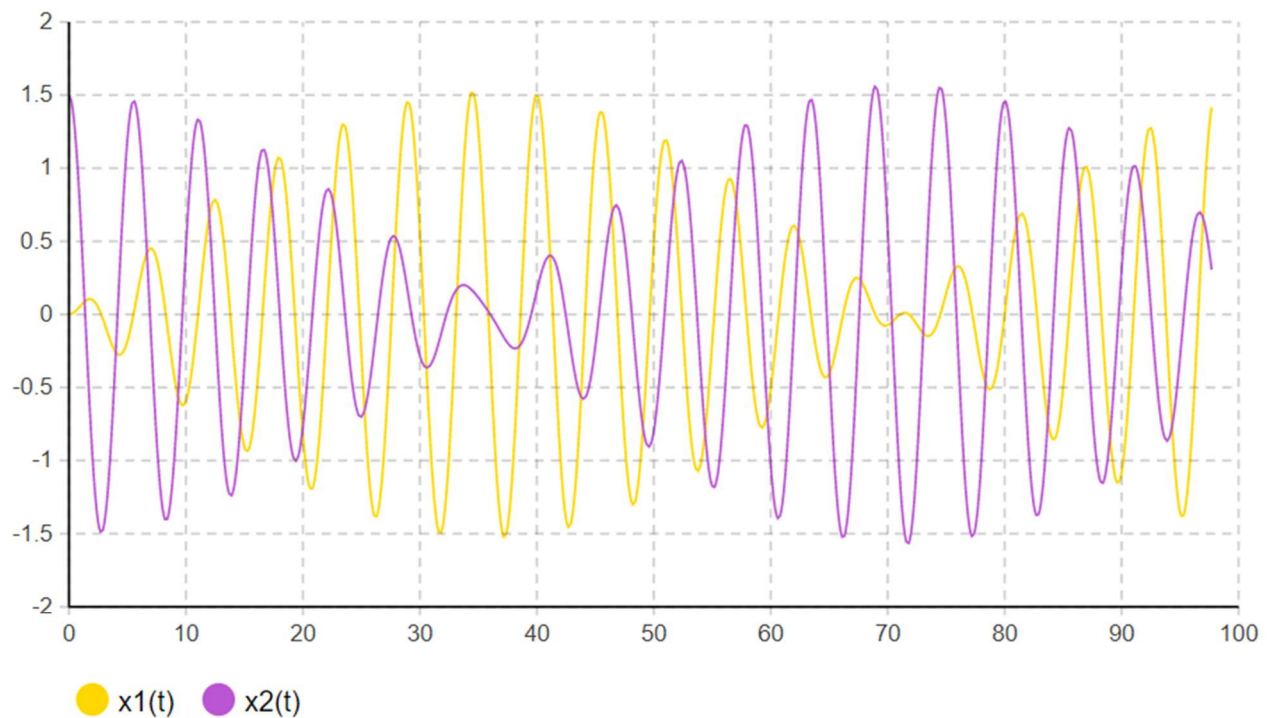


Рисунок 84 – График временной эволюции симпатических маятников

График в фазовой плоскости (x_1, x_2) , отражённый на рисунке 85, имеет вид фигуры Лиссажу.

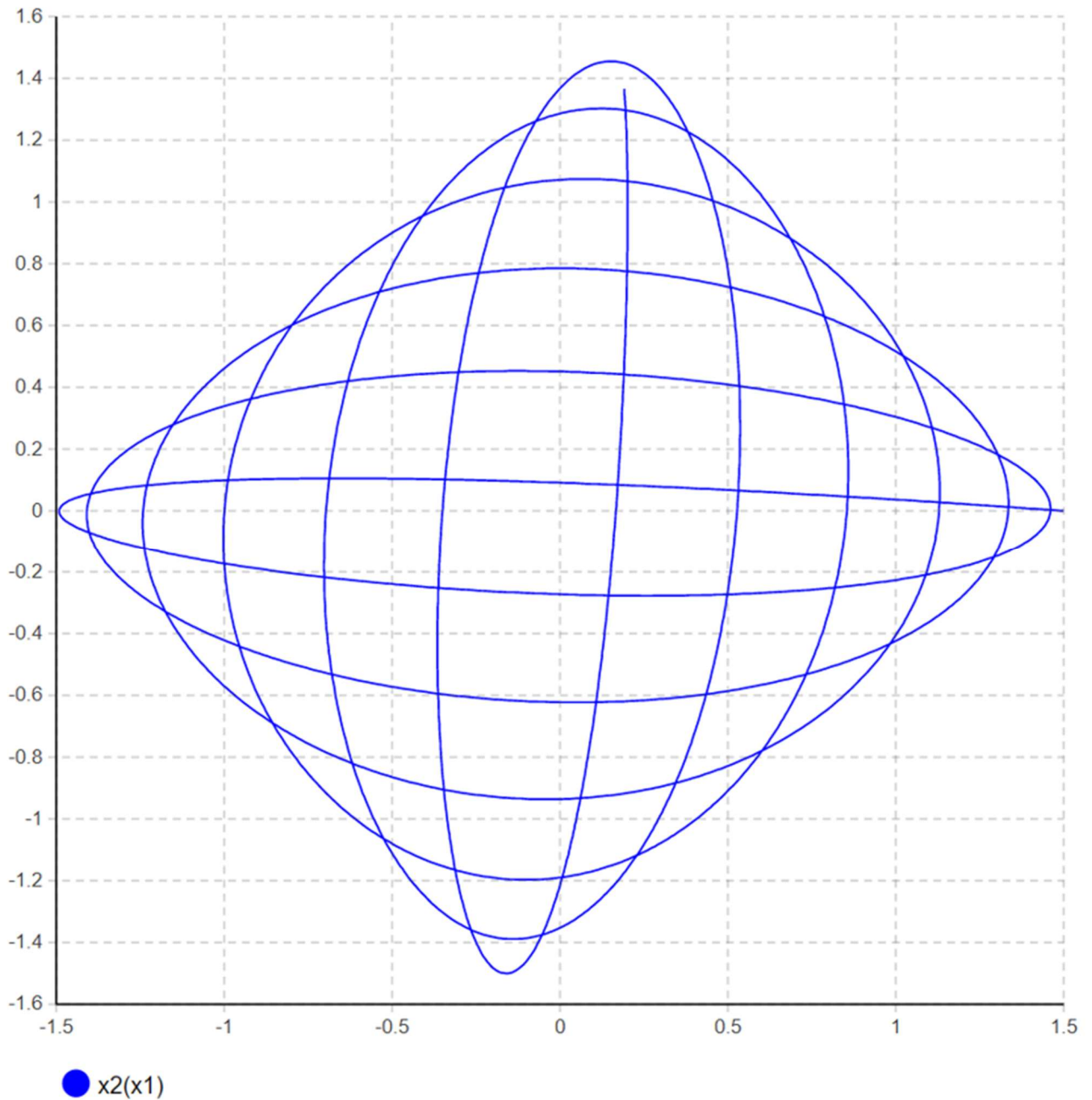


Рисунок 85 – Фазовый портрет

Варьирование $x_2(0)$ при прочих равных оказывает влияние на амплитуду колебаний маятников. При $\omega_1 = \omega_2$ колебания происходят в резонансе.

4. ДИНАМИЧЕСКИЕ МОДЕЛИ ЭКОНОМИКИ

4.1. Модель Самуэльсона-Хикса

Модель Самуэльсона-Хикса (модель мультипликатора-акселератора) – это динамическая экономическая модель, связывающая экономические циклы с взаимодействием мультипликатора инвестиций (большой рост выпуска по сравнению с вызвавшим его ростом инвестиций) и акселератора (увеличение инвестиций, индуцированное ростом выпуска).

Обозначения:

- Y – валовый внутренний продукт (ВВП);
- I – инвестиции;
- C – конечное потребление;
- C_a – автономное потребление;
- I_a – автономные инвестиции.

Предположения модели:

- экономика замкнута;
- предложение эластично;
- цены и процентная ставка фиксированы;
- потребление в текущем периоде определяется доходом в предшествующем периоде, спрос зависит от $Y(t)$ линейно:

$$Y(t + 1) = C_a + c \cdot Y(t) + I(t) \quad (59)$$

где c – норма потребления, $0 < c < 1$;

- динамика инвестиций определяется динамикой ВВП:

$$I(t) = r \cdot (Y(t) - Y(t - 1)) + I_a \quad (60)$$

где r – коэффициент акселерации, $r > 0$;

Из данных предположений следует, что динамика внутреннего валового продукта в модели Самуэльсона-Хикса выражается дифференциальным уравнением второго порядка:

$$\frac{d^2Y}{dt^2} + (1-r) \cdot \frac{dY}{dt} + (1-c) \cdot Y(t) = C_a + I_a \quad (61)$$

Уравнение (61) можно записать в виде системы ОДУ первого порядка:

$$\begin{cases} \dot{y} = u, \\ \dot{u} = -(1-r) \cdot u - (1-c) \cdot y + A, \end{cases} \quad (62)$$

где $y(t) = Y(t)$;

$$A = C_a + I_a.$$

Задание 1

Провести качественный анализ модели Самуэльсона-Хикса.

Решение

Путём приравнивания правых частей уравнений системы (62) к нулю была получена точка равновесия:

$$y^* = \frac{A}{1-c} \quad (63)$$

Затем была найдена матрица линеаризованной системы M и вычислены её основные характеристики: $\det M = 1 - c$, $\text{tr } M = r - 1$, опираясь на которые были определены возможные типы точки покоя (63), отражённые в таблице 11.

Таблица 11 – Варианты фазового портрета в зависимости от параметров модели

Тип точки и характер устойчивости	Условие
устойчивый узел	$0 < r < 1 - 2\sqrt{1-c}$, $c > 0.75$
устойчивый фокус	$1 - 2\sqrt{1-c} < r < 1$
фокус	$r = 1$
неустойчивый фокус	$1 < r < 1 + 2\sqrt{1-c}$
неустойчивый узел	$r > 1 + 2\sqrt{1-c}$

Фазовые портреты для рассмотренных случаев отражены на рисунке 86.

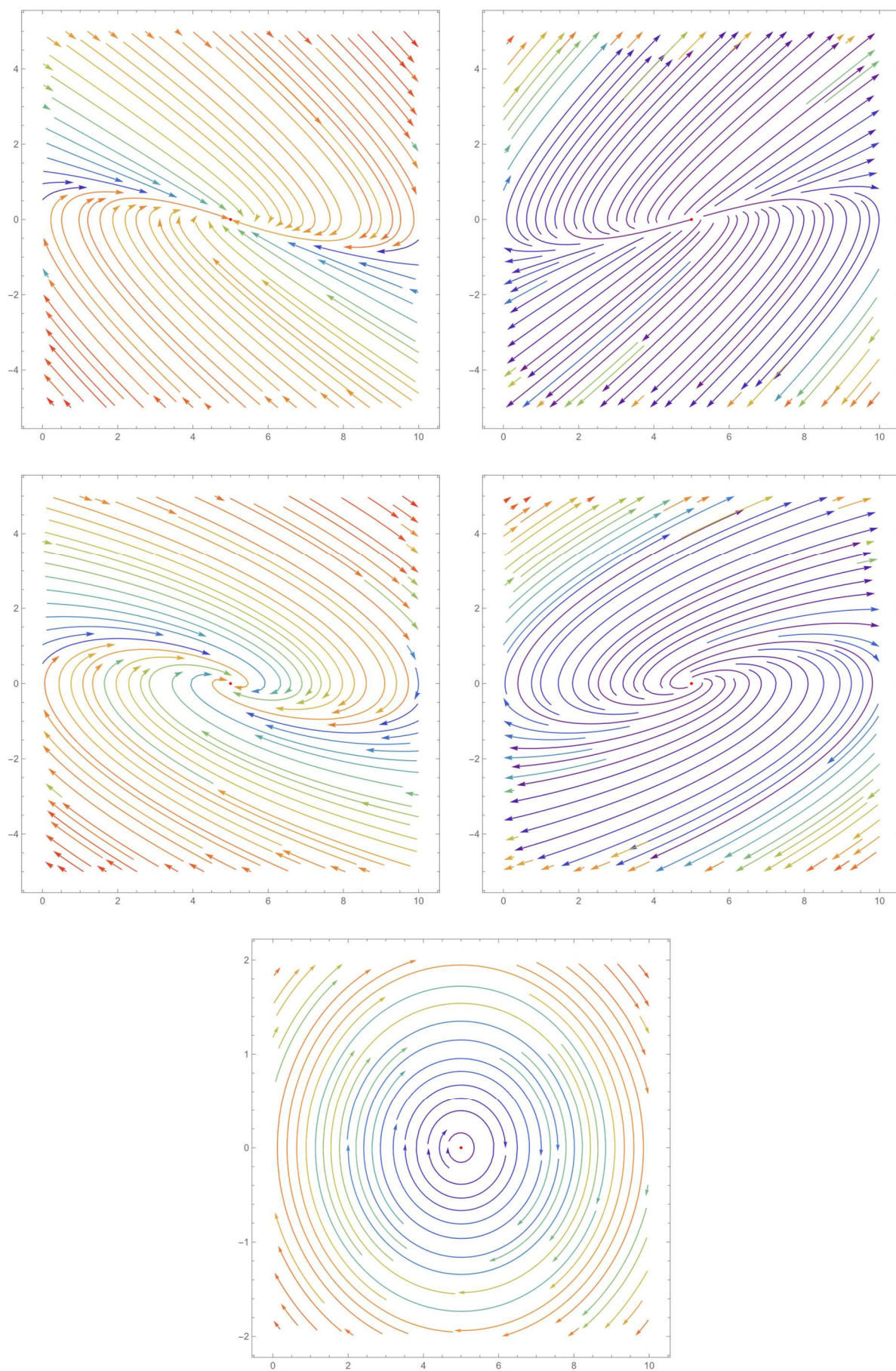


Рисунок 86 – Фазовые портреты

Задание 2

Выполнить численный анализ модели Самуэльсона-Хикса.

Решение

Для численного исследования модели средствами системной динамики AnyLogic была построена имитационная модель. Путём варьирования значения коэффициента акселерации r были рассмотрены возможные варианты развития экономики. Значения $c = 0.8, A = 1, y(0) = 8, u(0) = 0$ были выбраны по собственному усмотрению исходя из экономического смысла параметров. Стационарное значение ВВП оказалось равным 5. На рисунке 87 отражён случай незатухающих гармонических колебаний ВВП около положения равновесия (63).

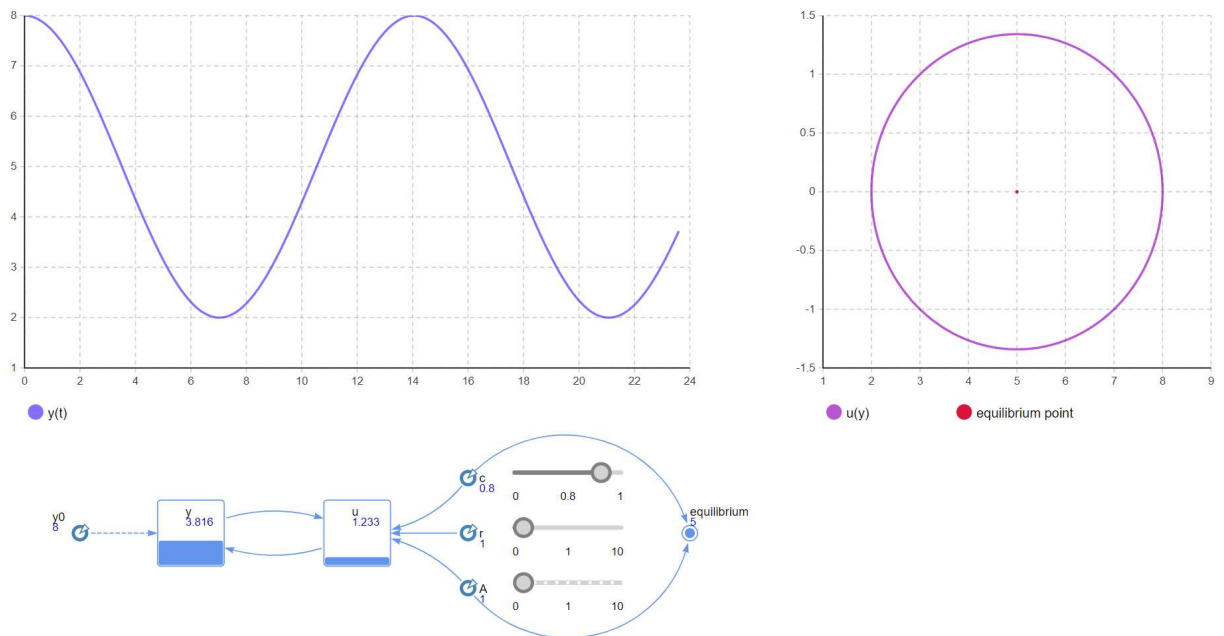


Рисунок 87 – Центр при $r = 1$

На рисунке 88 демонстрируется стремление к устойчивому узлу.

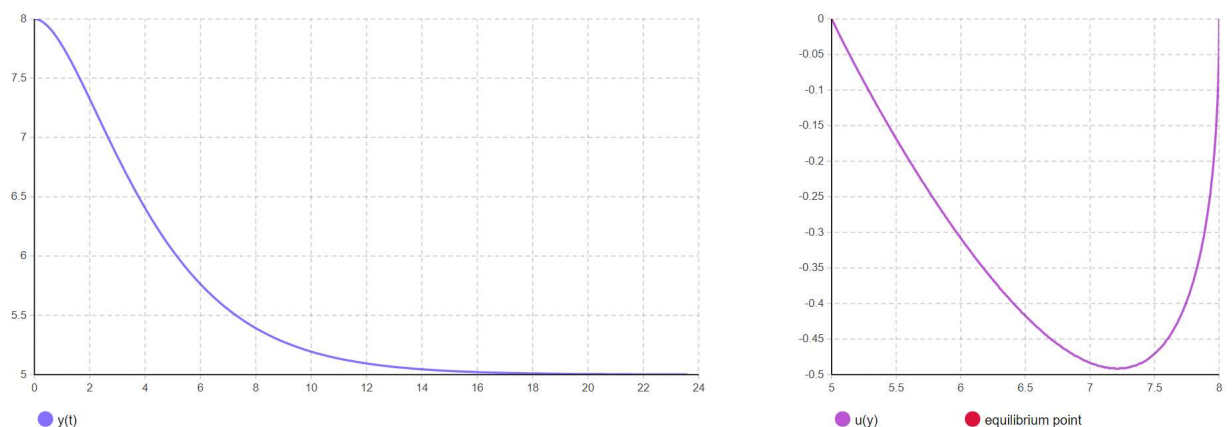


Рисунок 88 – Устойчивый узел при $r = 0.1$

Для построения графиков, приведённых на рисунке 89, начальная точка была выбрана вблизи стационарной, чтобы скорость движения от неустойчивого узла была не слишком велика.

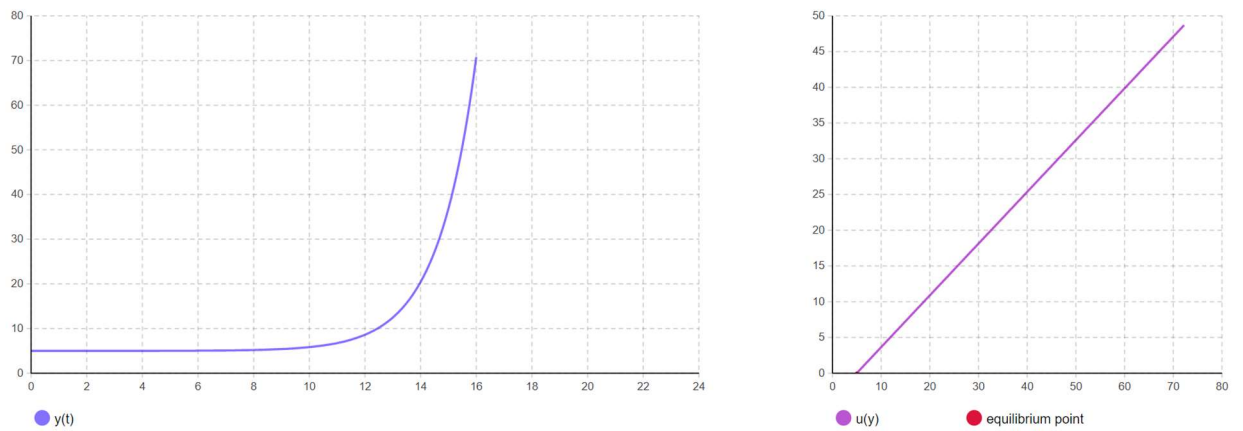


Рисунок 89 – Неустойчивый узел при $r = 2$

Из рисунка 90 видны затухающие гармонические колебания ВВП.

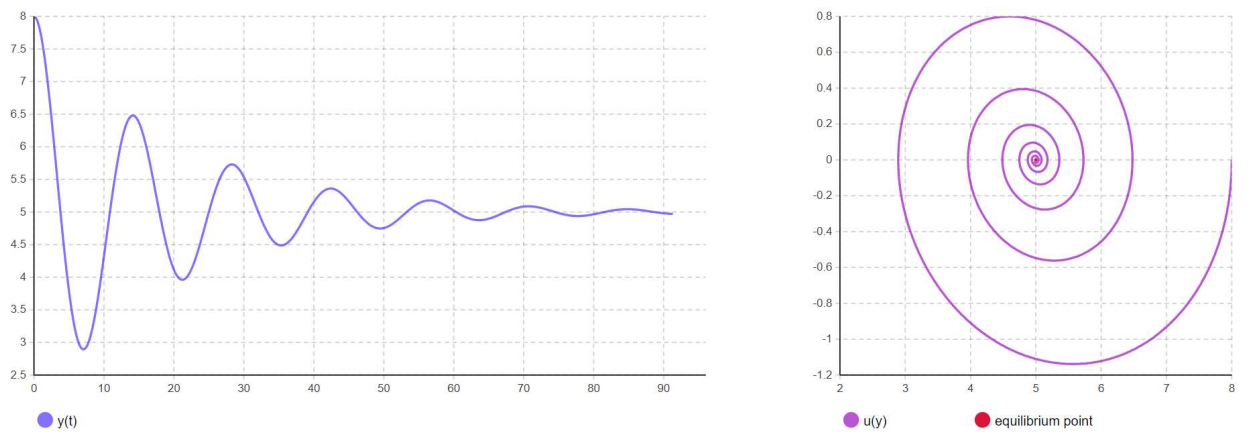


Рисунок 90 – Устойчивый фокус при $r = 0.9$

Движение от неустойчивого фокуса показано на рисунке 91.

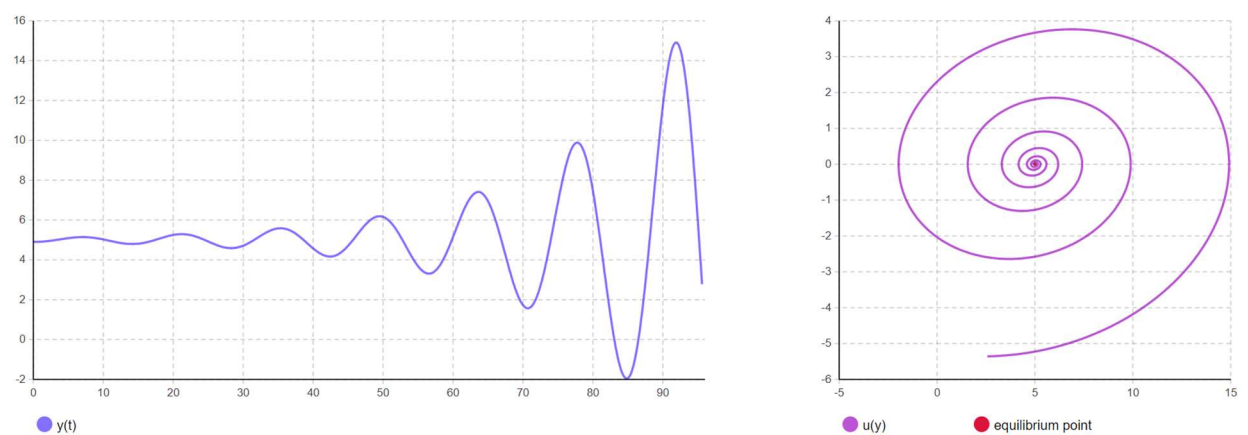


Рисунок 91 – Неустойчивый фокус при $r = 1.1$

4.2. Двухсекторная модель экономики

4.2.1. Модель двухсекторной закрытой экономики

Двухсекторная модель – модель кругооборота средств в закрытой экономике, состоящей из двух макроэкономических агентов: фирм (производственный сектор) и домашних хозяйств.

Схематичное изображение модели двухсекторной экономики представлено на рисунке 92.

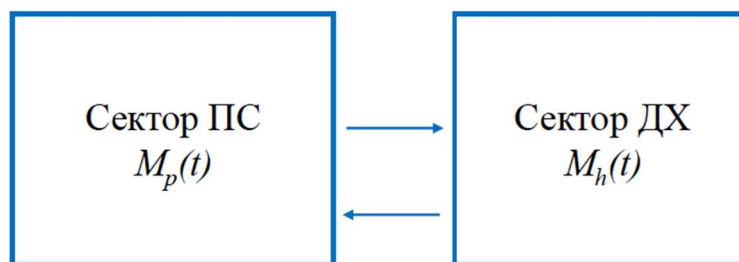


Рисунок 92 – Схема двухсекторной модели экономики

Обозначения:

- M_p – величина динамики средств в производственном секторе;
- M_h – величина динамики средств в домашних хозяйствах;
- p – уровень цен;
- k_h – доля динамики средств, которую население тратит на потребление в единицу времени;
- h – доля дохода домашних хозяйств от стоимости производимой продукции;
- F – производство продукции в единицу времени;
- k_p – доля динамики средств, которую производственный сектор тратит на воспроизводство основных фондов;
- a – скорость установления цен;
- f – эффективность отдачи от вложений;
- c – отдача от вложений;
- ΔM_p – эмиссия производственного сектора;
- ΔM_h – эмиссия домохозяйств.

Динамика модели описывается следующей системой ОДУ:

$$\begin{cases} \frac{dM_p}{dt} = k_h \cdot M_h - h \cdot F \cdot p, \\ \frac{dM_h}{dt} = h \cdot F \cdot p - k_h \cdot M_h, \\ \frac{dp}{dt} = a \cdot (k_h \cdot M_h + k_p \cdot M_p - F \cdot p), \end{cases} \quad (64)$$

где $F = f \cdot \left(\frac{k_p \cdot M_p}{p}\right)^c$.

Можно заметить, что вне зависимости от значений параметров модели сумма переменных $M_p + M_h$ в любой момент времени постоянна.

Приравнивая правые части уравнений системы (64) к нулю, получаем условия *простого воспроизводства* (сохранение постоянства значений переменных модели с течением времени):

$$\begin{cases} k_h \cdot M_h - h \cdot F \cdot p = 0, \\ M_p + M_h = M, \\ k_h \cdot M_h + k_p \cdot M_p - F \cdot p = 0, \end{cases} \quad (65)$$

где $F = f \cdot \left(\frac{k_p \cdot M_p}{p}\right)^c$.

При $c = 1$ в условиях (65) значения переменных M_p и M_h в любой момент времени выражаются формулами:

$$\begin{cases} M_p = M \cdot \frac{k_h}{k_h + k_p \cdot (f - 1)}, \\ M_h = M \cdot \frac{k_p \cdot (f - 1)}{k_h + k_p \cdot (f - 1)}. \end{cases} \quad (66)$$

где $M = M_p + M_h$.

Также из условий простого воспроизводства (65) и предположения $c = 1$ следует ограничение на значение h , которое определяется только параметром f :

$$h^* = \frac{f - 1}{f} \quad (67)$$

Задание

Построить имитационную модель двухсекторной модели экономики в общем виде. Выполнить имитационный анализ модели с параметрами $f = 4$, $h = 0.75$, $k_p = k_h = 1$, $\Delta M_p = \Delta M_h = 0$ и начальными данными $M_p(0) = 1$, $M_h(0) = 3$, $p(0) = 1$. Рассмотреть случаи $h = h^*$, $h < h^*$, $h > h^*$. Ввести эмиссию по секторам.

Реализация

С помощью инструментов AnyLogic переменные модели M_p , M_h , p были реализованы как объекты системной динамики типа накопитель с соответствующими ДУ согласно системе (64), F была объявлена динамической переменной. Параметры h , ΔM_p , ΔM_h были снабжены бегунками для возможности изменения значений в режиме реального времени. На рисунке 93 демонстрируется влияние параметра h на переменные модели.

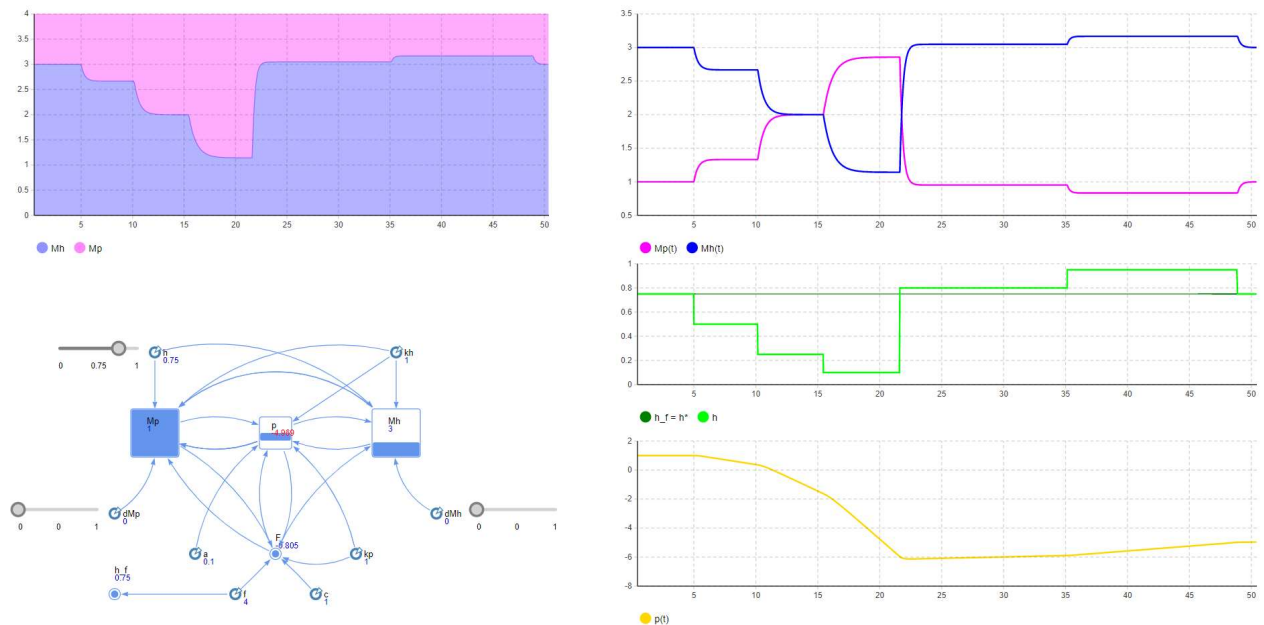


Рисунок 93 – Имитационный анализ модели двухсекторной экономики

Из графиков на приведённом рисунке можно заметить, что при заданных параметрах значение h определяет угол наклона прямой $p(t)$, а также соотношение между M_p и M_h в долгосрочной перспективе. При изменении h непродолжительное время наблюдается нелинейное изменение M_p и M_h , после чего их значения стабилизируются в новом соотношении и остаются постоянными.

4.2.2. Взаимодействие двух экономик

Рассмотрим модель кругооборота средств в паре открытых взаимодействующих между собой двухсекторных экономик. Схематичное отображение структуры модели приведено на рисунке 94.

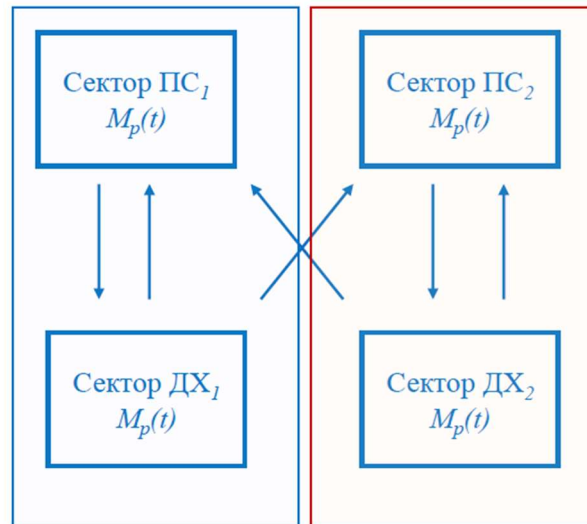


Рисунок 94 – Схема модели взаимодействия двух экономик

Обозначения:

- M_{pi} – величина динамики средств в ПС i -й экономики;
- M_{hi} – величина динамики средств в ДХ i -й экономики;
- p_i – уровень цен в i -й экономике;
- k_{hi} – доля динамики средств, которую ДХ i -й экономики тратят на потребление продукции ПС j -й экономики в единицу времени;
- h_i – доля дохода ДХ i -й экономики от стоимости продукции, производимой ПС i -й экономики;
- F_i – производство продукции ПС i -й экономики в единицу времени;
- k_{pi} – доля динамики средств, которую ПС i -й экономики тратит на воспроизводство основных фондов;
- a_i – скорость установления цен в i -й экономике ($i = 1, 2$);
- a_3 – скорость установления курса валют;
- b_{ij} – курс обмена валюты i -й экономики на валюту j -й экономики;
- f_i – эффективность отдачи от вложений в i -й экономике;
- c_i – отдача от вложений в i -й экономике.

Динамика модели описывается следующими уравнениями:

$$\begin{cases} \frac{dM_{pi}}{dt} = k_{hii} \cdot M_{hi} + k_{hji} \cdot M_{hj} \cdot b_{ji} - h_i \cdot F_i \cdot p_i + \Delta M_{pi}, \\ \frac{dM_{hi}}{dt} = h_i \cdot F_i \cdot p_i - (k_{hii} + k_{hji}) \cdot M_{hi} + \Delta M_{hi}, \\ \frac{dp_i}{dt} = a_i \cdot (k_{hii} \cdot M_{hi} + k_{hji} \cdot M_{hj} \cdot b_{ji} + k_{pi} \cdot M_{pi} - F_i \cdot p_i), \\ \frac{db_{21}}{dt} = a_3 \cdot \frac{k_{h12} \cdot M_{h1}}{k_{h21} \cdot M_{h2} - b_{21}}. \end{cases} \quad (68)$$

где $F_i = f_i \cdot \left(\frac{k_{pi} \cdot M_{pi}}{p_i} \right)^{c_i}$,

$b_{21} \cdot b_{12} = 1$.

Задание

Создать имитационную модель взаимодействия двухсекторных экономик.

Реализация

Имитационная модель была реализована в AnyLogic согласно уравнениям динамики (68). Результаты эксперимента с двумя одинаковыми экономиками, в рамках которого оценивалось влияние параметров h_1, h_2 на ключевые переменные модели, отражены на рисунке 95.

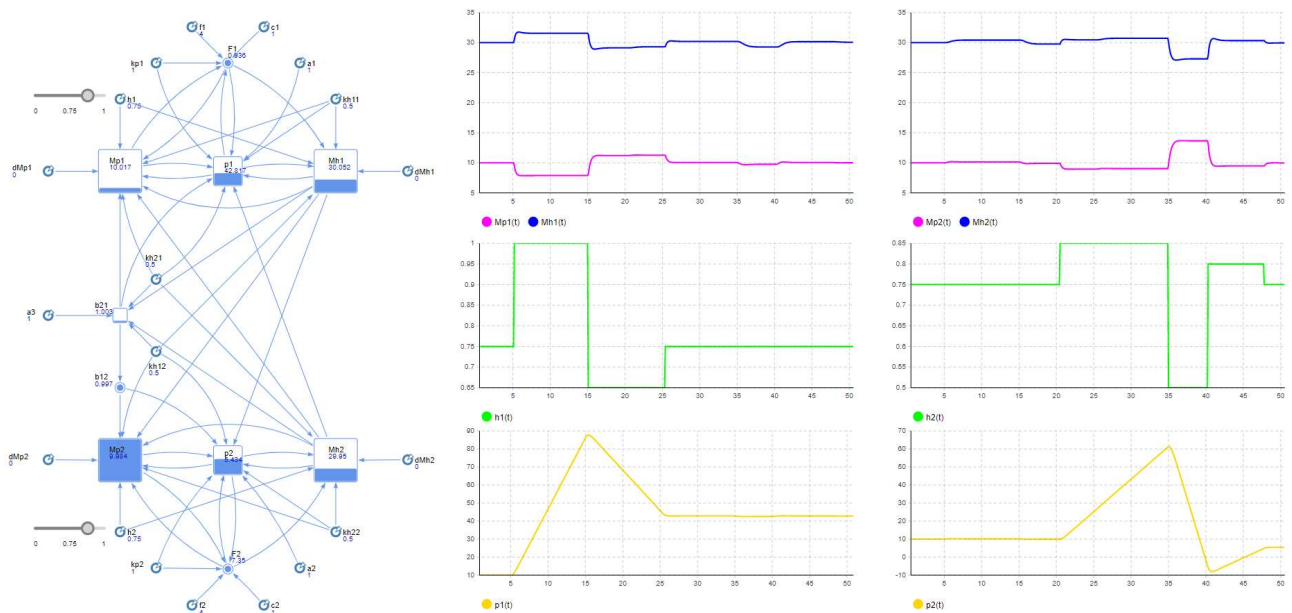


Рисунок 95 – Модель взаимодействия пары двухсекторных экономик

Итогом варьирования h_1, h_2 стало установление новых цен p_1, p_2 .

4.3. Трёхсекторная модель экономики

Трёхсекторная модель экономики – динамическая модель закрытой экономики, производственная система которой подразделяется на три сектора:

- материальный (0);
- фондосоздающий (1);
- потребительский (2).

Обозначения:

- X_i – объём продукции, произведённой в i -м секторе;
- K_i – основные производственные фонды i -го сектора;
- L_i – число занятых в i -м секторе;
- I_i – объём инвестиций в i -м секторе.

Предположения и уравнения модели:

- экономика замкнута;
- имеет место баланс трудовых ресурсов:

$$L = L_0 + L_1 + L_2, \quad (69)$$

где L – общее число занятых.

- технологический уклад описывается с помощью неоклассических производственных функций (например, с помощью функции Кобба-Дугласа (22)):

$$X_i(t) = F_i(K_i(t), L_i(t)) = A_i \cdot K_i^{\alpha_i}(t) \cdot L_i^{1-\alpha_i}(t), \quad (70)$$

где $A_i > 0$, $0 < \alpha_i < 1$, $i = 0, 1, 2$.

- общее число занятых в производственной сфере и число занятых в i -м секторе изменяется с постоянным темпом:

$$\begin{aligned} \frac{dL}{dt} &= \nu \cdot L, & L(0) &= L^0, \\ \frac{dL_i}{dt} &= \nu \cdot L_i, & L_i(0) &= L_i^0, & i &= 0, 1, 2, \end{aligned} \quad (71)$$

где ν – темп прироста трудовых ресурсов, $\nu = const$.

- лаг капиталовложений отсутствует, коэффициенты износа основных производственных фондов постоянны, из чего следует, что динамика основных производственных фондов K_i описывается дифференциальными уравнениями:

$$\frac{dK_i}{dt} = -\mu_i \cdot K_i + I_i, \quad i = 0, 1, 2, \quad (72)$$

где μ_i – доля выбывших за единицу времени основных производственных фондов i -го сектора, $\mu_i > 0$, $\mu_i = const$.

- коэффициенты прямых материальных затрат секторов a_i постоянны:

$$X_0 = a_0 \cdot X_0 + a_1 \cdot X_1 + a_2 \cdot X_2, \quad (73)$$

где a_i – коэффициент прямых материальных затрат i -го сектора, $a_i > 0$, $a_i = const$.

- имеет место баланс инвестиций:

$$X_1 = I_0 + I_1 + I_2 \quad (74)$$

Уравнения (69)-(74) составляют математическую модель трёхсекторной экономики, схематичное изображение которой представлено на рисунке 96.

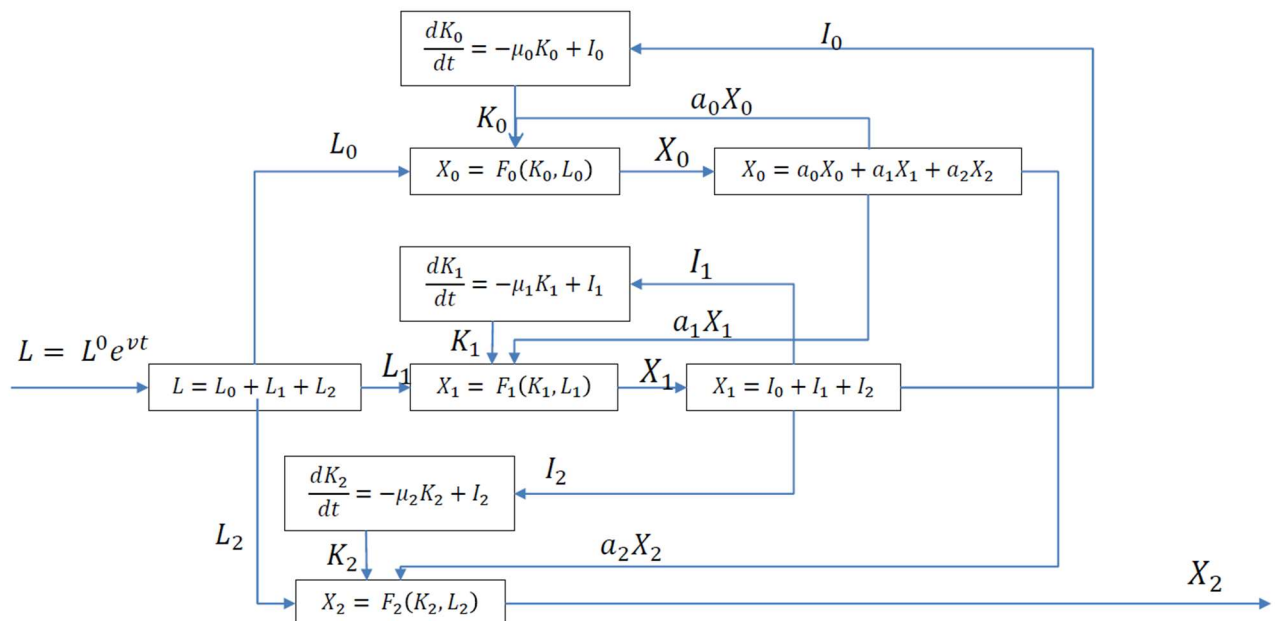


Рисунок 96 – Схема трёхсекторной модели экономики

Задание

Выполнить имитационный анализ трёхсекторной модели экономики. В качестве примера рассмотреть производственные функции с параметрами $A_0 = 6.19, \alpha_0 = 0.46, A_1 = 1.35, \alpha_1 = 0.68, A_2 = 2.71, \alpha_2 = 0.49$.

Реализация

Имитационная модель для анализа трёхсекторной экономики была построена с помощью средств системной динамики AnyLogic. Переменная L была реализована как накопитель с динамикой, задаваемой уравнением (71). Для распределения рабочей силы по секторам были добавлены неотрицательные параметры l_i , предназначенные для задания соотношения между занятыми в секторах. Таким образом, L_i были заданы как динамические переменные, вычисляемые по формуле:

$$L_i = L \cdot \frac{l_i}{l_0 + l_1 + l_2}, \quad i = 0, 1, 2 \quad (75)$$

Аналогичным образом с помощью неотрицательных параметров i_j было задано распределение инвестиций по секторам:

$$I_j = X_1 \cdot \frac{i_j}{i_0 + i_1 + i_2}, \quad j = 0, 1, 2 \quad (76)$$

Для подсчёта выпуска по секторам согласно формуле (70) были созданы динамические переменные X_i . Параметры производственных функций F_i были выбраны согласно условию задания, тогда как значения других параметров модели были выбраны по собственному усмотрению согласно их экономическому смыслу. В начальный момент времени $L(0) = K_i(0) = 10, \mu_i = 0.2, \nu = 0.01$, распределение рабочей силы и инвестиций по секторам производилось в равных пропорциях. Состояние созданной имитационной модели в момент запуска симуляции отражено на рисунке 97.

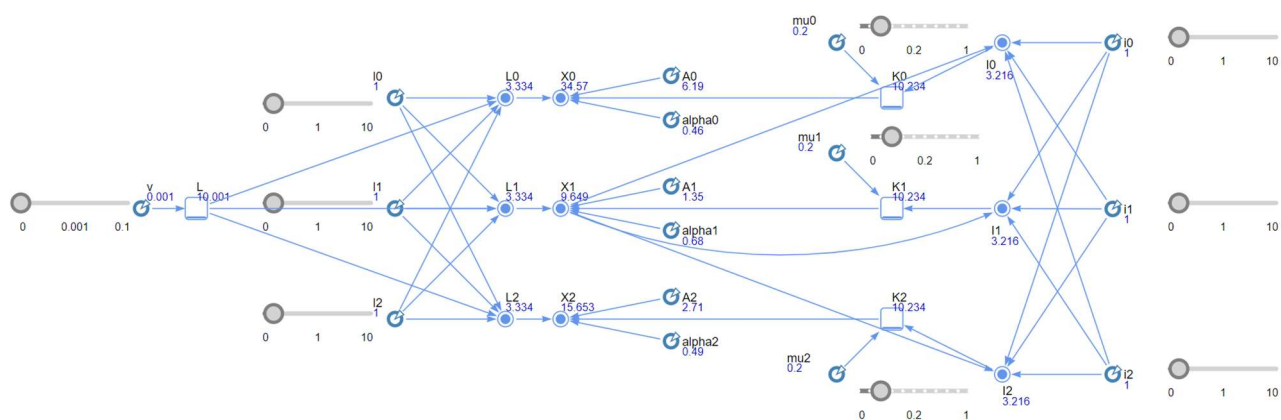


Рисунок 97 – Трёхсекторная модель экономики в AnyLogic

В рамках имитационного анализа модели путём варьирования отдельных параметров в режиме реального времени оценивалось их влияние на динамику развития экономики в целом и её секторов по отдельности. Результаты эксперимента демонстрируются на рисунке 98.

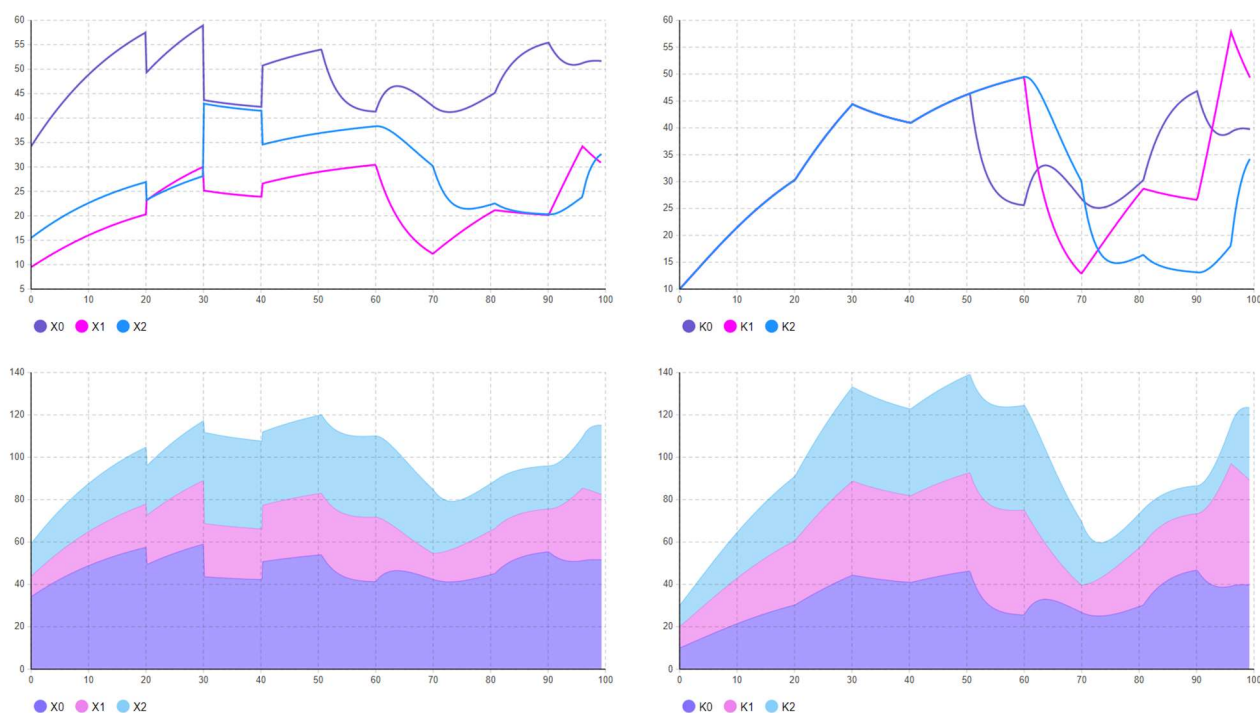


Рисунок 98 – Временная эволюция модели трёхсекторной экономики

Из приведённого рисунка видно, что первое время ($t \in [0, 20]$) во всех секторах наблюдается рост фондов и, как следствие, выпуска, обусловленный низкими нормами амортизации. В связи с одинаковым распределением труда и капитала по секторам отличие между X_i обусловлено исключительно параметрами производственных функций. На промежутке $t \in [20, 50]$ были рассмотре-

ны следующие варианты соотношений l_i : 1:1:2, 1:2:4, 1:2:2. Непропорциональное влияние изменения распределения занятых на выпуск отраслей при равных основных фондах объясняется степенной зависимостью выпуска от труда в функции Кобба-Дугласа и отличием α_i . Затем на промежутке $t \in [50, 80]$ проводилась оценка влияния нормы амортизации μ_i на сектора и динамику экономики в целом путём поочерёдного удвоения μ_i . Из графиков, приведённых на рисунке выше, можно заметить, что увеличение μ_0 в два раза повлияло (отрицательным образом) исключительно на материальный сектор. Аналогичная ситуация наблюдается в потребительском секторе. В то время как удвоение μ_1 закономерно отразилось на всех секторах и вызвало значительный экономический спад. На промежутках $[80, 90]$, $[90, 95]$, $[95, 100]$ временных графиков отражён экономический эффект от перераспределения инвестиций между секторами в соотношениях 2:1:1, 1:2:1, 1:1:2 соответственно. Для получения более явной картины был проведён дополнительный эксперимент с последовательностью пропорций $\{i_0:i_1:i_2\} = \{1:1:1, 2:1:1, 1:1:1, 1:2:1, 1:1:1, 1:1:2\}$, результаты которого отображены на рисунке 99.

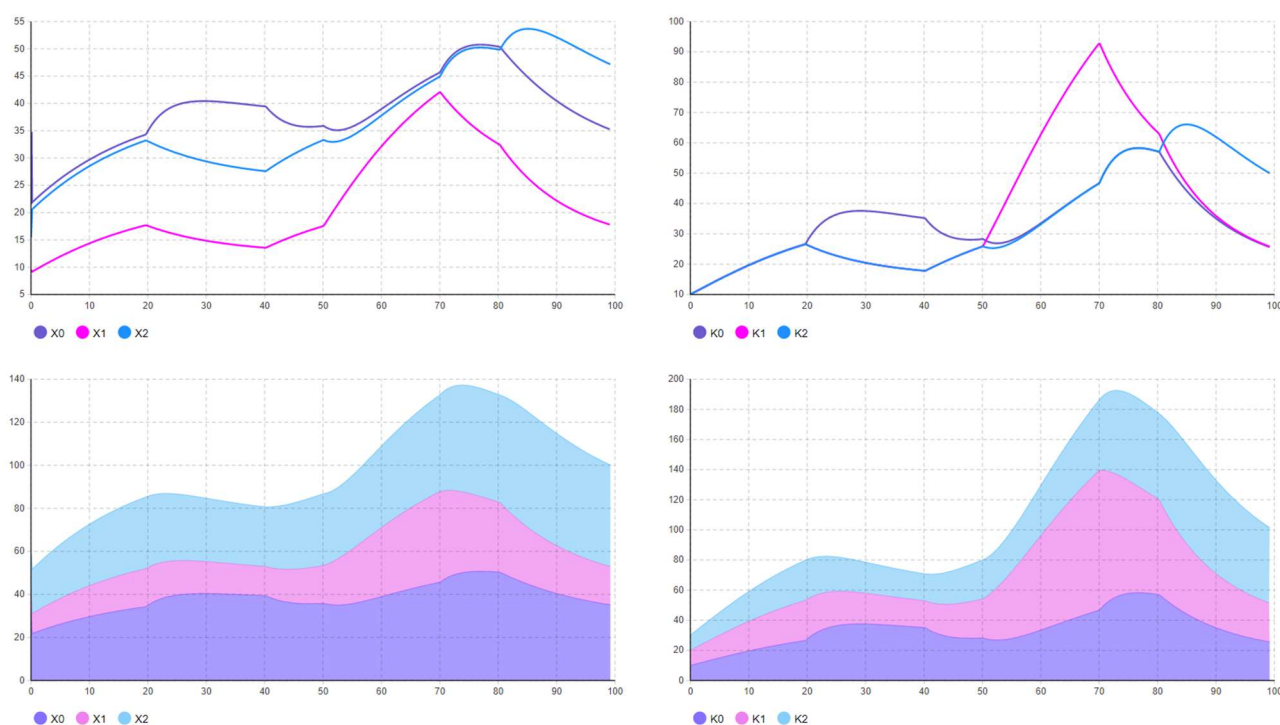


Рисунок 99 – Влияние распределения инвестиций в трёхсекторной модели

Наилучший эффект достигается при инвестировании во второй сектор.

5. МОДЕЛИ С РАСПРЕДЕЛЁННЫМИ ПАРАМЕТРАМИ

5.1. Модель распространения тепла в однородном тонком стержне

5.1.1. Явная разностная схема. Метод сеток

Рассматривается задача моделирования распространения тепла в однородном тонком стержне с теплоизолированной боковой поверхностью. Схема модели отражена на рисунке 100.

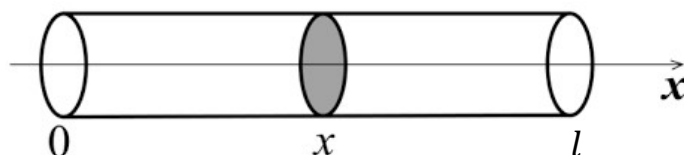


Рисунок 100 – Модель распространения тепла в тонком стержне

Обозначения и параметры модели:

- l – длина стержня;
- a – коэффициент температуропроводности стержня, $a > 0$;
- $\varphi(x)$ – функция, которая задаёт начальное распределение температуры;
- $u(x, t)$ – функция, которая задаёт температуру в поперечном сечении стержня с координатой x в момент времени t ;
- $g_0(t)$, $g_l(t)$ – функции, которые задают граничные условия для концов стержня.

Динамика распространения тепла в рассматриваемом одномерном случае задаётся однородным уравнением теплопроводности:

$$\frac{\partial u}{\partial t} = a^2 \cdot \frac{\partial^2 u}{\partial x^2}, \quad x \in (0, l), \quad t \in (0, T] \quad (77)$$

Начальное распределение температуры в стержне задано функцией φ :

$$u(x, 0) = \varphi(x), \quad x \in [0, l] \quad (78)$$

Краевые задачи для уравнения (77) позволяют моделировать различные температурные режимы на концах стержня. Граничные условия *первого рода* означают, на концах стержня задана температура, которая может меняться с течением времени или поддерживаться постоянной:

$$u(0, t) = g_0(t), \quad t \in (0, T] \quad (79)$$

$$u(l, t) = g_l(t), \quad t \in (0, T] \quad (80)$$

Граничные условия *второго рода* определяют тепловой поток на концах стержня:

$$\frac{\partial u}{\partial x}(0, t) = g_0(t), \quad t \in (0, T] \quad (81)$$

$$\frac{\partial u}{\partial x}(l, t) = g_l(t), \quad t \in (0, T] \quad (82)$$

В частности, теплоизоляция концов задаётся граничными условиями второго рода (81)-(82) при $g_0(t) = g_l(t) = 0$.

Для численного решения уравнения теплопроводности (77) воспользуемся методом сеток. Введём в рассматриваемой области $\{0 \leq x \leq l, 0 \leq t \leq T\}$ равномерную прямоугольную сетку с шагом $\Delta x = h$ по координате и шагом $\Delta t = \tau$ по времени.

Запишем уравнение теплопроводности (77) в дискретной форме:

$$u(x, t + \Delta t) = u(x, t) \cdot \left(1 - \frac{2a^2 \Delta t}{(\Delta x)^2}\right) + \frac{a^2 \Delta t}{(\Delta x)^2} \cdot [u(x + \Delta x, t) + u(x - \Delta x, t)] \quad (83)$$

Таким образом, переход от производных к конечным разностям позволил получить простейшую явную разностную схему с минимальным шаблоном из 4 точек, отражённых на рисунке 101.

	$u(x, t + \Delta t)$	
$u(x - \Delta x, t)$	$u(x, t)$	$u(x + \Delta x, t)$

Рисунок 101 – Шаблон явной схемы для уравнения теплопроводности

Порядок разностной аппроксимации равен $O(\tau + h^2)$. Стоит отметить, что шаг по временной оси должен удовлетворять следующему условию:

$$\frac{\Delta t}{(\Delta x)^2} \leq \frac{1}{2a^2} \quad (84)$$

Задание

Реализовать метод сеток для решения уравнения (77) в Excel и Python.

Начальное распределение температуры в стержне $u(x, 0)$:

- $\varphi(x) = 100x(1 - x)$;

- $\varphi(x) = \sin \frac{3\pi x}{2}$.

Варианты граничных условий:

- постоянная температура на концах стержня:

$$u(0, t) = g_0(t) \equiv \text{const}, u(l, t) = g_l(t) \equiv \text{const};$$

- оба конца теплоизолированы:

$$u_x(0, t) = g_0(t) = 0, u_x(l, t) = g_l(t) = 0;$$

- на одном конце постоянная температура, а другой теплоизолирован:

$$u(0, t) = g_0(t) \equiv \text{const}, u_x(l, t) = g_l(t) = 0.$$

1. Реализация в LibreOffice Calc

Для упрощения вычислений значение коэффициента теплопроводности a и длины стержня l были выбраны равными единице. Исходя из этого, а также с учётом условия (84) были выбраны шаги разбиения области по координате и по времени: $h = 0.1$, $\tau = 0.005$. При подстановке $\frac{a^2 \Delta t}{(\Delta x)^2} = \frac{1}{2}$ в формулу (83) имеем:

$$u(x, t + \Delta t) = \frac{u(x + \Delta x, t) + u(x - \Delta x, t)}{2} \quad (85)$$

Фрагмент полученного решения для случая с начальным распределением температуры $\varphi(x) = 100x(1 - x)$ и постоянной нулевой температурой на обоих концах стержня, приведён на рисунке 102.

	A	B	C	D	E	F	G	H	I	J	K	L
1	t \ x	0	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9	1
2	0,000	0	9,000	16,000	21,000	24,000	25,000	24,000	21,000	16,000	9,000	0
3	0,005	0	8,000	15,000	20,000	23,000	24,000	23,000	20,000	15,000	8,000	0
4	0,010	0	7,500	14,000	19,000	22,000	23,000	22,000	19,000	14,000	7,500	0
5	0,015	0	7,000	13,250	18,000	21,000	22,000	21,000	18,000	13,250	7,000	0
6	0,020	0	6,625	12,500	17,125	20,000	21,000	20,000	17,125	12,500	6,625	0

Рисунок 102 – Фрагмент таблицы для случая нулевой температуры на концах

График изменения температуры в стержне, построенный на выбранных строках таблицы, приведён на рисунке 103.

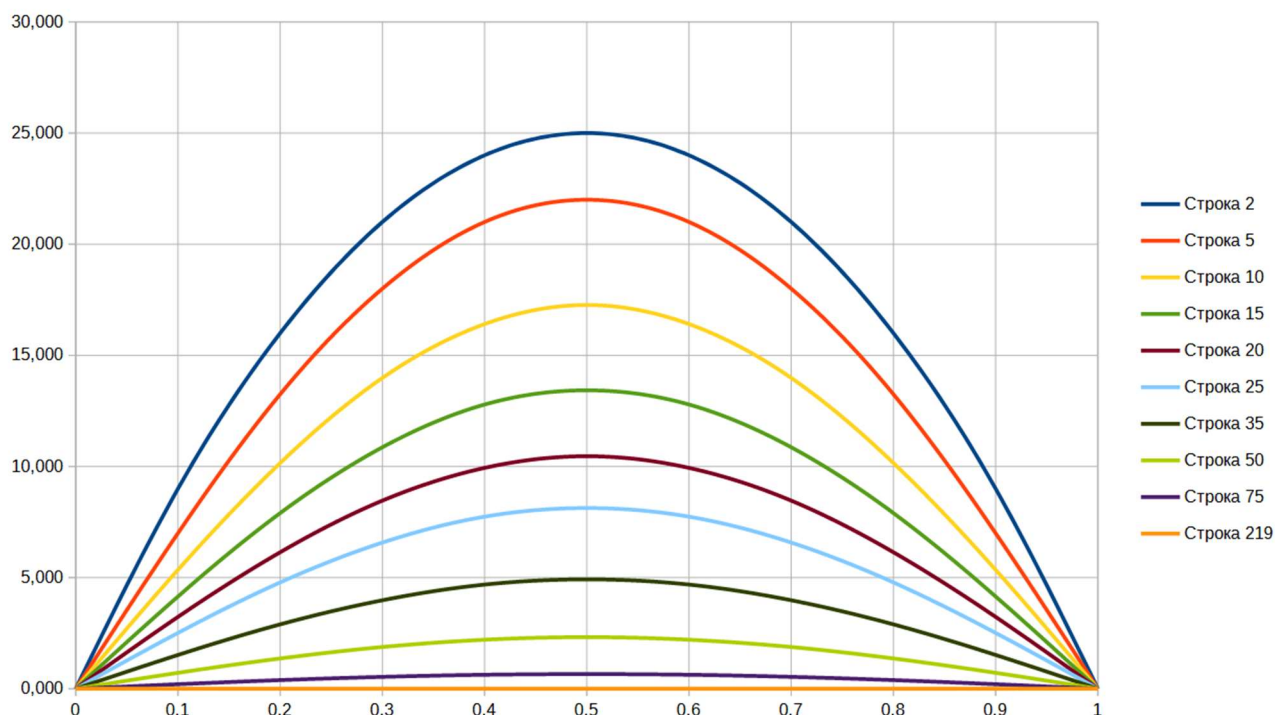


Рисунок 103 – График изменения температуры в стержне, на концах которого поддерживается постоянная нулевая температура

Аналогичным образом было получено решение для случая, когда концы стержня теплоизолированы. Из соответствующих граничных условий (81)-(82) получаем выражения для теплоизолированных концов:

$$u(0, t + \Delta t) = u(\Delta x, t + \Delta t) \quad (86)$$

$$u(l, t + \Delta t) = u(l - \Delta x, t + \Delta t) \quad (87)$$

Фрагмент полученного решения приведён на рисунке 104.

	A	B	C	D	E	F	G	H	I	J	K	L
1	t \ x	0	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9	1
2	0,000	0	9,000	16,000	21,000	24,000	25,000	24,000	21,000	16,000	9,000	0
3	0,005	8,000	8,000	15,000	20,000	23,000	24,000	23,000	20,000	15,000	8,000	8,000
4	0,010	11,500	11,500	14,000	19,000	22,000	23,000	22,000	19,000	14,000	11,500	11,500
5	0,015	12,750	12,750	15,250	18,000	21,000	22,000	21,000	18,000	15,250	12,750	12,750
6	0,020	14,000	14,000	15,375	18,125	20,000	21,000	20,000	18,125	15,375	14,000	14,000
7	0,025	14,688	14,688	16,063	17,688	19,563	20,000	19,563	17,688	16,063	14,688	14,688
8	0,030	15,375	15,375	16,188	17,813	18,844	19,563	18,844	17,813	16,188	15,375	15,375
9	0,035	15,781	15,781	16,594	17,516	18,688	18,844	18,688	17,516	16,594	15,781	15,781

Рисунок 104 – Фрагмент таблицы для случая теплоизолированных концов

Соответствующая диаграмма демонстрируется на рисунке 105.

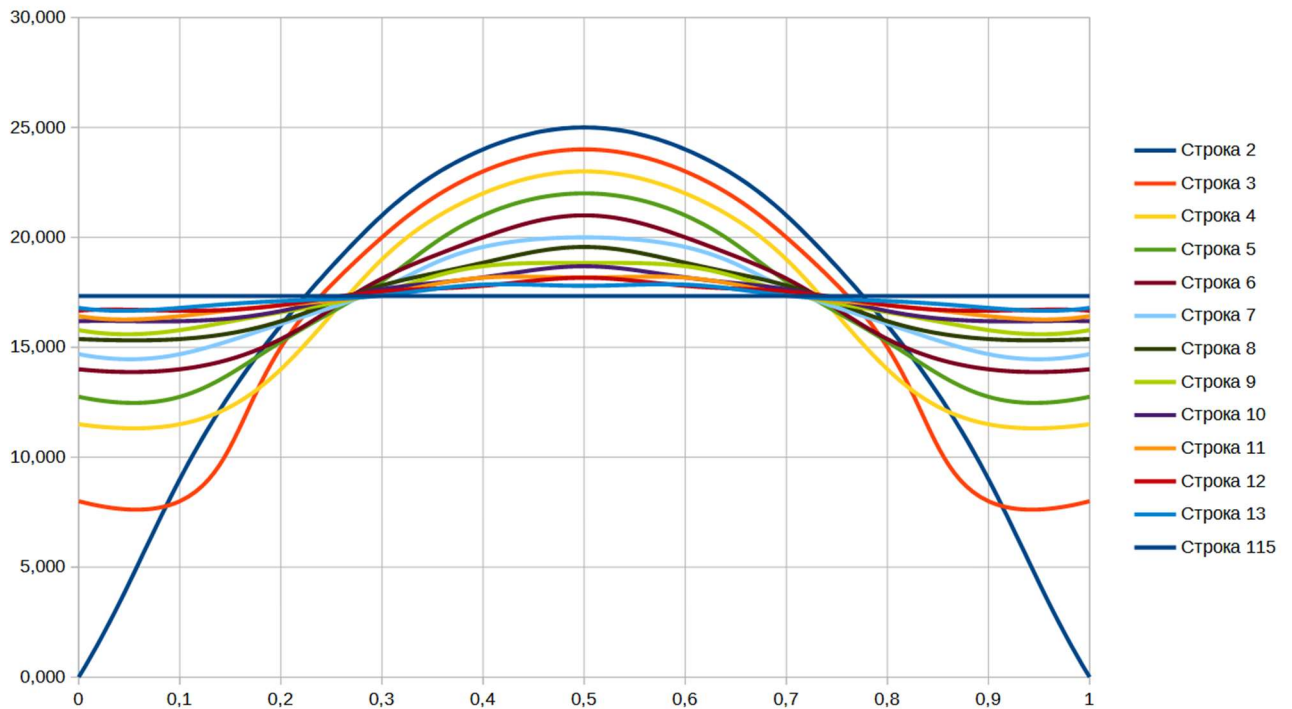


Рисунок 105 – График изменения температуры в стержне с теплоизолированными концами

Путём комбинирования граничных условий удалось смоделировать остывание стержня, на левом конце которого поддерживается постоянная нулевая температура, а правый теплоизолирован. График приведён на рисунке 106.

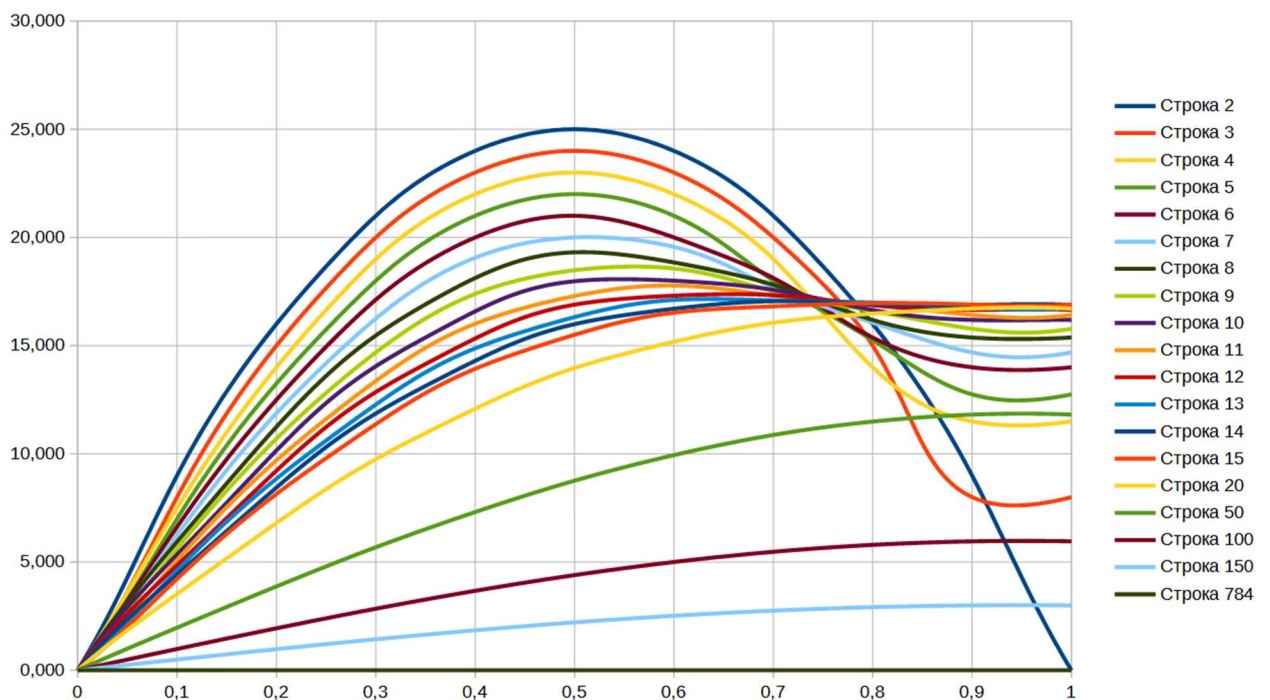


Рисунок 106 – График изменения температуры в стержне, на левом конце которого поддерживается нулевая температура, а правый теплоизолирован

2. Реализация в Python

Для численного решения краевых задач для уравнения теплопроводности при любых допустимых значениях параметров с помощью языка Python был реализован класс HeatEquation, описание и программный код ключевых методов которого приведены на рисунке 107.

```
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib import cm
from math import sin, pi
from decimal import Decimal

class HeatEquation(object):
    def __init__(self, a: float, L: float, dx: float):
        """
        Setting key params of heat equation  $u''t = a**2 * u''xx$ 
        :param a: thermal diffusivity coefficient ( $a > 0$ )
        :param L: rod length ( $x_{stop}$ )
        :param dx: x-axis step
        """
        self.a = a
        self.L = L
        self.dx = dx
        self.u = None

    def solve_explicit_scheme(self, T: float, dt: float, f, u0=None, ul=None, ux0=None, uxL=None):
        """
        Finds numerical solution of heat equation by using explicit finite difference scheme
        :param T: t_stop
        :param dt: t-axis step
        :param f: function name:  $f(x) = u(x, 0)$ 
        :param u0: function name:  $u0(t) = u(0, t)$ 
        :param ul: function name:  $ul(t) = u(L, t)$ 
        :param ux0: function name:  $ux0(t) = du/dx(0, t)$ 
        :param uxL: function name:  $uxL(t) = du/dx(L, t)$ 
        :return:  $u[x][t]$  as np.array.
        """
        k = self.a * self.a * dt / self.dx / self.dx
        if 2 * k > 1:
            raise ValueError()
        time = np.arange(0, T + dt, dt)
        coordinates = np.arange(0, self.L + self.dx, self.dx)
        m = time.shape[0]
        n = coordinates.shape[0]
        self.u = np.empty([n, m])

        for x in range(n):
            self.u[x][0] = f(coordinates[x])
        for t in range(m - 1):
            for x in range(1, n - 1):
                self.u[x][t + 1] = self.u[x][t] * (1 - 2 * k) + k * (self.u[x + 1][t] + self.u[x - 1][t])

            if u0 is not None:
                self.u[0][t + 1] = u0(time[t + 1])
            if ux0 is not None:
                self.u[0][t + 1] = self.u[1][t + 1] - self.dx * ux0(time[t + 1])
            if ul is not None:
                self.u[n - 1][t + 1] = ul(time[t + 1])
            if uxL is not None:
                self.u[n - 1][t + 1] = self.u[n - 2][t + 1] + self.dx * uxL(time[t + 1])

        return self.u
```

Рисунок 107 – Программный код инициализации объекта класса HeatEquation и метода нахождения численного решения уравнения

Анимированная визуализация найденного решения осуществлялась с помощью соответствующего метода, использующего инструменты библиотеки matplotlib, как показано на рисунке 108.

```
def plot2DAnimation(self, u, dt: float, speed=1.):
    """
    Visualization of temperature distribution changes over time
    :param u: u(x, t) as np.array.
    :param dt: t-axis step
    :param speed: animation speed coefficient
    """
    x_coordinates = np.arange(0, self.L + self.dx, self.dx)
    y_data = u.transpose()
    fig, ax = plt.subplots()
    fig.suptitle(f'Solution of heat equation: temperature = u(x, t)')
    fig.supxlabel(f'a = {self.a}, L = {self.L}, dx = {self.dx}, dt = {dt}, '
                  f'λ = {round(self.a * self.a * dt / self.dx / self.dx, 3)}')
    ax.set_xlabel('x')
    ax.set_ylabel('temperature')
    ax.set_xlim([0, self.L])
    ax.set_ylim([np.min(u), np.max(u)])

    line, = ax.plot(x_coordinates, y_data[0], label='time = 0', linewidth=2)
    plot_legend = ax.legend()

    dp = Decimal(str(dt)).as_tuple().exponent * (-1)

    def animate(i):
        line.set_ydata(y_data[i])
        plot_legend.get_texts()[0].set_text(f'time = {dt * i:.{dp}f}')
        return line,

    ani = animation.FuncAnimation(fig, animate, repeat=True, frames=y_data.shape[0],
                                  interval=round(100 / speed))
    plt.show()
```

Рисунок 108 – Программный код анимации изменения температуры в стержне

На рисунке 109 демонстрируется пример работы метода.

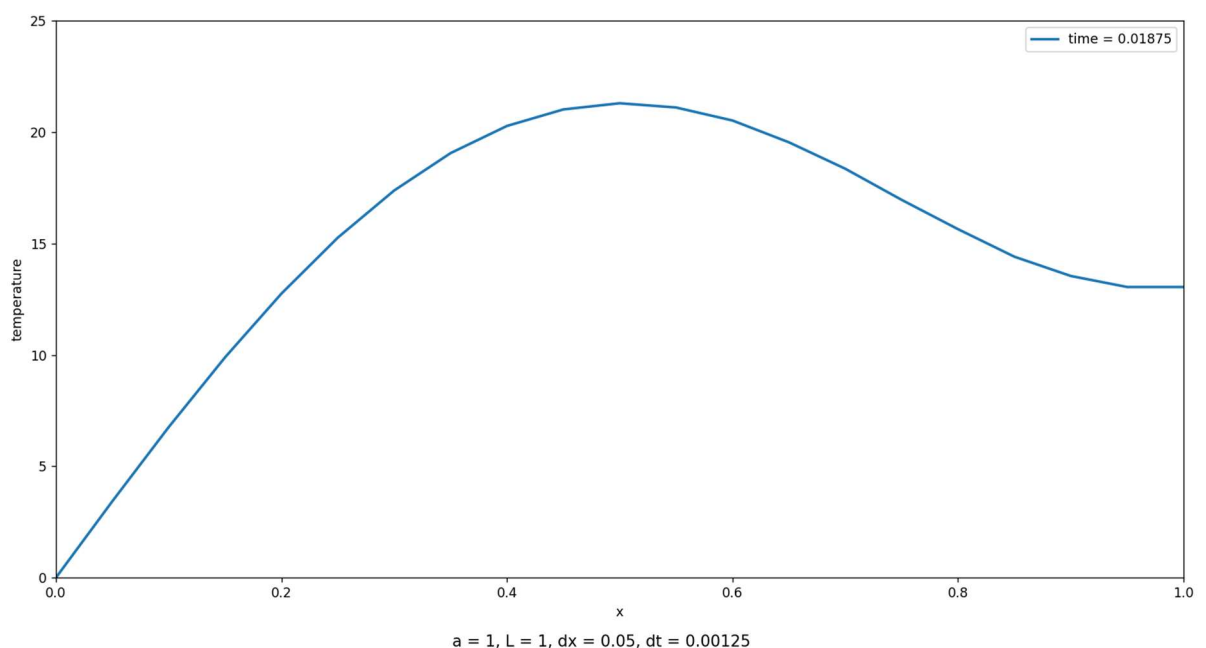


Рисунок 109 – Пример запуска программы

5.1.2. Неявная разностная схема. Метод прогонки

Существует и другой способ нахождения численного решения уравнения теплопроводности (77), который обладает большей устойчивостью по сравнению с явной схемой (83). После введения аналогичным образом в рассматриваемой области $\{0 \leq x \leq l, 0 \leq t \leq T\}$ равномерной прямоугольной сетки с шагом $\Delta x = h$ по координате и шагом $\Delta t = \tau$ по времени уравнение теплопроводности (77) записывается в дискретной форме, после чего значения на следующем временном слое $t + \Delta t$ связываются между собой в предположении о завершенности предыдущего слоя t в виде неявной формулы:

$$-u(x, t) = -u(x, t + \Delta t) \cdot \left(1 + \frac{2a^2 \Delta t}{(\Delta x)^2}\right) + \frac{a^2 \Delta t}{(\Delta x)^2} \cdot [u(x + \Delta x, t + \Delta t) + u(x - \Delta x, t + \Delta t)] \quad (88)$$

Таким образом, получаем простейшую неявную разностную схему с минимальным шаблоном из 4 точек, отражённых на рисунке 101.

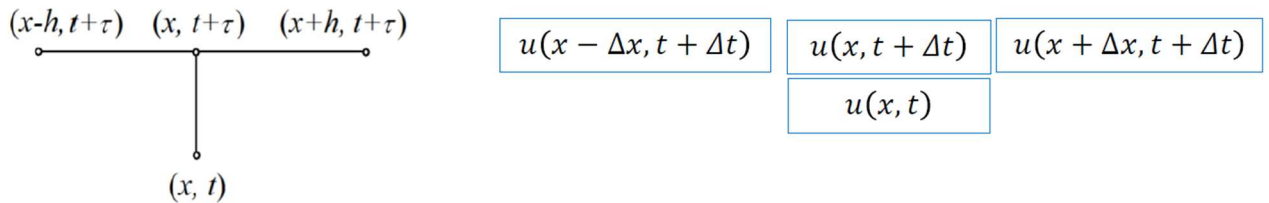


Рисунок 110 – Шаблон неявной схемы для уравнения теплопроводности

Для удобства дальнейших преобразований введём обозначение:

$$\lambda = \frac{a^2 \Delta t}{(\Delta x)^2} \quad (89)$$

Тогда соотношения (88) для каждой точки $u(x, t + \Delta t)$ следующего временного слоя можно переписать в более лаконичном виде:

$$-u(x, t) = -u(x, t + \Delta t) \cdot (1 + 2\lambda) + \lambda \cdot [u(x + \Delta x, t + \Delta t) + u(x - \Delta x, t + \Delta t)] \quad (90)$$

В результате получаем СЛАУ относительно значений температуры на следующем временном слое, решение которой является абсолютно устойчивым и не накладывает ограничений на шаг по оси времени. Перейдём от $u(x, t)$ к u_i^j , как показано на рисунке 111.

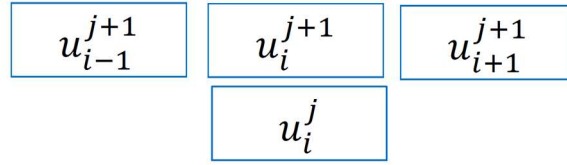


Рисунок 111 – Шаблон неявной схемы с двусторонней нумерацией

Выражение (90) в новых обозначениях примет следующий вид:

$$-u_i^j = -u_i^{j+1} \cdot (1 + 2\lambda) + \lambda \cdot (u_{i+1}^{j+1} + u_{i-1}^{j+1}) \quad (91)$$

Граничные условия первого и второго рода могут быть записаны в общем виде через соответствующую линейную комбинацию.

$$g_0(j+1) = k_1 \cdot u_1^{j+1} - u_0^{j+1}, \quad (92)$$

$$g_n(j+1) = u_n^{j+1} - k_{n-1} \cdot u_{n-1}^{j+1}, \quad (93)$$

где $k_i = \begin{cases} 0, & \text{если условие первого рода,} \\ 1, & \text{если условие второго рода.} \end{cases}$

Также выполним замену $y_i = u_i^{j+1}$ и введём единообразные обозначения для коэффициентов линейных уравнений (91)-(93). Тогда СЛАУ примет вид:

$$\begin{cases} -b_0 \cdot y_0 + c_0 \cdot y_1 = -f_0, & i = 0, \\ a_i \cdot y_{i-1} - b_i \cdot y_i + c_i \cdot y_{i+1} = -f_i, & i = 1, \dots, n-1 \\ a_n \cdot y_{n-1} - b_n \cdot y_n = -f_n, & i = n. \end{cases} \quad (94)$$

где $a_i = c_i = \lambda$;

$b_i = 1 + 2\lambda$;

$f_i = u_i^j$.

Таким образом, для того чтобы получить значения на очередном временном слое, требуется решить СЛАУ (94) с трёхдиагональной матрицей.

Метод прогонки (алгоритм Томаса) – метод для решения систем линейных алгебраических уравнений вида

$$A \cdot x = F, \quad (95)$$

где A – трёхдиагональная матрица коэффициентов;

x – вектор-столбец неизвестных;

F – вектор-столбец правых частей.

Поскольку матрица A является трёхдиагональной, каждое уравнение системы (95) можно представить в виде:

$$a_i \cdot x_{i-1} + b_i \cdot x_i + c_i \cdot x_{i+1} = f_i \quad (96)$$

Метод прогонки основывается на предположении, что искомые неизвестные x_i связаны рекуррентным соотношением:

$$x_i = \alpha_i \cdot x_{i+1} + \beta_i \quad (97)$$

Первый этап метода заключается в нахождении прогоночных коэффициентов α , β по формулам:

$$\begin{aligned} \alpha_1 &= -\frac{c_1}{b_1}, \\ \beta_1 &= \frac{f_1}{b_1}, \\ \alpha_i &= -\frac{c_i}{a_i \cdot \alpha_{i-1} + b_i}, \quad i = 2, \dots, n \\ \beta_i &= \frac{f_i - a_i \cdot \beta_{i-1}}{a_i \cdot \alpha_{i-1} + b_i}, \quad i = 2, \dots, n \end{aligned} \quad (98)$$

После чего искомые неизвестные вычисляются рекуррентно согласно формуле (97) в обратном порядке:

$$\begin{aligned} x_n &= \beta_n, \\ x_i &= \alpha_i \cdot x_{i+1} + \beta_i, \quad i = n-1, \dots, 1 \end{aligned} \quad (99)$$

Для нахождения неизвестных y_i системы (94) методом прогонки требуется вычислить прогоночные коэффициенты, формулы для которых с учётом введённых обозначений примут вид:

$$\begin{aligned}\alpha_0 &= \frac{c_0}{b_0}, & \beta_0 &= \frac{f_0}{b_0}, \\ \alpha_i &= \frac{c_i}{b_i - a_i \cdot \alpha_{i-1}}, & i &= 1, \dots, n-1 \\ \beta_i &= \frac{f_i + a_i \cdot \beta_{i-1}}{b_i - a_i \cdot \alpha_{i-1}}, & i &= 1, \dots, n-1\end{aligned}\quad (100)$$

Вычисления по методу прогонки в случае уравнения (77) можно упростить подстановкой значений некоторых коэффициентов согласно замене (94):

$$\begin{aligned}\alpha_i &= \frac{\lambda}{1 + 2\lambda - \lambda \cdot \alpha_{i-1}}, & i &= 1, \dots, n-1 \\ \beta_i &= \frac{f_i + \lambda \cdot \beta_{i-1}}{1 + 2\lambda - \lambda \cdot \alpha_{i-1}}, & i &= 1, \dots, n-1\end{aligned}\quad (101)$$

После этого можно последовательно найти искомые значения $u_i^{j+1} = y_i$:

$$\begin{cases} y_n = \frac{f_n + a_n \cdot \beta_{n-1}}{b_n - a_n \cdot \alpha_{n-1}}, \\ y_i = \alpha_i \cdot y_{i+1} + \beta_i, \end{cases} \quad i = n-1, \dots, 0. \quad (102)$$

Задание

Реализовать метод прогонки для решения уравнения (77) в Excel и Python.

Начальное распределение температуры в стержне: $u(x, 0) = 100x(1 - x)$.

Варианты граничных условий:

- постоянная температура на концах стержня:

$$b_0 = 1, c_0 = 0, f_0 \equiv \text{const}, a_n = 0, b_n = 1, f_n \equiv \text{const};$$

- оба конца теплоизолированы:

$$b_0 = c_0 = a_n = b_n = 1, f_0 = f_n = 0;$$

- на одном конце постоянная температура, а другой теплоизолирован:

$$b_0 = 1, c_0 = 0, f_0 \equiv \text{const}, a_n = b_n = 1, f_n = 0.$$

1. Реализация в LibreOffice Calc

Для упрощения вычислений были выбраны следующие значения параметров: $l = 1$, $a = 0.1$, $h = 0.1$, $\tau = 1$, в результате чего получено $\lambda = 1$.

На рисунке 112 приведён фрагмент расчётов 5 временных слоёв для третьего варианта граничных условий.

t \ x	0	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9	1
t = 0	0	9,000	16,000	21,000	24,000	25,000	24,000	21,000	16,000	9,000	0
t = 1	0	7,766	14,298	19,129	22,088	23,135	22,316	19,814	16,126	12,563	12,563
t = 2	0	6,881	12,876	17,448	20,340	21,485	20,979	19,137	16,618	14,590	14,590
t = 3	0	6,186	11,678	15,973	18,792	20,064	19,914	18,698	17,044	15,817	15,817
t = 4	0	5,616	10,663	14,694	17,445	18,849	19,039	18,353	17,322	16,570	16,570
t = 5	0	5,138	9,797	13,591	16,281	17,808	18,294	18,035	17,457	17,013	17,013

a	0	1	1	1	1	1	1	1	1	1	1
b	1	3	3	3	3	3	3	3	3	3	1
c	0	1	1	1	1	1	1	1	1	1	0
α	0	0,33333	0,375	0,38095	0,38182	0,38194	0,38196	0,38197	0,38197	0,38197	0

β	0	3	7,125	10,7143	13,2545	14,6111	14,748	13,6545	11,327	7,76423	12,5628
β	0	2,58869	6,33261	9,69954	12,137	13,4718	13,6697	12,7896	11,0446	9,01722	14,5902
β	0	2,29353	5,68845	8,81396	11,1316	12,4576	12,7717	12,188	11,0027	9,77562	15,8173
β	0	2,0621	5,15265	8,04783	10,248	11,5773	12,0284	11,7366	10,9933	10,2407	16,5699
β	0	1,87211	4,70056	7,38822	9,48175	10,8208	11,4052	11,3667	10,9583	10,5148	17,0133

Рисунок 112 – Расчёт временных слоёв методом прогонки

График полученного решения приведён на рисунке 113.

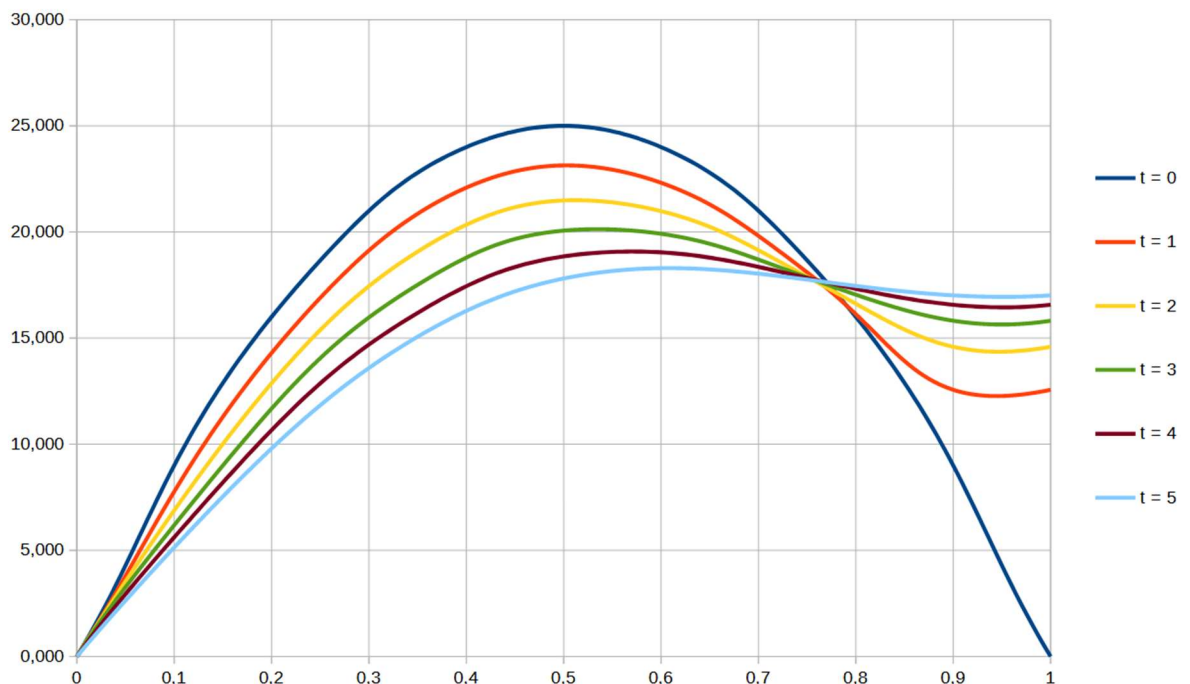


Рисунок 113 – График решения для третьего варианта граничных условий

Аналогичная диаграмма для случая, когда оба конца стержня теплоизолированы, демонстрируется на рисунке 114.

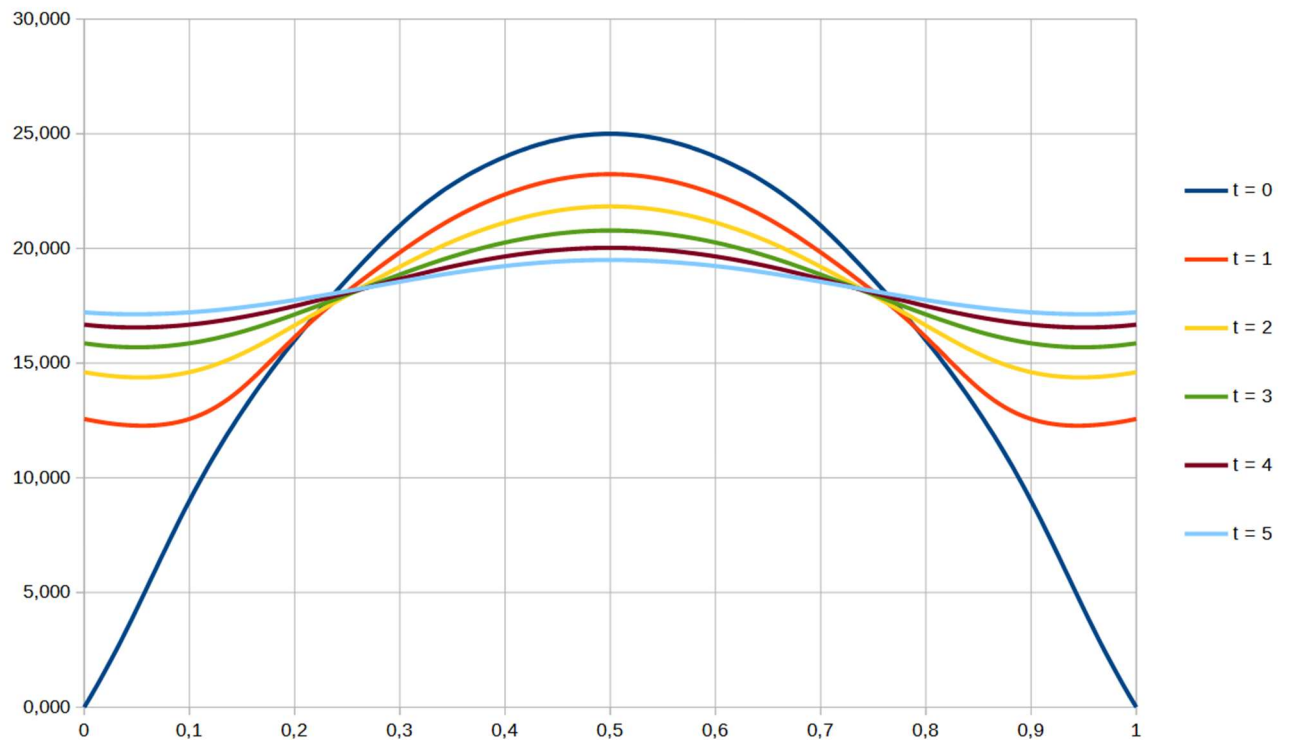


Рисунок 114 – График решения для второго варианта граничных условий

Динамика остывания стержня, на обоих концах которого поддерживается нулевая температура, отражена на рисунке 115.

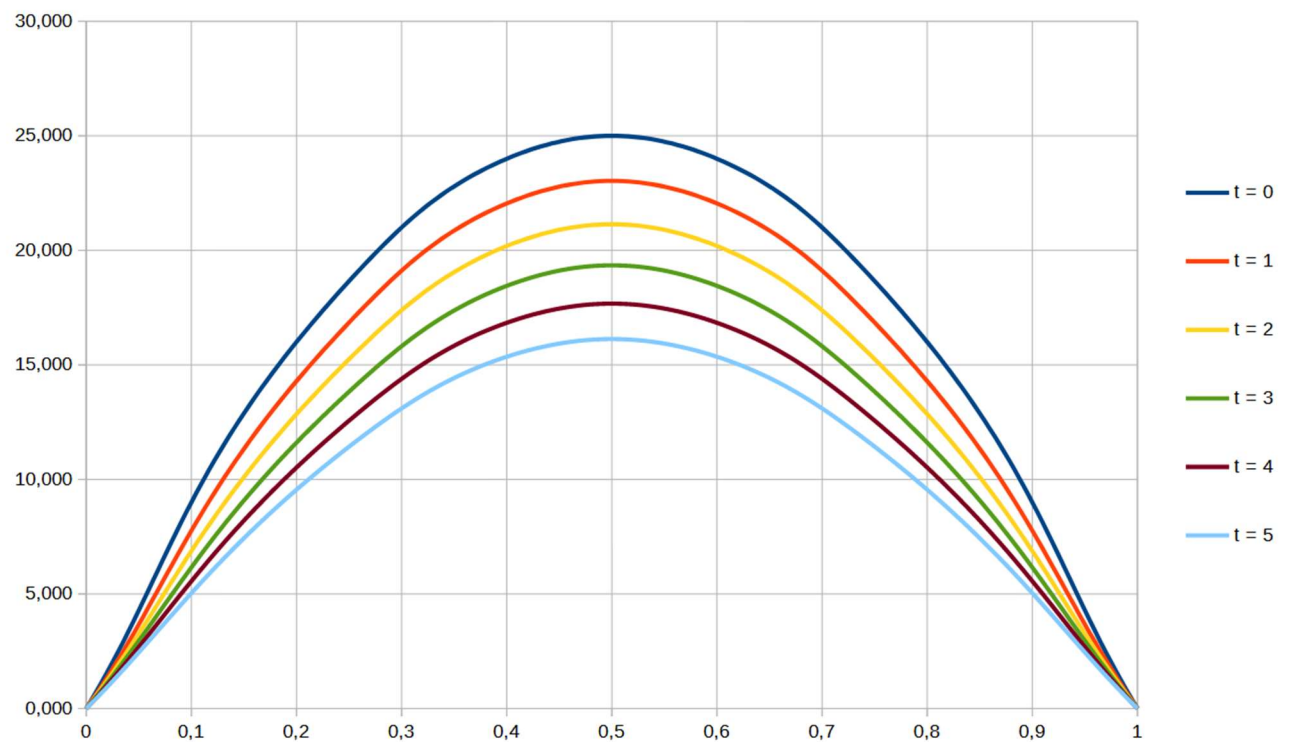


Рисунок 115 – График решения для первого варианта граничных условий

2. Реализация в Python

Численное решение уравнения теплопроводности (77) с использованием неявной схемы (91) и метода прогонки (100)-(102) было реализовано в Python как метод класса HeatEquation (см. рисунок 116). Исходный код метода приведён на рисунке 116.

```
def solve_implicit_scheme(self, T: float, dt: float, f, u0=None, ul=None, ux0=None, uxl=None):
    """
    Finds numerical solution of heat equation by using implicit finite difference scheme
    :param T: t_stop
    :param dt: t-axis step
    :param f: function name: f(x) = u(x, 0)
    :param u0: function name: u0(t) = u(0, t)
    :param ul: function name: ul(t) = u(L, t)
    :param ux0: function name: ux0(t) = du/dx(0, t)
    :param uxl: function name: uxl(t) = du/dx(L, t)
    :return: u[x][t] as np.array.
    """
    b0, c0, f0, d0 = 1, 0, zero, 1
    an, bn, fn, dn = 1, 0, zero, 1
    time = np.arange(0, T + dt, dt)
    coordinates = np.arange(0, self.L + self.dx, self.dx)
    m = time.shape[0]
    n = coordinates.shape[0]
    self.u = np.empty([n, m])
    k = self.a * self.a * dt / self.dx / self.dx
    for x in range(n):
        self.u[x][0] = f(coordinates[x])
    if u0 is not None:
        b0, c0, f0, d0 = 1, 0, u0, 1
    if ux0 is not None:
        b0, c0, f0, d0 = 1, 1, ux0, -self.dx
    if ul is not None:
        an, bn, fn, dn = 0, 1, ul, 1
    if uxl is not None:
        an, bn, fn, dn = 1, 1, uxl, self.dx

    alpha = np.empty(n - 1)
    beta = np.empty(n - 1)
    alpha[0] = c0 / b0
    for i in range(1, n - 1):
        alpha[i] = k / (1 + 2 * k - k * alpha[i - 1])

    for j in range(m - 1):
        beta[0] = d0 * f0(time[j + 1]) / b0
        for i in range(1, n - 1):
            beta[i] = (self.u[i][j] + k * beta[i - 1]) / (1 + 2 * k - k * alpha[i - 1])
        self.u[n - 1][j + 1] = (dn * fn(time[j + 1]) + an * beta[n - 2]) / (bn - an * alpha[n - 2])

        for i in range(n - 2, -1, -1):
            self.u[i][j + 1] = alpha[i] * self.u[i + 1][j + 1] + beta[i]

    return self.u
```

Рисунок 116 – Исходный код решения с помощью неявной схемы

В ходе тестирования программы на различных наборах входных данных решения, полученные с помощью явной и неявной схемы, совпадали. Значительным преимуществом неявного метода является отсутствие ограничений на шаг по времени, что делает его универсальным.

Также была добавлена возможность визуализировать решение как поверхность в трёхмерном пространстве, как показано на рисунке 117.

```
def plot3d(self, u, T: float, dt: float):
    fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
    fig.suptitle(f'Solution of heat equation: temperature = u(x, t)')
    fig.supxlabel(f'a = {self.a}, L = {self.L}, dx = {self.dx}, dt = {dt}, '
                  f'λ = {round(self.a * self.a * dt / self.dx / self.dx, 3)}')
    ax.set_xlabel('time')
    ax.set_ylabel('x')
    ax.set_zlabel('temperature')

    time = np.arange(0, T + dt, dt)
    x_coordinates = np.arange(0, self.L + self.dx, self.dx)
    X, Y = np.meshgrid(time, x_coordinates)
    surface = ax.plot_surface(X, Y, u, cmap=cm.plasma, linewidth=0, rstride=1, cstride=1)

    plt.show()
```

Рисунок 117 – Исходный код метода для трёхмерной визуализации решения

На рисунке 118 демонстрируется пример работы программы.

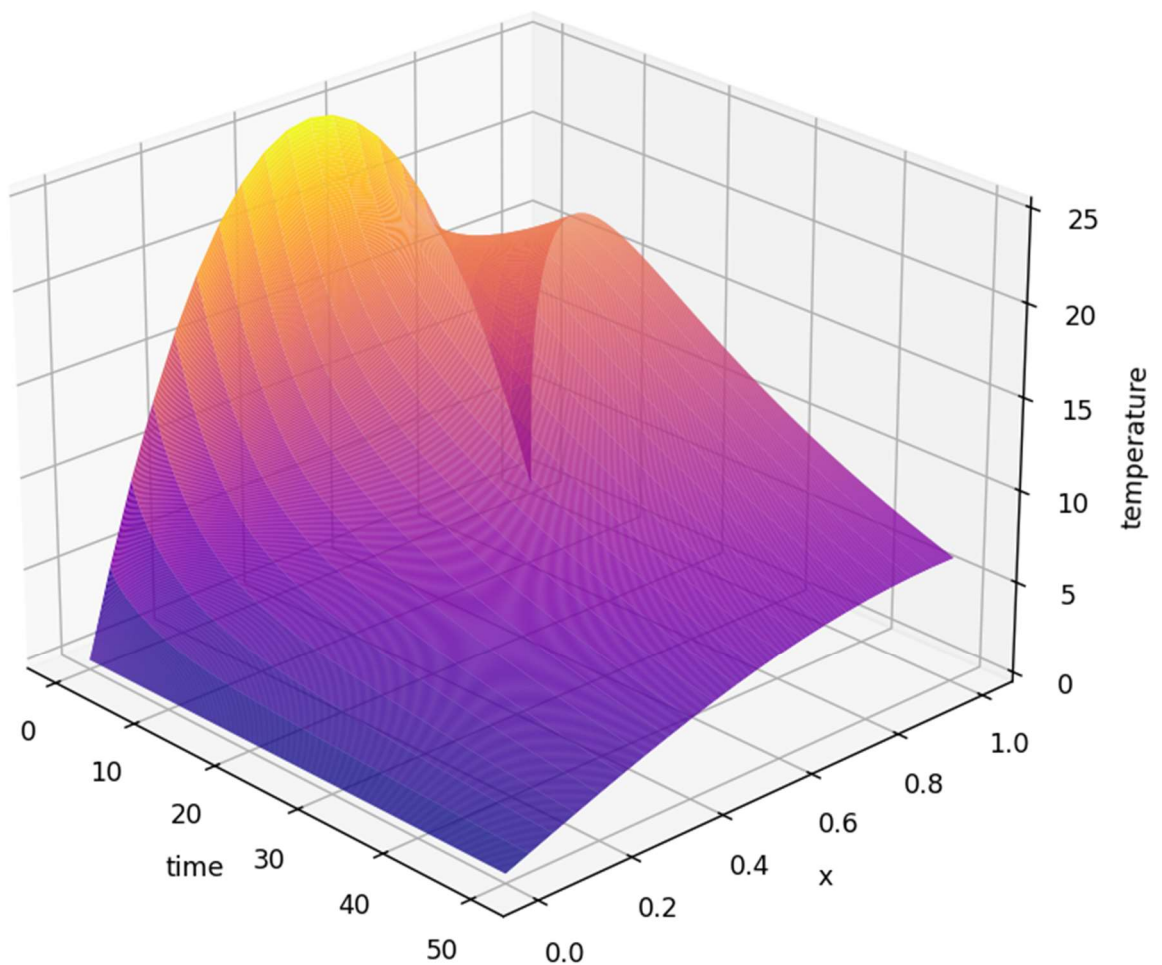


Рисунок 118 – Трёхмерная визуализация решения, полученного для третьего варианта граничных условий с параметрами $a = 0.1$, $h = 0.05$, $\tau = 0.1$

5.2. Уравнение колебания струны

Рассматривается задача моделирования малых колебаний однородной абсолютно гибкой струны с жёстко закреплёнными концами. *Струной* будем называть тонкую туго натянутую упругую нить. В состоянии покоя струна расположена вдоль оси Ox . При разовом возбуждении возникают свободные поперечные колебания, которые происходят в одной плоскости. Сопротивлением среды и потерей энергии системы пренебрегаем.

Схема модели отражена на рисунке 100.

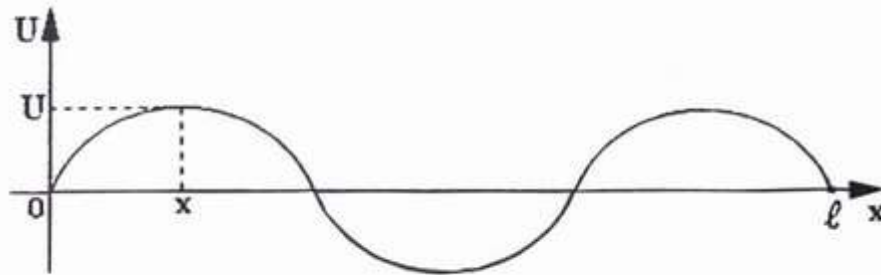


Рисунок 119 – Модель колебания струны

Обозначения и параметры модели:

- l – длина струны;
- T_0 – натяжение струны в состоянии покоя;
- ρ – линейная плотность струны;
- $u(x, t)$ – функция, которая задаёт абсолютное отклонение от положения покоя точки с координатой x в момент времени t ;
- $f(x)$ – функция, которая задаёт начальное отклонение точек;
- $g(x)$ – функция, которая задаёт начальную скорость движения точек.

Временная эволюция системы описывается однородным одномерным волновым уравнением:

$$\frac{\partial^2 u}{\partial t^2} = a^2 \cdot \frac{\partial^2 u}{\partial x^2}, \quad x \in (0, l), \quad t \in (0, T], \quad (103)$$

где $a^2 = \frac{T_0}{\rho}$.

Начальные условия с физической точки зрения задают значения отклонения и скорости движения точек в начальный момент времени ($t = 0$):

$$\begin{cases} u(x, 0) = f(x), \\ \frac{\partial u}{\partial t}(x, 0) = g(x), \end{cases} \quad x \in [0, l] \quad (104)$$

Для рассматриваемого случая струны с жёстко закреплёнными концами записываются граничные условия первого рода:

$$\begin{cases} u(0, t) = 0, \\ u(L, t) = 0, \end{cases} \quad t \in [0, T] \quad (105)$$

Для численного решения волнового уравнения (103) воспользуемся алгоритмом, описанным в главе 5.1.1. Введём в области $\{0 \leq x \leq l, 0 \leq t \leq T\}$ равномерную прямоугольную сетку с шагом $\Delta x = h$ по координате и шагом $\Delta t = \tau$ по времени. Аппроксимируя производные конечными разностями, перепишем уравнение (103) в дискретной форме:

$$\begin{aligned} u(x, t + \Delta t) = & 2 \cdot u(x, t) \cdot \left(1 - \frac{a^2(\Delta t)^2}{(\Delta x)^2}\right) + \\ & + \frac{a^2(\Delta t)^2}{(\Delta x)^2} \cdot [u(x + \Delta x, t) + u(x - \Delta x, t)] - u(x, t - \Delta t) \end{aligned} \quad (106)$$

Таким образом, получаем явную разностную схему с шаблоном из 5 точек, отражённых на рисунке 120.

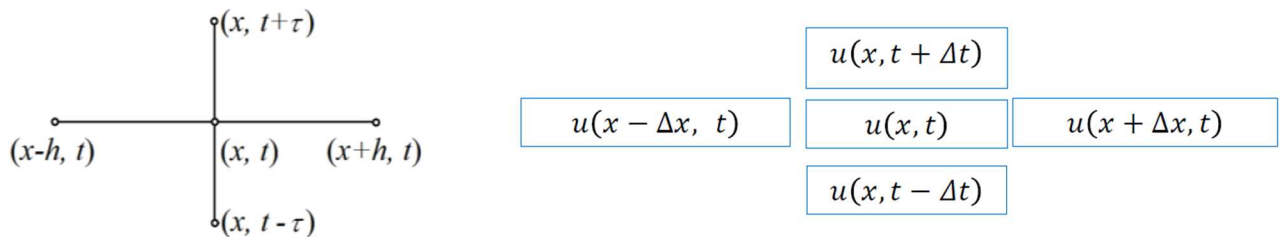


Рисунок 120 – Шаблон явной схемы для уравнения колебания струны

Для устойчивости шаг по временной оси должен удовлетворять условию:

$$\Delta t \leq a \cdot \Delta x \quad (107)$$

Задание

Промоделировать колебания струны, закреплённой на концах отрезка $[0, 1]$, при заданных начальных условиях в Excel и Python.

Варианты начальных условий:

$$\begin{cases} u(x, 0) = x(1 - x), \\ \frac{\partial u}{\partial t}(x, 0) = 0. \end{cases}$$

$$\begin{cases} u(x, 0) = x(x^2 - 1), \\ \frac{\partial u}{\partial t}(x, 0) = 0. \end{cases}$$

$$\begin{cases} u(x, 0) = x(x^2 - 1), \\ \frac{\partial u}{\partial t}(x, 0) = \begin{cases} 3, & \text{если } x \in [0.2, 0.3], \\ -2, & \text{если } x \in [0.7, 0.8], \\ 0, & \text{если } x \in [0, 0.2) \cup (0.3, 0.7) \cup (0.8, 1]. \end{cases} \end{cases}$$

1. Реализация в LibreOffice Calc

В целях упрощения вычислений были выбраны следующие значения параметров модели: $a = 1$, $h = 0.1$, $\tau = 0.1 \Rightarrow \lambda = \frac{a^2 h^2}{\tau^2} = 1$.

На рисунке 121 демонстрируется полученный график для первого случая.

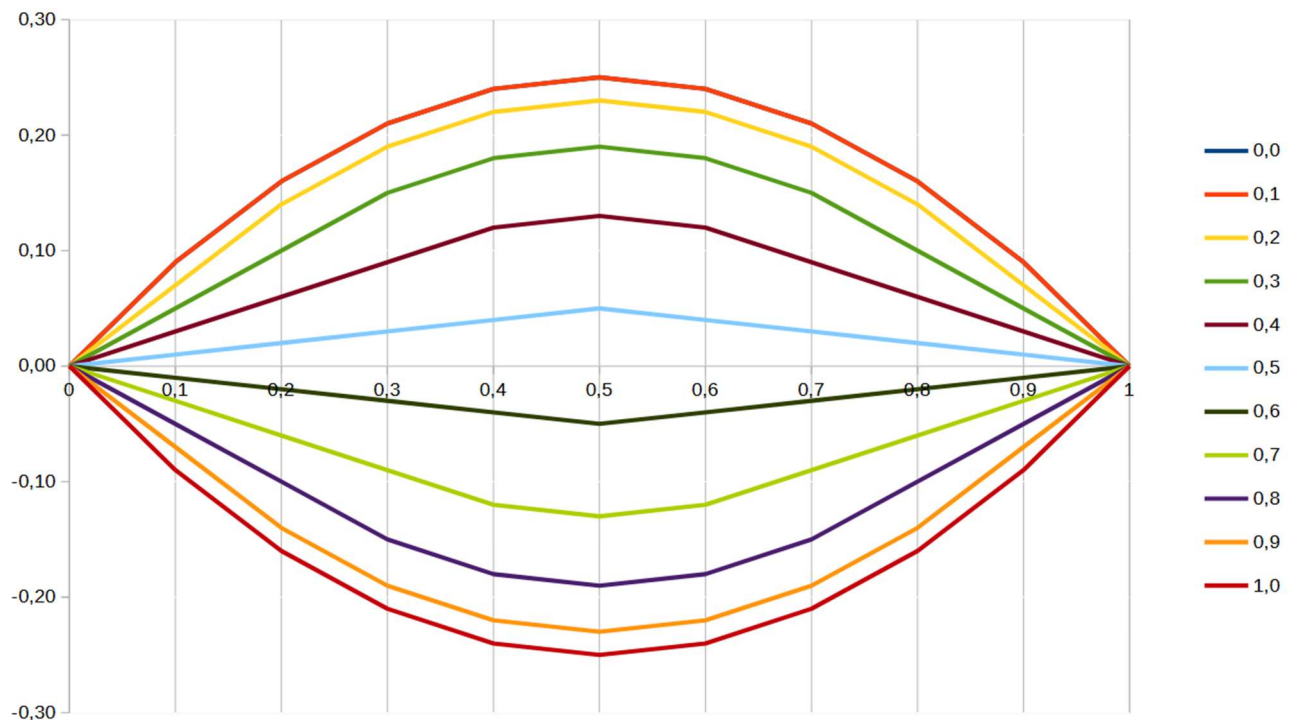


Рисунок 121 – Колебания струны при первом варианте начальных условий

Результаты, полученные с помощью табличных расчётов на втором наборе начальных данных, отражены на рисунке 122.

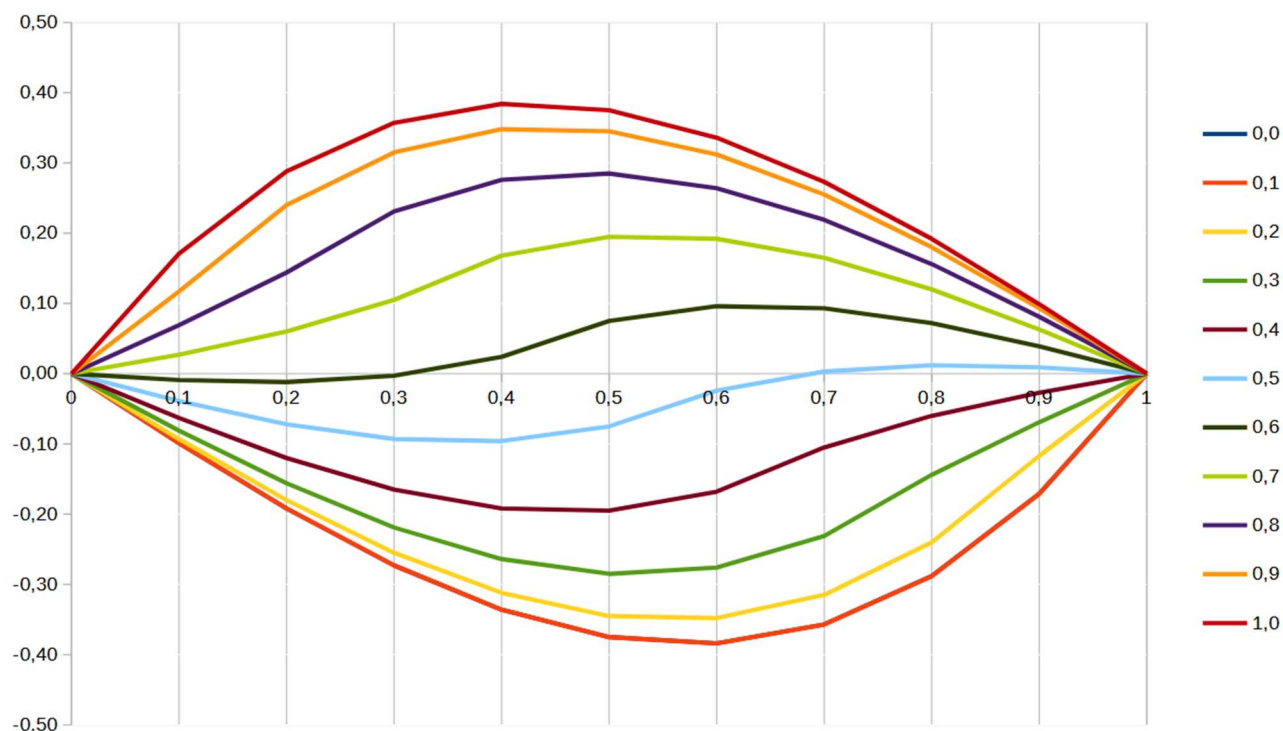


Рисунок 122 – Колебания струны при втором варианте начальных условий

График для третьего набора условий приведён на рисунке 123.

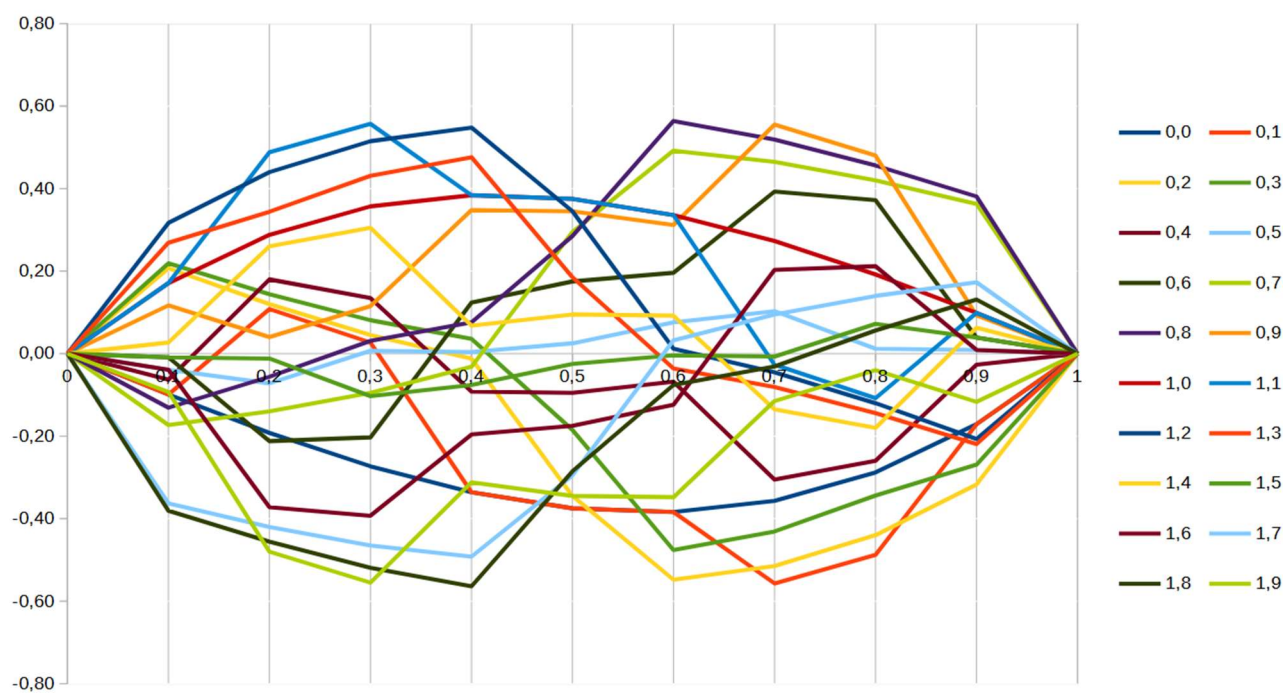


Рисунок 123 – Колебания струны при третьем варианте начальных условий

Можно заметить, что во всех рассмотренных случаях колебания происходят с периодом в 2 секунды.

2. Реализация в Python

Для численного решения уравнения колебания струны (103) с помощью явной схемы (106) при любых допустимых значениях параметров в Python был реализован класс StringVibration. Программный код инициализации объекта и численного решения уравнения приведены на рисунке 124.

```
class StringVibration(object):
    def __init__(self, a: float, L: float, dx: float):
        """
        Setting key params of wave equation  $u''_{tt} = a^2 * u''_{xx}$ 
        :param a: thermal diffusivity coefficient ( $a > 0$ )
        :param L: string length ( $x_{stop}$ )
        :param dx: x-axis step
        """
        self.a = a
        self.L = L
        self.dx = dx
        self.u = None

    def solve_explicit_scheme(self, T: float, dt: float, f, g):
        """
        Finds numerical solution of one-dimensional wave equation by using explicit finite
        difference scheme
        :param T: t_stop
        :param dt: t-axis step
        :param f: function name:  $f(x) = u(x, 0)$ 
        :param g: function name:  $g(x) = du/dt(x, 0)$ 
        :return:  $u[x][t]$  as np.array.
        """
        k = self.a * self.a * dt * dt / self.dx / self.dx
        if k > 1:
            raise ValueError()
        time = np.arange(0, T + dt, dt)
        coordinates = np.arange(0, self.L + self.dx, self.dx)
        m = time.shape[0]
        n = coordinates.shape[0]
        self.u = np.empty([n, m])

        for x in range(n):
            self.u[x][0] = f(coordinates[x])
            self.u[x][1] = self.u[x][0] + dt * g(coordinates[x])
        for t in range(m - 1):
            self.u[0][t + 1] = 0
            for x in range(1, n - 1):
                self.u[x][t + 1] = (2 * (1 - k) * self.u[x][t] +
                                     k * (self.u[x + 1][t] + self.u[x - 1][t]) - self.u[x][t - 1])
            self.u[n - 1][t + 1] = 0

        return self.u
```

Рисунок 124 – Исходный код класса StringVibration

Для визуализации решений были использованы разработаны ранее методы (см. рисунки 108, 117), с помощью которых стало возможным отображать анимацию вибрации струны, а также строить график в трёхмерном пространстве.

На рисунке 125 демонстрируется пример визуализации колебания струны, построенный на втором наборе начальных данных.

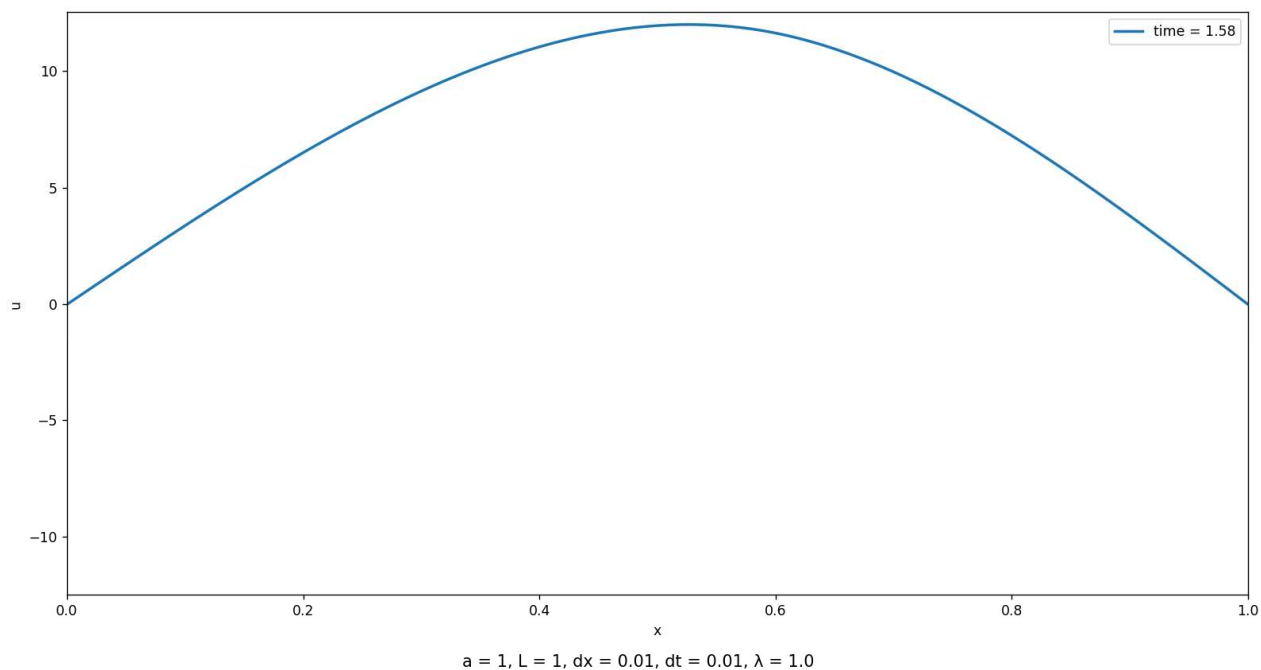


Рисунок 125 – Анимация колебания струны

Трёхмерный график для третьего случая, приведён на рисунке 126.

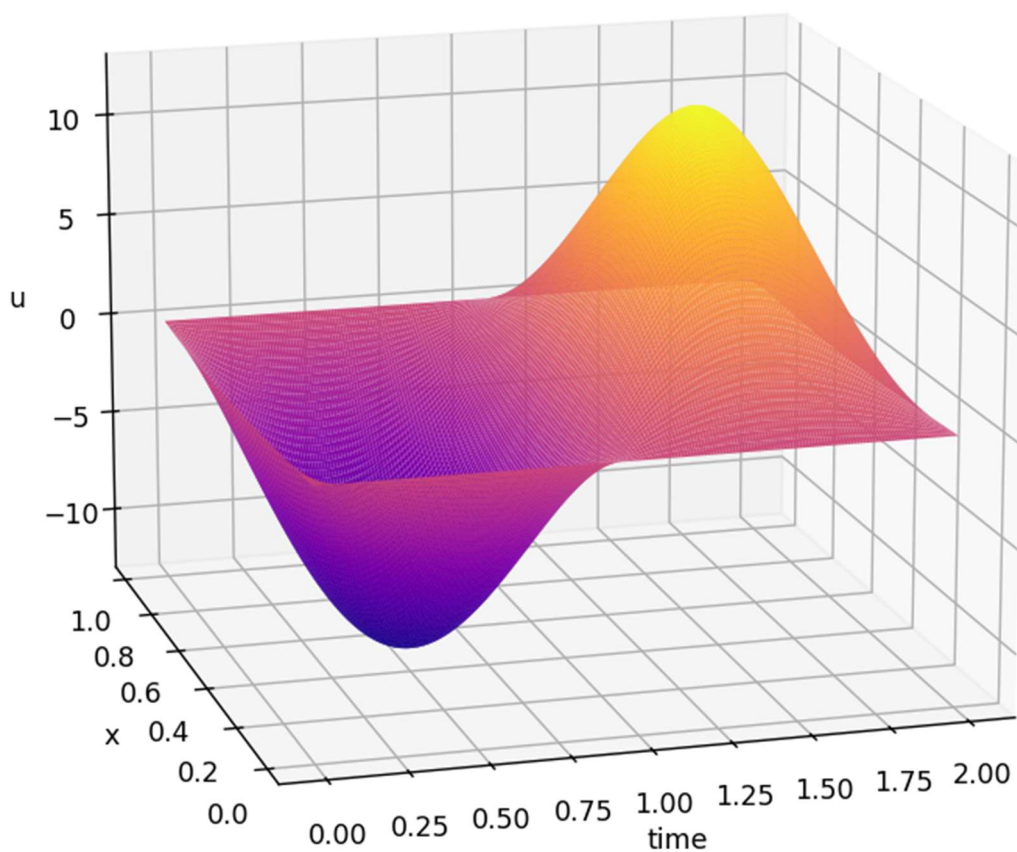


Рисунок 126 – График решения, построенный в трёхмерном пространстве

5.3. Двумерное волновое уравнение. Колебание мембраны

Волновое уравнение является одним из основных уравнений математической физики. Преимущественно используется для описания колебательных процессов в сплошных средах и электродинамике, но также находит применение в других областях теоретической физики. В общем виде представляет собой неоднородное линейное гиперболическое дифференциальное уравнение в частных производных второго порядка:

$$\frac{\partial^2 u}{\partial t^2} = v^2 \cdot \Delta u + f(x_1, \dots, x_n, t), \quad (108)$$

где v – фазовая скорость;

$\Delta u = \frac{\partial^2 u}{\partial x_1^2} + \dots + \frac{\partial^2 u}{\partial x_n^2}$ – n -мерный оператор Лапласа;

f – функция внешнего воздействия.

Уравнение (108) определяет распространение бегущей волны в n -мерной однородной среде со скоростью v под воздействием вынуждающей силы f .

В главе 5.2 одномерный частный случай волнового уравнения (108) применялся для описания колебаний струны, здесь же подробно остановимся на двумерном, который будет рассмотрен на примере колебаний мембраны.

Постановка задачи:

Рассматривается задача описания малых свободных поперечных колебаний однородной мембраны. Под *мембраной* подразумевается натянутая тонкая упругая свободно изгибающаяся плёнка, как правило, закреплённая по периметру. Пусть в положении равновесия мембрана занимает область D , целиком лежащую в плоскости xOy и ограниченную замкнутой непересекающейся кривой (контуром) Γ . Изначально мембрана находится под воздействием равномерного натяжения T . Будем считать, что, будучи выведенной из состояния покоя, мембрана начинает совершать малые поперечные колебания (все её точки движутся перпендикулярно плоскости xOy). Требуется найти смещение точек от положения равновесия как функцию $u(x, y, t)$. Сопротивлением среды и потерей энергии системы пренебрегаем.

Обозначения и параметры модели:

- Γ – контур мембраны;
- D – область, ограниченная контуром Γ ;
- T – равномерное натяжение мембраны в состоянии покоя;
- ρ – поверхностная плотность мембраны;
- $u(x, y, t)$ – искомая функция, которая задаёт абсолютное отклонение от положения покоя точки с координатами (x, y) в момент времени t ;
- $\varphi(x, y)$ – функция, которая задаёт начальное отклонение точек;
- $\psi(x, y)$ – функция, которая задаёт начальную скорость движения точек.

Вывод уравнения:

Рассматривая только малые колебания, будем считать, что искомая функция $u(x, y, t)$, а также её частные производные по x и y малы, так что квадратами и произведениями их можно пренебречь по сравнению с самими этими величинами.

Выделим произвольный участок мембраны площадью σ , ограниченный в положении равновесия контуром γ . В процессе колебаний он деформируется в участок σ' , ограниченный замкнутой пространственной кривой γ' . Нормаль к рассматриваемой поверхности имеет проекции $\left\{-\frac{\partial u}{\partial x}, -\frac{\partial u}{\partial y}, 1\right\}$. Площадь участка σ' в момент времени t выражается формулой:

$$\sigma' = \iint_{\sigma} \sqrt{1 + u_x'^2 + u_y'^2} dx dy \approx \iint_{\sigma} dx dy = \sigma \quad (109)$$

Таким образом, при малых колебаниях изменением площади и, как следствие, сил натяжения можно пренебречь. Это распространяется как на отдельно взятые части, так и на мембрану целиком.

Если в любой момент времени вырезать рассматриваемый участок σ' , то действие отброшенной части следует заменить силами натяжения, распределёнными вдоль контура γ' и лежащими в касательной плоскости к поверхности мембраны по направлению нормалей к контуру γ' . Поскольку мембрана по

условию находится под действием равномерного натяжения, величина силы, приложенной к любому элементу разреза, пропорциональна длине элемента и перпендикулярна его направлению. Найдём проекцию равнодействующей сил натяжения, приложенных к контуру γ' , на ось Ou . Выберем направление обхода контура против часовой стрелки и обозначим через ds' бесконечно малый элемент дуги кривой γ' . Тогда на этот элемент действует сила натяжения величиной $T \cdot ds'$, проекция которой на ось Ou выражается формулой:

$$T \left(-\frac{\partial u}{\partial y} dx + \frac{\partial u}{\partial x} dy \right) \quad (110)$$

Для нахождения проекции равнодействующей сил натяжения, выражение (110) требуется проинтегрировать по контуру γ' .

$$T \oint_{\gamma'} \left(-\frac{\partial u}{\partial y} dx + \frac{\partial u}{\partial x} dy \right) ds' \quad (111)$$

При малых колебаниях можем считать $ds' \approx ds$ и заменить путь интегрирования γ' на γ .

$$T \oint_{\gamma} \left(-\frac{\partial u}{\partial y} dx + \frac{\partial u}{\partial x} dy \right) ds \quad (112)$$

К полученному интегралу применима теорема Грина:

$$T \iint_{\sigma} \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) dx dy \quad (113)$$

Предположим, что на мембрану параллельно оси Ou действует внешняя сила $p(x, y, t)$, рассчитанная на единицу площади. Тогда проекция на ось Ou внешней силы, действующей на рассматриваемый участок σ' , равна

$$\iint_{\sigma} p(x, y, t) dx dy \quad (114)$$

Считая участок малым, его можно рассматривать как материальную точку. Силы (113) и (114) в любой момент времени должны уравниваться силами инерции данного участка мембраны.

$$\iint_{\sigma} \rho(x, y) \frac{\partial^2 u}{\partial t^2} dx dy \quad (115)$$

где $\rho(x, y)$ – поверхностная плотность мембраны.

Применим к участку σ' II закон Ньютона в форме проекций на ось Ou и подставим найденные выражения (113)-(115).

$$\iint_{\sigma} \rho(x, y) \frac{\partial^2 u}{\partial t^2} dx dy = T \iint_{\sigma} \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) dx dy + \iint_{\sigma} p(x, y, t) dx dy \quad (116)$$

В результате преобразований получим:

$$\iint_{\sigma} \left[\rho(x, y) \frac{\partial^2 u}{\partial t^2} - T \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) - p(x, y, t) \right] dx dy = 0 \quad (117)$$

В силу произвольности участка мембраны σ' от интеграла можно избавиться.

$$\rho(x, y) \frac{\partial^2 u}{\partial t^2} = T \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) + p(x, y, t) \quad (118)$$

Полученное уравнение (118) описывает вынужденные колебания неоднородной мембраны. При $\rho(x, y) = const$ оно преобразуется в двумерное неоднородное волновое уравнение:

$$\frac{\partial^2 u}{\partial t^2} = a^2 \cdot \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) + f(x, y, t), \quad (119)$$

где $a^2 = \frac{T}{\rho}$;

$f(x, y, t) = \frac{p(x, y, t)}{\rho}$.

В отсутствие внешнего воздействия $f(x, y, t) = 0$ получаем искомое *уравнение свободных колебаний однородной мембраны*:

$$\frac{\partial^2 u}{\partial t^2} = a^2 \cdot \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \quad (120)$$

При фиксированных x и y функция $u(x, y, t)$ задаёт закон колебания точки мембраны с координатами (x, y) , при этом частные производные $\frac{\partial u}{\partial t}$ и $\frac{\partial^2 u}{\partial t^2}$ определяют скорость и ускорение движущейся точки соответственно. При фиксированном t поверхность $u = u(x, y, t)$ представляет форму колеблющейся мембраны в момент времени t .

Начальные условия с физической точки зрения задают значения отклонения и скорости движения точек мембраны в момент времени $t = 0$:

$$\begin{cases} u(x, y, 0) = \varphi(x, y), \\ \frac{\partial u}{\partial t}(x, y, 0) = \psi(x, y), \end{cases} \quad (x, y) \in D \quad (121)$$

В зависимости от условия задачи для точек контура мембраны записываются краевые условия первого, второго или третьего рода.

Функциональное условие *первого рода* определяет значения на границе:

$$u(x, y, t) = g(x, y, t), \quad (x, y) \in \Gamma, \quad t \in (0, T] \quad (122)$$

Дифференциальное краевое условие *второго рода* задаётся через производную в направлении внешней нормали на границе:

$$\frac{\partial u}{\partial n}(x, y, t) = g(x, y, t), \quad (x, y) \in \Gamma, \quad t \in (0, T], \quad (123)$$

где n – внешняя нормаль.

Функционально-дифференциальное граничное условие *третьего рода* задаётся через линейную комбинацию производной в направлении внешней нормали и значения функции на границе:

$$\alpha \cdot \frac{\partial u}{\partial n}(x, y, t) + \beta \cdot u(x, y, t) = g(x, y, t), \quad (x, y) \in \Gamma, \quad t \in (0, T] \quad (124)$$

где α, β – заданные числа, $\alpha^2 + \beta^2 > 0$.

Стоит отметить, что начально-краевые задачи для волнового уравнения в n -мерном пространстве (108) формулируются аналогичным образом.

Для рассматриваемого случая жёстко закреплённой по периметру мембраны записываются следующие граничные условия первого рода:

$$u(x, y, t) = 0, \quad (x, y) \in \Gamma, \quad t \in [0, T] \quad (125)$$

Аналитическое решение уравнения (119) с заданными начальными условиями (121) задаётся формулой Пуассона-Парсеваля, которая не слишком удобна для использования. Вместо этого прибегнем к нахождению численного решения с помощью метода конечных разностей. Пусть G – минимальная прямоугольная область в плоскости xOy такая, что $D \subseteq G$. Не умаляя общности, предположим, что мембрана лежит в первой четверти плоскости xOy и её граница касается осей Ox и Oy . Тогда $G = \left\{ [0, L] \times [0, M] \mid L = \max_x D, M = \max_y D \right\}$. Введём в G равномерную прямоугольную сетку с шагами $\Delta x = h_x$ и $\Delta y = h_y$ по координатам и $\Delta t = \tau$ по времени. Перейдём в уравнении (120) от частных производных к конечным разностям:

$$\begin{aligned} & \frac{u(x, y, t + \Delta t) - 2 \cdot u(x, y, t) + u(x, y, t - \Delta t)}{(\Delta t)^2} = \\ & = a^2 \cdot \frac{u(x + \Delta x, y, t) - 2 \cdot u(x, y, t) + u(x - \Delta x, y, t)}{(\Delta x)^2} + \\ & + a^2 \cdot \frac{u(x, y + \Delta y, t) - 2 \cdot u(x, y, t) + u(x, y - \Delta y, t)}{(\Delta y)^2} \end{aligned} \quad (126)$$

Выражая отсюда $u(x, y, t + \Delta t)$, получаем условно устойчивую явную разностную схему, имеющую второй порядок аппроксимации.

$$\begin{aligned}
 u(x, y, t + \Delta t) = & -u(x, y, t - \Delta t) + \\
 & + 2 \cdot \left(1 - ((\Delta x)^2 + (\Delta y)^2) \cdot \left(\frac{a \cdot \Delta t}{\Delta x \cdot \Delta y} \right)^2 \right) \cdot u(x, y, t) + \\
 & + \left(\frac{a \cdot \Delta t}{\Delta x} \right)^2 \cdot [u(x + \Delta x, y, t) + u(x - \Delta x, y, t)] + \\
 & + \left(\frac{a \cdot \Delta t}{\Delta y} \right)^2 \cdot [u(x, y + \Delta y, t) + u(x, y - \Delta y, t)]
 \end{aligned} \tag{127}$$

Схематичное изображение полученной трёхслойной схемы с шаблоном из 7 точек приведено на рисунке 127.

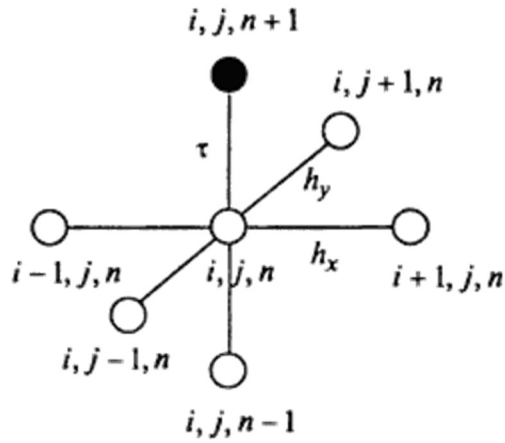


Рисунок 127 – Шаблон явной разностной схемы для однородного двумерного волнового уравнения (120)

Схема (127) является устойчивой, если шаг по времени удовлетворяет условию:

$$\Delta t \leq \frac{\Delta x \cdot \Delta y}{a \cdot \sqrt{(\Delta x)^2 + (\Delta y)^2}} \tag{128}$$

Для решения уравнения (120) можно также построить неявную схему с шаблоном из 15 точек, отражённых на рисунке 128, однако соответствующие расчёты и формулы здесь приведены не будут.

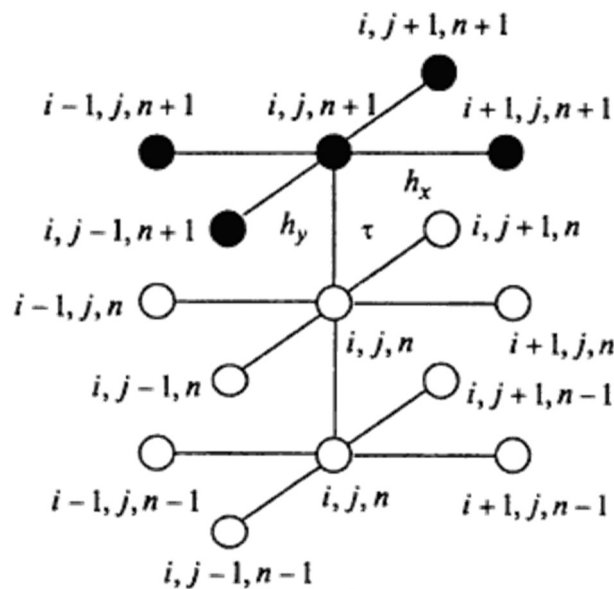


Рисунок 128 – Шаблон неявной разностной схемы для однородного двумерного волнового уравнения (120)

Реализация в Python

Программная реализация колебаний прямоугольной мембраны в Python была осуществлена с помощью класса `MembraneVibration`, атрибуты которого характеризуют параметры мембраны $a = \sqrt{\frac{T}{\rho}}$, L , M , а также задают шаги Δx , Δy по координатным осям. Исходный код инициализации приведён на рисунке 129.

```
class MembraneVibration(object):
    def __init__(self, a: float, L: float, dx: float, M: float, dy: float):
        """
        Setting key params of wave equation  $u''_{tt} = a^2 * (u''_{xx} + u''_{yy})$ 
        :param a: membrane parameter:  $a^2 = T/\rho$ ,  $a > 0$ 
        :param L: x_max
        :param dx: x-axis step
        :param M: y_max
        :param dy: y-axis step
        """
        self.a = a
        self.L = L
        self.dx = dx
        self.M = M
        self.dy = dy
```

Рисунок 129 – Инициализация объекта класса `MembraneVibration`

Решение волнового уравнения (120) с помощью явной разностной схемы (127) было оформлено как метод данного класса, представленный на рисунке 130.

```
def solve_explicit_scheme(self, T: float, dt: float, phi, psi, g):
    """
    Finds numerical solution of two-dimensional wave equation by using explicit finite difference scheme
    :param T: t_stop
    :param dt: t-axis step
    :param phi: function name:  $\varphi(x, y) = u(x, y, 0)$ 
    :param psi: function name:  $\psi(x, y) = du/dt(x, y, 0)$ 
    :param g: function name:  $g(x, y, t) = u(x, y, t)$  if  $(x, y) \in \Gamma$ 
    :return:  $u[x][y][t]$  as np.array.
    """
    k = (self.dx * self.dx + self.dy * self.dy) * (self.a * dt / self.dx / self.dy) ** 2
    if k > 1:
        raise ValueError()
    time = np.arange(0, T + dt, dt)
    x_coordinates = np.arange(0, self.L + self.dx, self.dx)
    y_coordinates = np.arange(0, self.M + self.dy, self.dy)
    p = time.shape[0]
    m = x_coordinates.shape[0]
    n = y_coordinates.shape[0]
    u = np.empty([m, n, p])
    # Начальные условия ( $t = 0$ )
    for x in range(m):
        for y in range(n):
            u[x][y][0] = phi(x_coordinates[x], y_coordinates[y])
            u[x][y][1] = u[x][y][0] + dt * psi(x_coordinates[x], y_coordinates[y])
    # Граничные условия 1 рода
    for t in range(1, p):
        for x in range(1, m - 1):
            u[x][0][t] = g(x_coordinates[x], 0, t)
            u[x][n - 1][t] = g(x_coordinates[x], self.M, t)
        for y in range(1, n - 1):
            u[0][y][t] = g(0, y_coordinates[y], t)
            u[m - 1][y][t] = g(self.L, y_coordinates[y], t)
    # Послойное заполнение
    for t in range(1, p - 1):
        for x in range(1, m - 1):
            for y in range(1, n - 1):
                u[x][y][t + 1] = (-u[x][y][t - 1] + 2 * (1 - k) * u[x][y][t] +
                                   ((self.a * dt / self.dx) ** 2) * (u[x + 1][y][t] + u[x - 1][y][t]) +
                                   ((self.a * dt / self.dy) ** 2) * (u[x][y + 1][t] + u[x][y - 1][t]))
    return u
```

Рисунок 130 – Реализация явной разностной схемы (127) в Python

Анимированная 3D-визуализация колебаний мембраны была выполнена с помощью соответствующих средств библиотеки matplotlib, как показано на рисунке 131.

```

def plot3Danimation(self, u, dt: float, speed=1.):
    """
    Visualization of membrane vibration
    :param u: u(x, y, t) as np.array.
    :param dt: t-axis step
    :param speed: animation speed coefficient
    """
    k = (self.dx * self.dx + self.dy * self.dy) * ((self.a * dt / self.dx / self.dy) ** 2)
    x_coordinates = np.arange(0, self.L + self.dx, self.dx)
    y_coordinates = np.arange(0, self.M + self.dy, self.dy)
    fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
    fig.suptitle(f'Membrane vibration')
    fig.supxlabel(f'a = {self.a}, L = {self.L}, dx = {self.dx}, M = {self.M}, dy = {self.dy}, '
                  f'dt = {dt}, k = {round(k, 3)}')
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('u')
    ax.set_xlim([0, self.L])
    ax.set_ylim([0, self.M])
    ax.set_zlim([np.min(u), np.max(u)])
    plot_args = {'cmap': 'plasma', 'linewidth': 0, 'rstride': 1, 'cstride': 1}
    X, Y = np.meshgrid(x_coordinates, y_coordinates)
    surface = ax.plot_surface(X, Y, u[:, :, 0], label='time = 0', **plot_args)
    dp = Decimal(str(dt)).as_tuple().exponent * (-1)

    def animate(i):
        old_surface = [c for c in ax.collections if isinstance(c, Poly3DCollection)][0]
        old_surface.remove()
        surface = ax.plot_surface(X, Y, u[:, :, i], label=f'time = {dt * i:.{dp}f}', **plot_args)
        ax.legend()
        return surface,

    ani = animation.FuncAnimation(fig, animate, repeat=True, frames=u.shape[2],
                                  interval=round(1 / speed))

    plt.show()

```

Рисунок 131 – Программный код 3D-анимации колебания мембраны

Рисунок 132 отражает колебание мембраны при начальных условиях $\varphi(x, y) = 10 \cdot x \cdot (1 - x) \cdot y \cdot (1 - y)$, $\psi(x, y) = 0$.

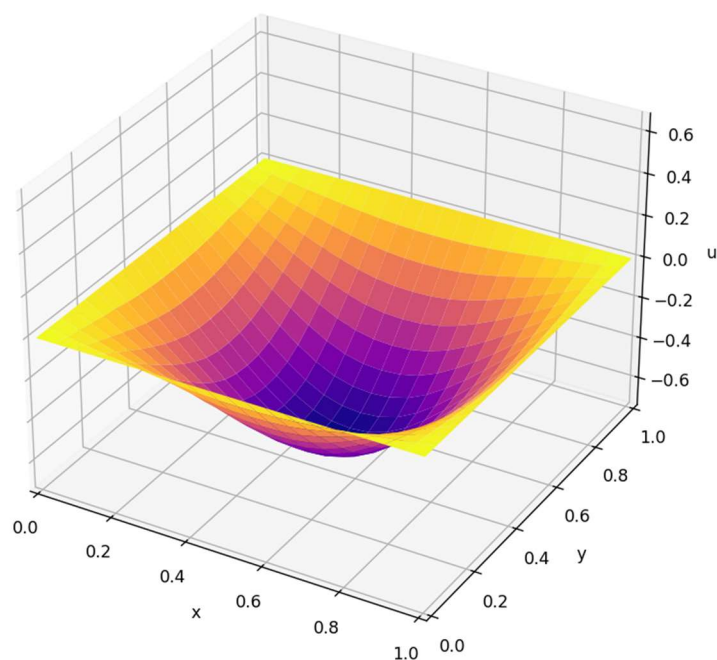


Рисунок 132 – Запуск программы на первом наборе начальных данных

Другие любопытные результаты тестирования программы приведены на рисунках 133 и 134.

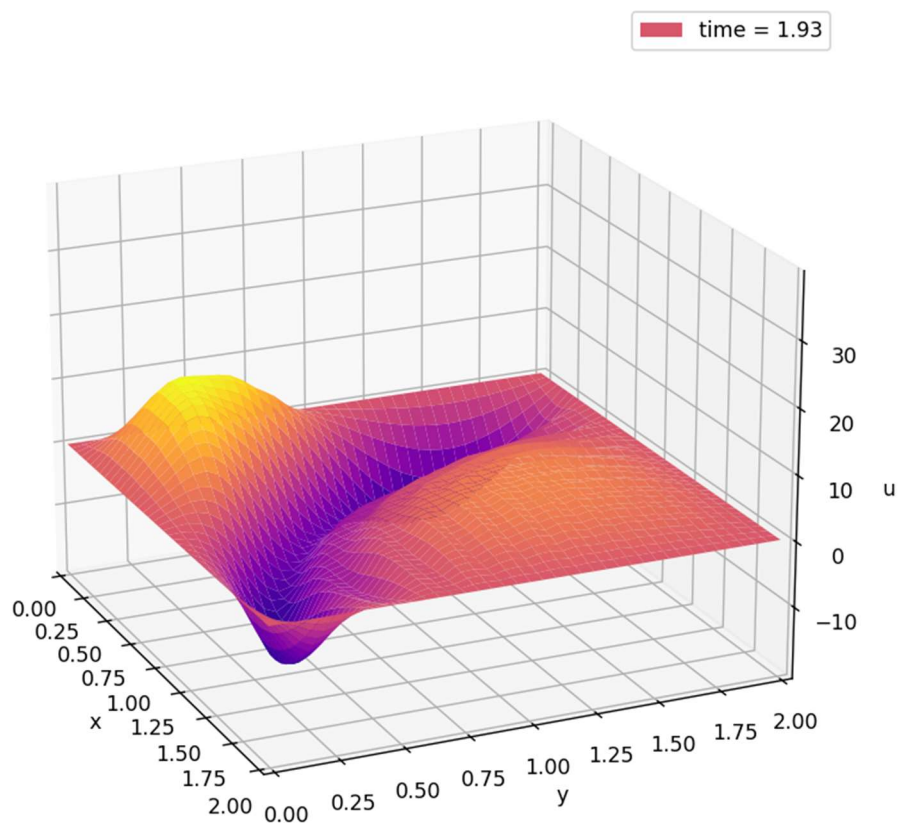


Рисунок 133 – Запуск программы на втором наборе начальных данных

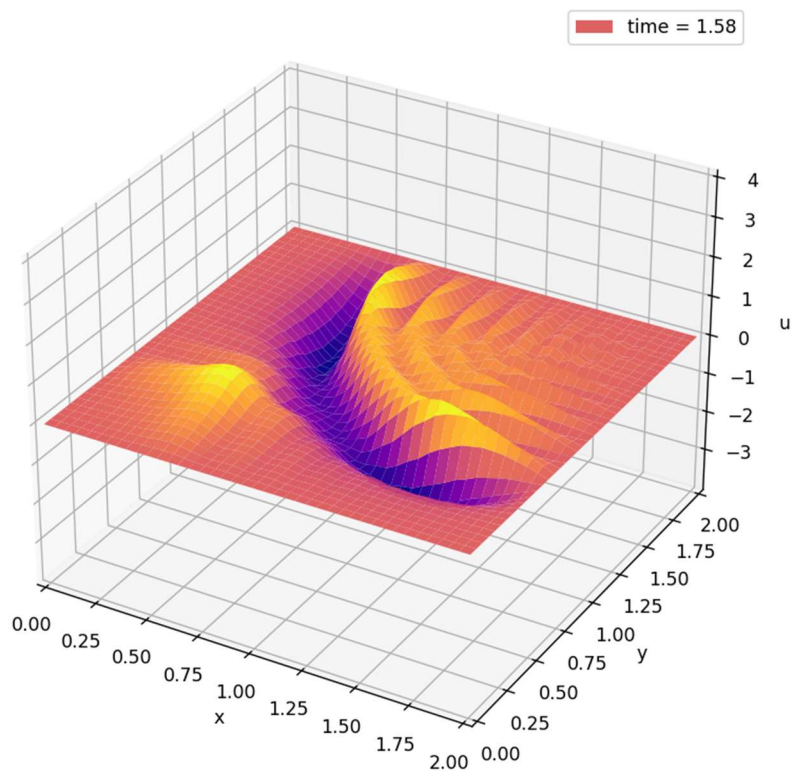


Рисунок 134 – Запуск программы на третьем наборе начальных данных

Реализация в Wolfram Mathematica

Преимуществом использования данной программной среды является отсутствие необходимости ручного численного решения уравнения в частных производных. На рисунке отражена анимированная визуализация колебания квадратной мембраны, полученная в Wolfram Mathematica.

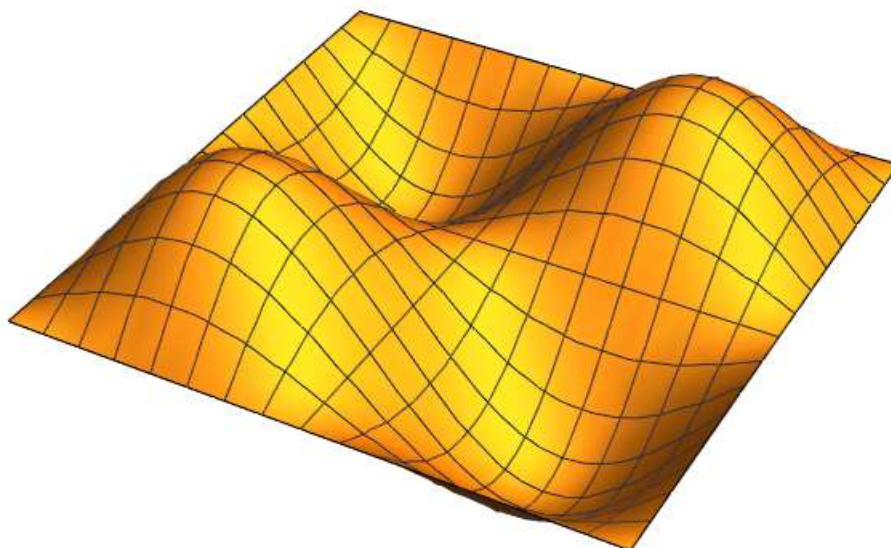


Рисунок 135 – Визуализация в Wolfram Mathematica

Соответствующая визуализация для круглой мембраны требует нетривиального задания краевых условий. На рисунке демонстрируется пример, визуализирующий искривление поверхности мембраны в результате резкого удара.

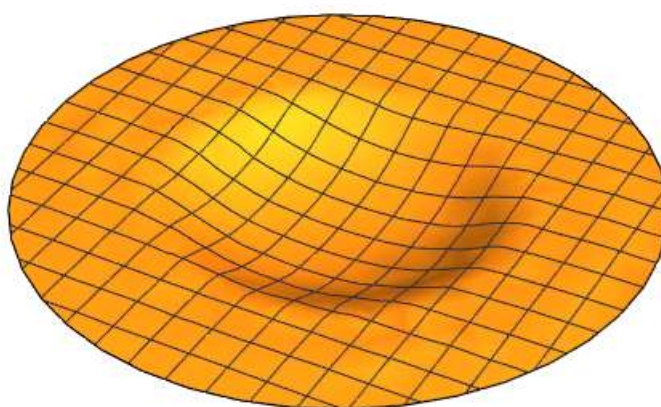


Рисунок 136 – Удар по круглой мембране

6. ОПТИМИЗАЦИОННЫЕ МОДЕЛИ

6.1. Оптимальный план производства

Описание кейса

Менеджер фирмы хочет установить оптимальный план производства пяти типов продукции. Информация, необходимая для принятия решения, приведена в таблице на рисунке 137.

	На единицу продукции		
	Сборка (часов)	Складские площади (кв.м.)	Прибыль (\$)
Тип А	2.5	6	100
Тип В	4	8	150
Тип С	5	8	170
Тип D	3	9.5	180
Тип E	3.5	9	160

Рисунок 137 – Исходная таблица

Общее количество доступного ресурса рабочего времени – 680 часов. Складские площади ограничены 1500 кв.м.

Вопросы и задачи

1. Решите, какое количество каждого типа продукции нужно произвести, чтобы максимизировать прибыль. Все ли типы моделей выгодно производить?

2. Какое количество каждого типа продукции нужно произвести, чтобы максимизировать прибыль, если имеются затраты на наладку оборудования в количестве: 200\$ для типа А, 500\$ для типа В, 1000\$ для С, 1400\$ для типа D, 900\$ для E. Сколько моделей продукции теперь выгодно производить?

3. Как изменится оптимальный план и количество производимых типов продукции, если складские площади

а) увеличить на 30 кв. м.?

б) уменьшить на 30 кв. м.?

4. Какой ресурс оказывается лимитирующим в каждом из этих двух случаев?

Решение

Первым делом следует формализовать задачу. Для этого требуется ввести обозначения и составить целевую функцию.

Обозначения:

- x_i – объём выпуска i -й модели;
- t_i – время сборки единицы i -й модели;
- s_i – складские площади, требуемые для производства единицы i -й модели;
- p_i – прибыль от продажи единицы i -й модели;
- c_i – начальные вложения для производства i -й модели.

Целевая функция прибыли имеет вид:

$$f(x_1, x_2, x_3, x_4, x_5) = \sum_{i=1}^5 p_i x_i \rightarrow \max \quad (129)$$

Ограничения из условия задачи:

$$\sum_{i=1}^5 t_i x_i \leq 680 \quad (130)$$

$$\sum_{i=1}^5 s_i x_i \leq 1500 \quad (131)$$

Естественные ограничения:

$$x_i \geq 0 \quad (132)$$

Формулы (129) - (132) представляют собой формальную постановку задачи линейной оптимизации.

Поиск оптимального решения был произведён с помощью симплекс-метода в Excel, а также с использованием встроенных функций системы Wolfram Mathematica. Нахождение наиболее прибыльного плана производства в Excel демонстрируется на рисунке 138.

	На единицу продукции			Оптимальный план		
	Сборка (ч)	Складские помещения (кв.м.)	Прибыль (\$)	Количество (ед.)	Время сборки товаров (ч)	Площадь для сборки (кв.м.)
Тип А	2,5	6,0	100	0	0	0
Тип В	4,0	8,0	150	0	0	0
Тип С	5,0	8,0	170	83	415	664
Тип D	3,0	9,5	180	88	264	836
Тип E	3,5	9,0	160	0	0	0
Ограничения					680	1500
Целевая функция			29950		679	1500

Рисунок 138 – Решение задачи 1 в Excel

На рисунке 139 отражено получение аналогичного результата с помощью функции FindMaximum в Wolfram Mathematica.

```
FindMaximum[
{100 x1 + 150 x2 + 170 x3 + 180 x4 + 160 x5,
 2.5 x1 + 4 x2 + 5 x3 + 3 x4 + 3.5 x5 <= 680 && 6 x1 + 8 x2 + 8 x3 + 9.5 x4 + 9 x5 <= 1500 &&
 x1 >= 0 && x2 >= 0 && x3 >= 0 && x4 >= 0 && x5 >= 0 &&
 x1 ∈ Integers && x2 ∈ Integers && x3 ∈ Integers && x4 ∈ Integers && x5 ∈ Integers},
{x1, x2, x3, x4, x5}
]
{29950., {x1 → 0, x2 → 0, x3 → 83, x4 → 88, x5 → 0}}
```

Рисунок 139 – Решение задачи 1 в Wolfram Mathematica

Можно сделать вывод, что при заданных условиях наиболее выгодно производить только модели С и D в количествах 83 и 88 соответственно.

При наличии начальных затрат на наладку оборудования целевая функция прибыли примет вид:

$$f(x_1, x_2, x_3, x_4, x_5) = \sum_{i=1}^5 p_i x_i - \delta_i c_i \rightarrow \max \quad (133)$$

$$\delta_i = \begin{cases} 1, & \text{если } x_i > 0, \\ 0, & \text{если } x_i = 0. \end{cases} \quad (134)$$

Ограничения останутся неизменными. Решение задачи 2 (130) - (134) в Excel приведено на рисунке 140.

	На единицу продукции			Оптимальный план				
	Сборка (ч)	Складские помещения (кв.м.)	Прибыль (\$)	Количество (ед.)	Время сборки товаров (ч)	Площадь для сборки (кв.м.)	Начальные вложения (\$)	Итоговые вложения (\$)
Тип А	2,5	6,0	100	0	0	0	200	0
Тип В	4,0	8,0	150	0	0	0	500	0
Тип С	5,0	8,0	170	83	415	664	1000	1000
Тип D	3,0	9,5	180	88	264	836	1400	1400
Тип E	3,5	9,0	160	0	0	0	900	0
Ограничения					680	1500		
Целевая функция					679	1500	4000	2400
					25950			
					27550			

Рисунок 140 – Решение задачи 2 в Excel

При увеличении доступных складских площадей на 30 кв. м. решение задачи изменится, как показано на рисунке 141.

	На единицу продукции			Оптимальный план				
	Сборка (ч)	Складские помещения (кв.м.)	Прибыль (\$)	Количество (ед.)	Время сборки товаров (ч)	Площадь для сборки (кв.м.)		
Тип А	2,5	6,0	100	0	0	0		
Тип В	4,0	8,0	150	0	0	0		
Тип С	5,0	8,0	170	79	395	632		
Тип D	3,0	9,5	180	94	282	893		
Тип E	3,5	9,0	160	0	0	0		
Ограничения					680	1530		
Целевая функция					677	1525		
					30350			

Рисунок 141 – Решение задачи 3.1 в Excel

Это подтверждают результаты, полученные в Wolfram Mathematica, что видно из рисунка 142.

```
FindMaximum[
  {100 x1 + 150 x2 + 170 x3 + 180 x4 + 160 x5,
   2.5 x1 + 4 x2 + 5 x3 + 3 x4 + 3.5 x5 ≤ 680 && 6 x1 + 8 x2 + 8 x3 + 9.5 x4 + 9 x5 ≤ 1530 &&
   x1 ≥ 0 && x2 ≥ 0 && x3 ≥ 0 && x4 ≥ 0 && x5 ≥ 0 &&
   x1 ∈ Integers && x2 ∈ Integers && x3 ∈ Integers && x4 ∈ Integers && x5 ∈ Integers},
  {x1, x2, x3, x4, x5}
]
{30400., {x1 → 1, x2 → 2, x3 → 78, x4 → 93, x5 → 0}}
```

Рисунок 142 – Решение задачи 3.1 в Wolfram Mathematica

Стоит отметить, что оба ресурса (время сборки и складские площади) оказываются исчерпаны не полностью, однако остатков недостаточно для производства какой-либо модели.

На рисунке **Ошибка! Источник ссылки не найден.** представлено решение задачи при уменьшении складских помещений на 30 кв. м.

	На единицу продукции			Оптимальный план		
	Сборка (ч)	Складские помещения (кв.м.)	Прибыль (\$)	Количество (ед.)	Время сборки товаров (ч)	Площадь для сборки (кв.м.)
Тип А	2,5	6,0	100	0	0	0
Тип В	4,0	8,0	150	0	0	0
Тип С	5,0	8,0	170	88	440	704
Тип D	3,0	9,5	180	80	240	760
Тип Е	3,5	9,0	160	0	0	0
Ограничения					680	1470
Целевая функция					680	1464
					29360	

Рисунок 143 – Решение задачи 3.2 в Excel

Подтверждение результатов, полученных с помощью Wolfram Mathematica, приведено на рисунке **Ошибка! Источник ссылки не найден..**

```
FindMaximum[
  {100 x1 + 150 x2 + 170 x3 + 180 x4 + 160 x5,
    2.5 x1 + 4 x2 + 5 x3 + 3 x4 + 3.5 x5 ≤ 680 && 6 x1 + 8 x2 + 8 x3 + 9.5 x4 + 9 x5 ≤ 1470 &&
    x1 ≥ 0 && x2 ≥ 0 && x3 ≥ 0 && x4 ≥ 0 && x5 ≥ 0 &&
    x1 ∈ Integers && x2 ∈ Integers && x3 ∈ Integers && x4 ∈ Integers && x5 ∈ Integers},
  {x1, x2, x3, x4, x5}
]
{29400., {x1 → 0, x2 → 0, x3 → 84, x4 → 84, x5 → 0}}
```

Рисунок 144 – Решение задачи 3.2 в Wolfram Mathematica

В данном случае время сборки оказалось лимитирующим (дефицитным) ресурсом.